M&T z/OS COBOL Application Programming

Dave Nicolette
Programmer

neopragma.com

Course introduction

This is a practical introductory course to prepare you for entry-level work developing and supporting applications in COBOL on the IBM z/OS platform.

Course summary

The course covers the basics of several skills you will apply in working with z/OS COBOL applications.

- **Overview of z/OS**
- **The traditional UI: TSO and ISPF**
- **Job Control Language (JCL)**
- **z/OS Utilities**
- **z/OS Data Sets**
- **COBOL fundamentals**
- **Batch COBOL programming**
- **CICS overview**
- **CICS COBOL programming**

Class logistics

Dave Nicolette
Programmer

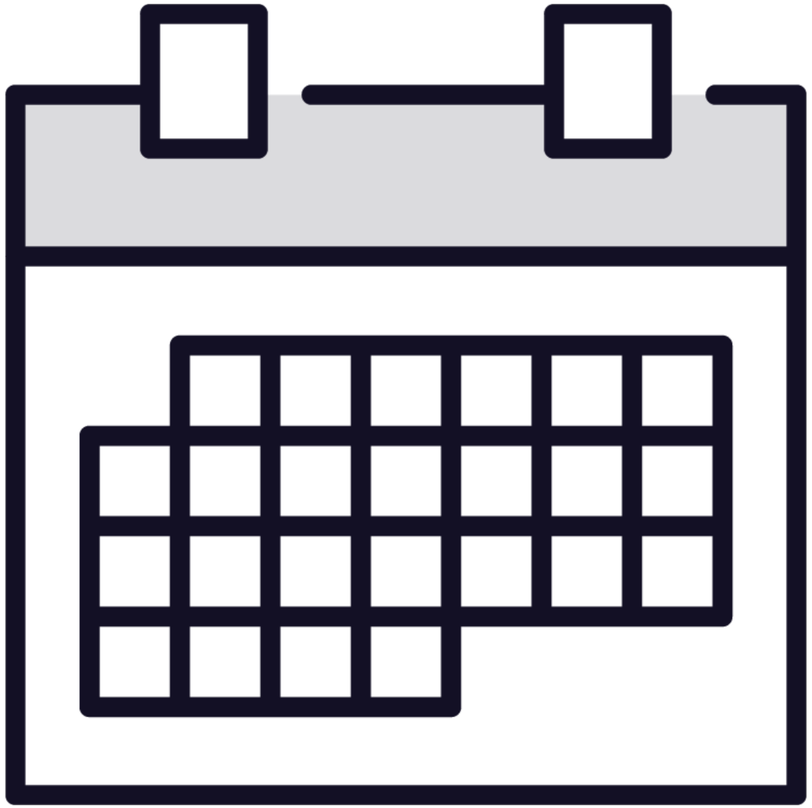
neopragma.com

- Dates
- Start and end times
- Lunch and breaks
- Questions
- Collaboration

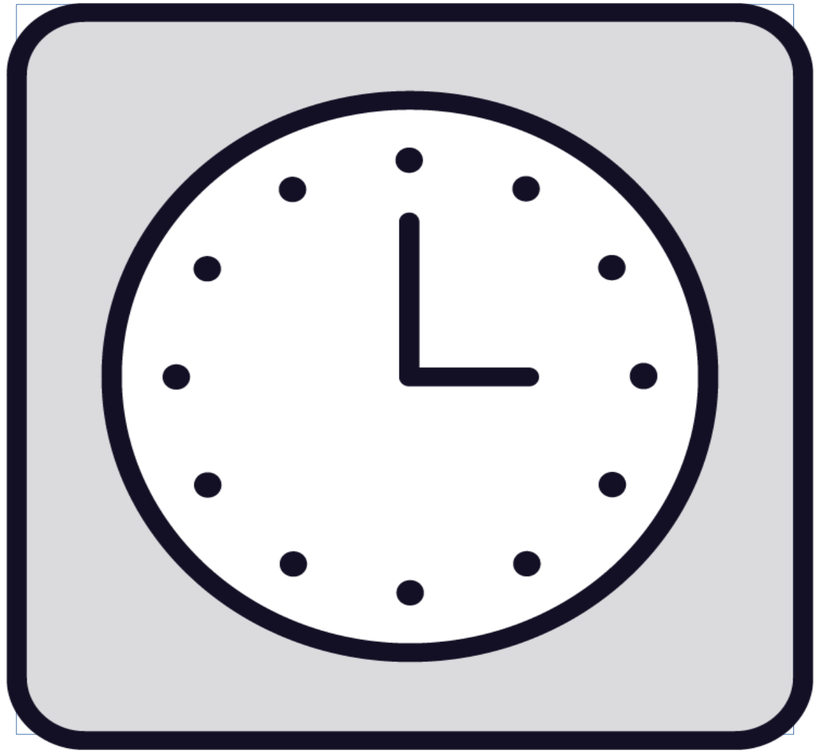
October 19-23

October 26-29

September 1



9:30 a.m. – 5:00 p.m. Eastern Time



Interrupt at any time

Ask questions and make observations

Don't hesitate or wait

Work together

Share information

Help each other

This is a class, but it is not school

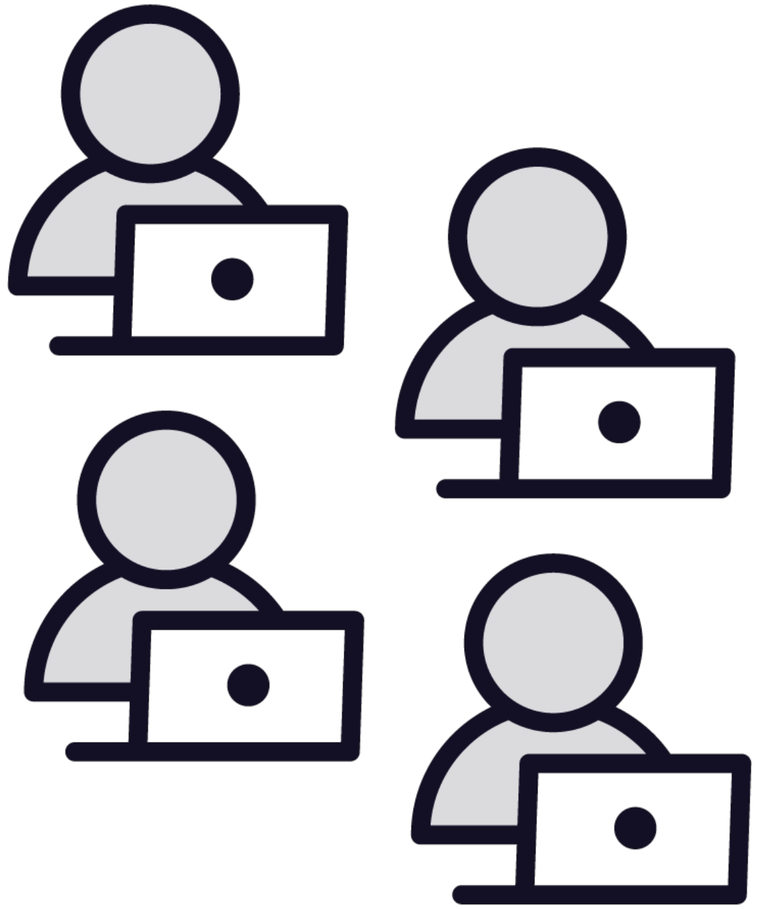
It is not a contest

Introductions

Dave Nicolette
Programmer

neopragma.com

- Your name
- Your role
- Your background
- Your learning goals



Administrative tasks

Dave Nicolette
Programmer

neopragma.com

Class evaluations

TBD

Participant survey

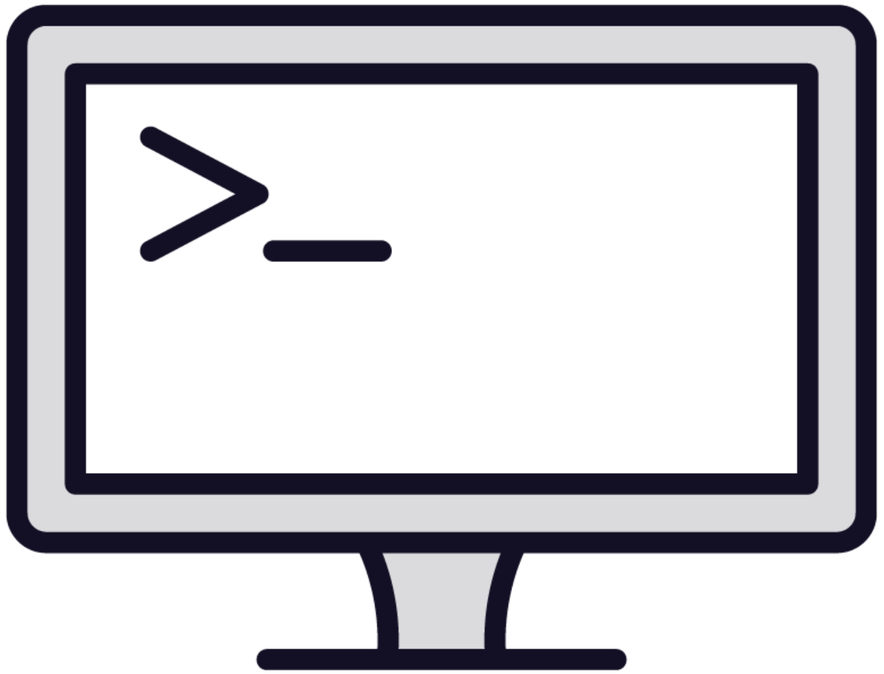
TBD

System access

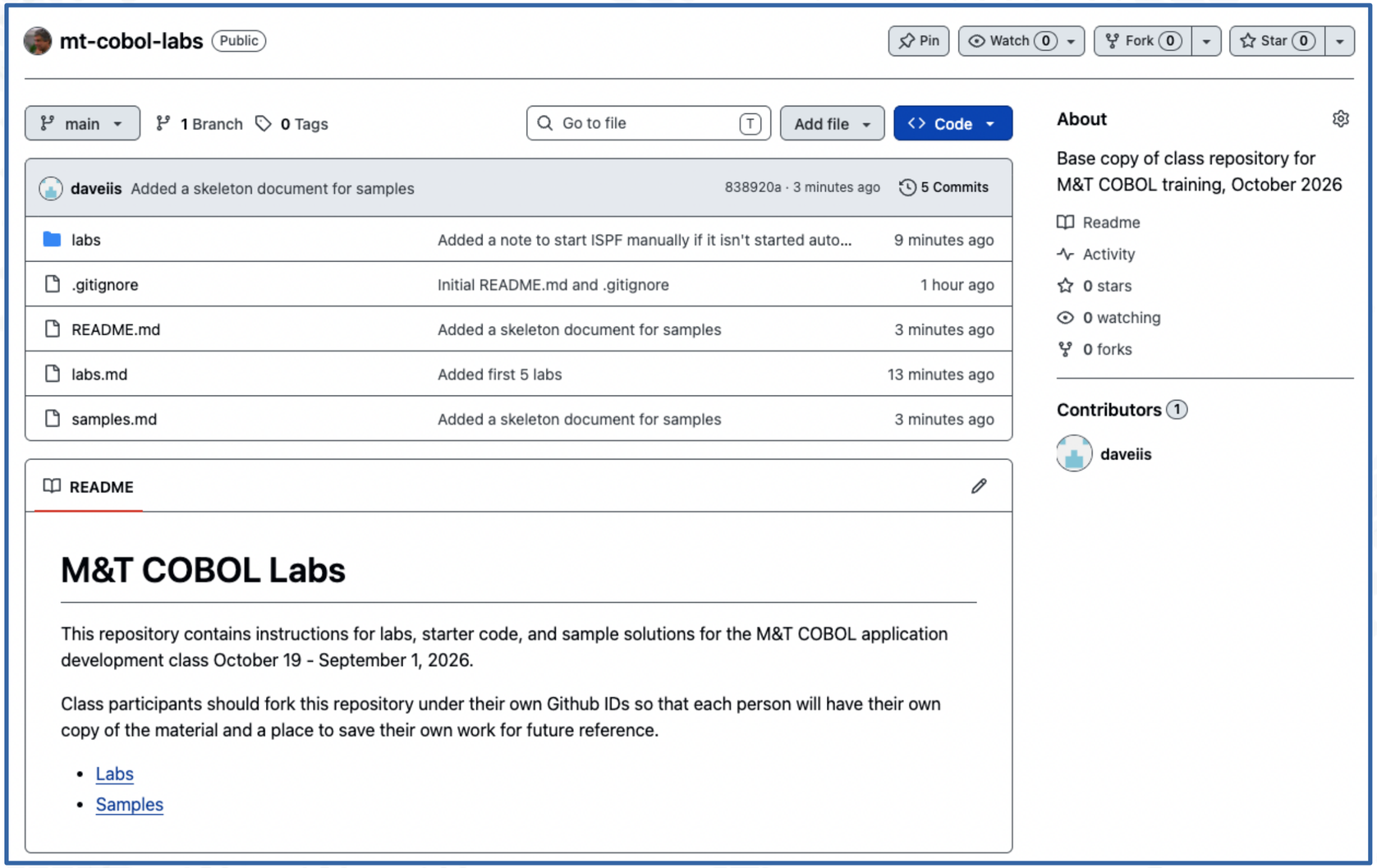
Dave Nicolette
Programmer

neopragma.com

- Github repository
- Virtual machine
- Terminal emulator



The class Github repository



Lab 1:
Fork the class repository

Create your own copy of the class Github repository.

In a Web browser, navigate to:

https://github.com/neopragma/mt-cobol-labs

Use the "Fork" option near the upper right-hand corner of the window to create your own copy of the repository.

If you don't already have a Github ID, create one by Following the instructions at https://github.com.

How you will access the z/OS environment

Your system

Each participant has a VM assigned to them. The VM is accessed via a Web browser on your system. The browser tab will present the virtual system's desktop.

Your VM (provided)

A 3270 terminal emulator is installed in the VM. It provides access to TSO on the z/OS system we will use in the class.

z/OS lab environment

Each participant has a userid defined on the z/OS system. We will be using a single, shared z/OS instance. A participant cannot modify or delete another participant's files on the instance.

Accessing your VM

TBD

Lab 2:
Access your virtual machine

Follow the instructions for this lab in your copy of the course Github repository.

Connecting to z/OS with a terminal emulator

TBD

Lab 3:
Connect to z/OS

Follow the instructions for this lab in your copy of the course Github repository.

Lab 4:
Log on and off TSO

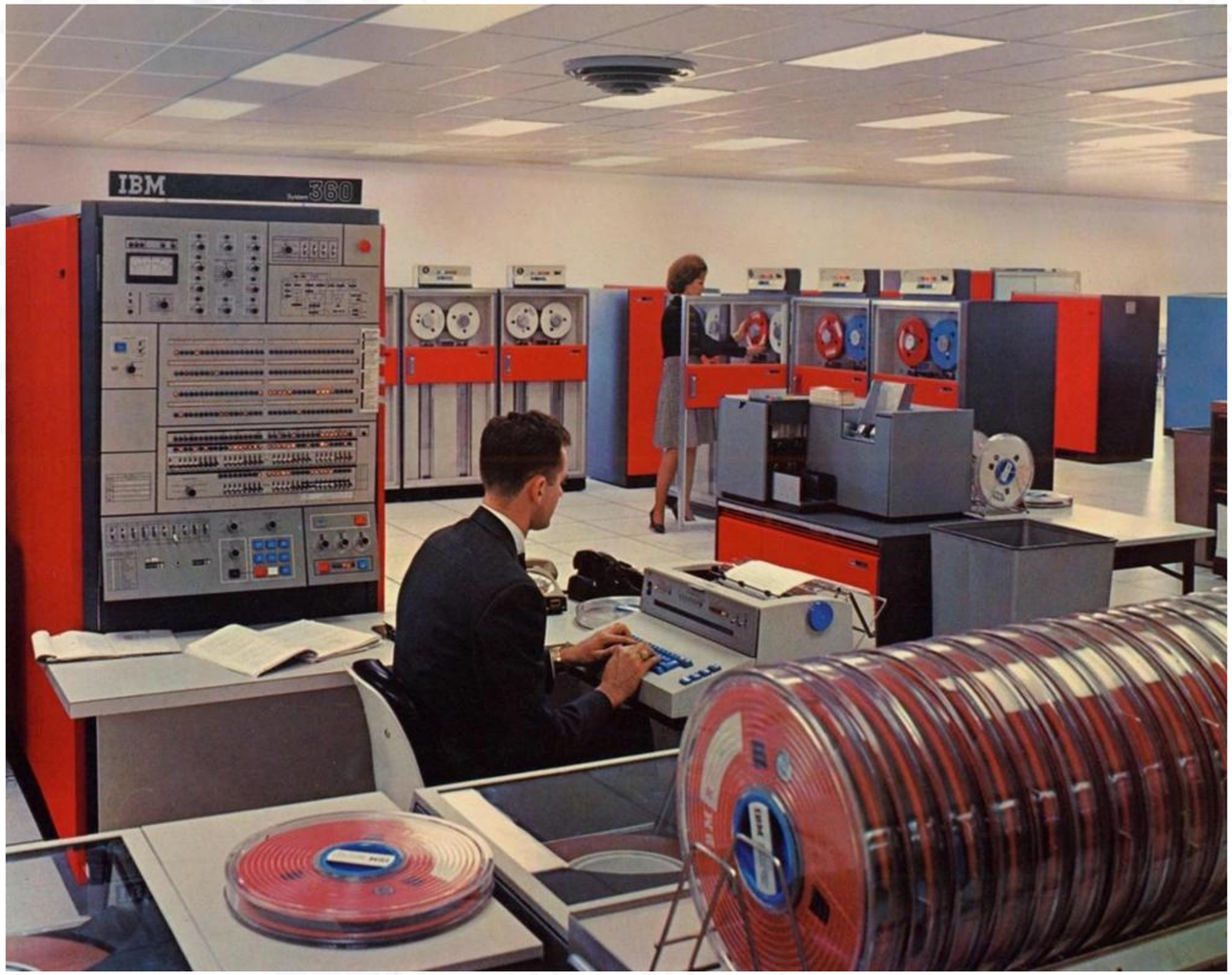
Follow the instructions for this lab in your copy of the course Github repository.

IBM z and z/OS Overview

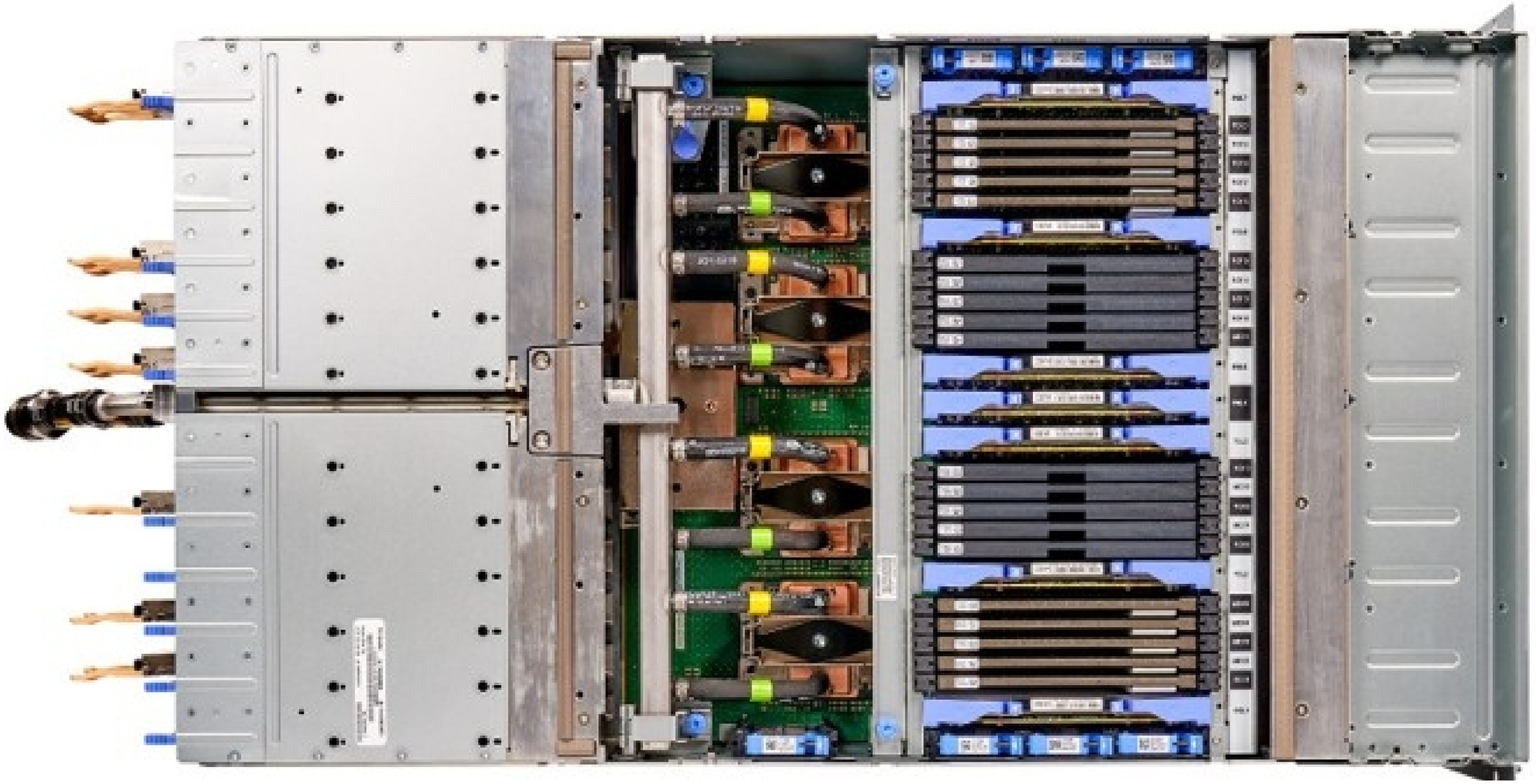
Dave Nicolette
Programmer

neopragma.com

IBM mainframes then and now



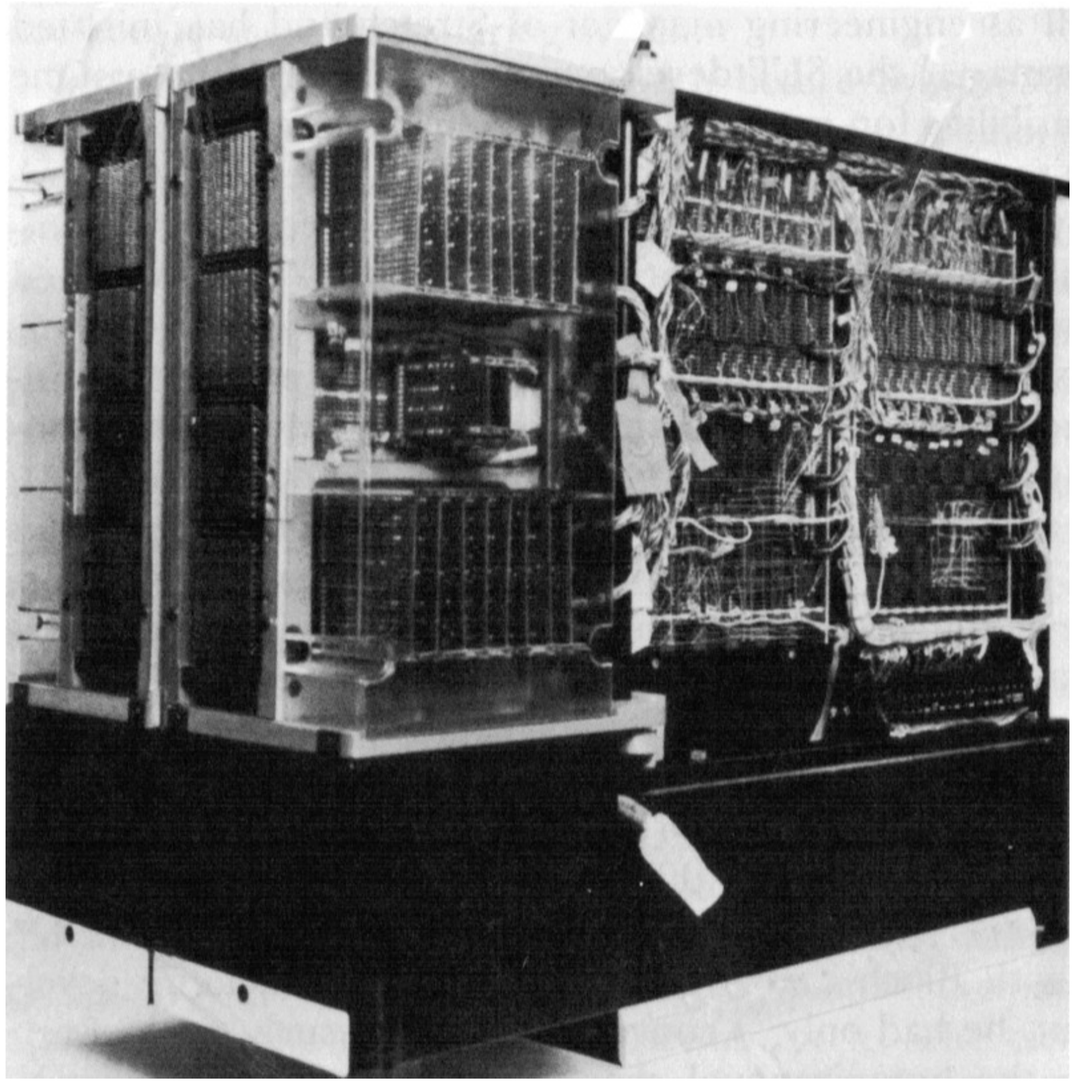
IBM z15 Central Processor Complex (CPC) Drawer



https://developer.ibm.com/technologies/systems/blogs/systems-inside-the-new-ibm-z15

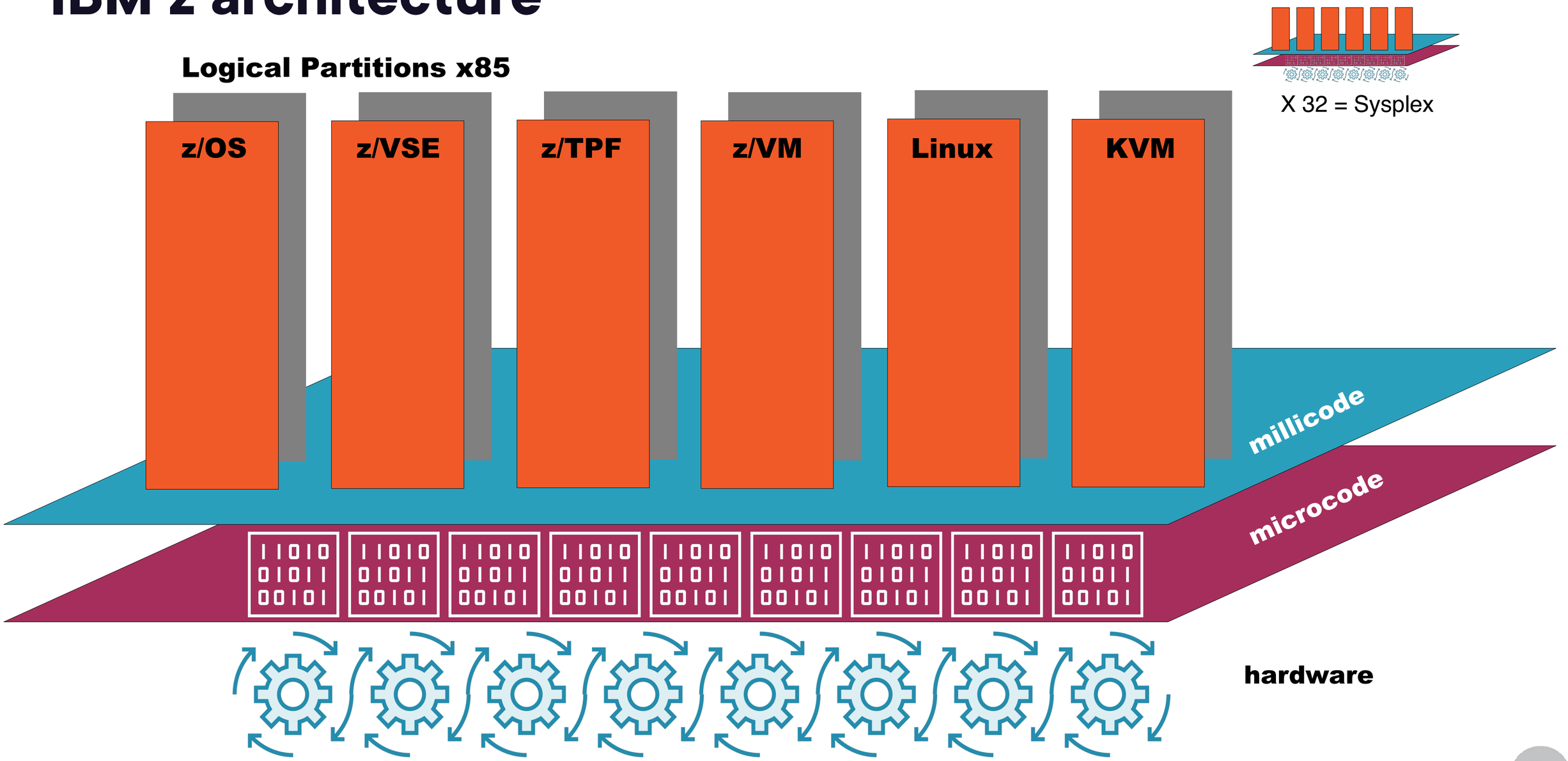
IBM S/360 Model 50 128KB Core Memory Module (1964)

Weight: 1150 lb
Height: 6 feet
Length: 5 feet
Width: 2 feet



https://www.righto.com

IBM z architecture



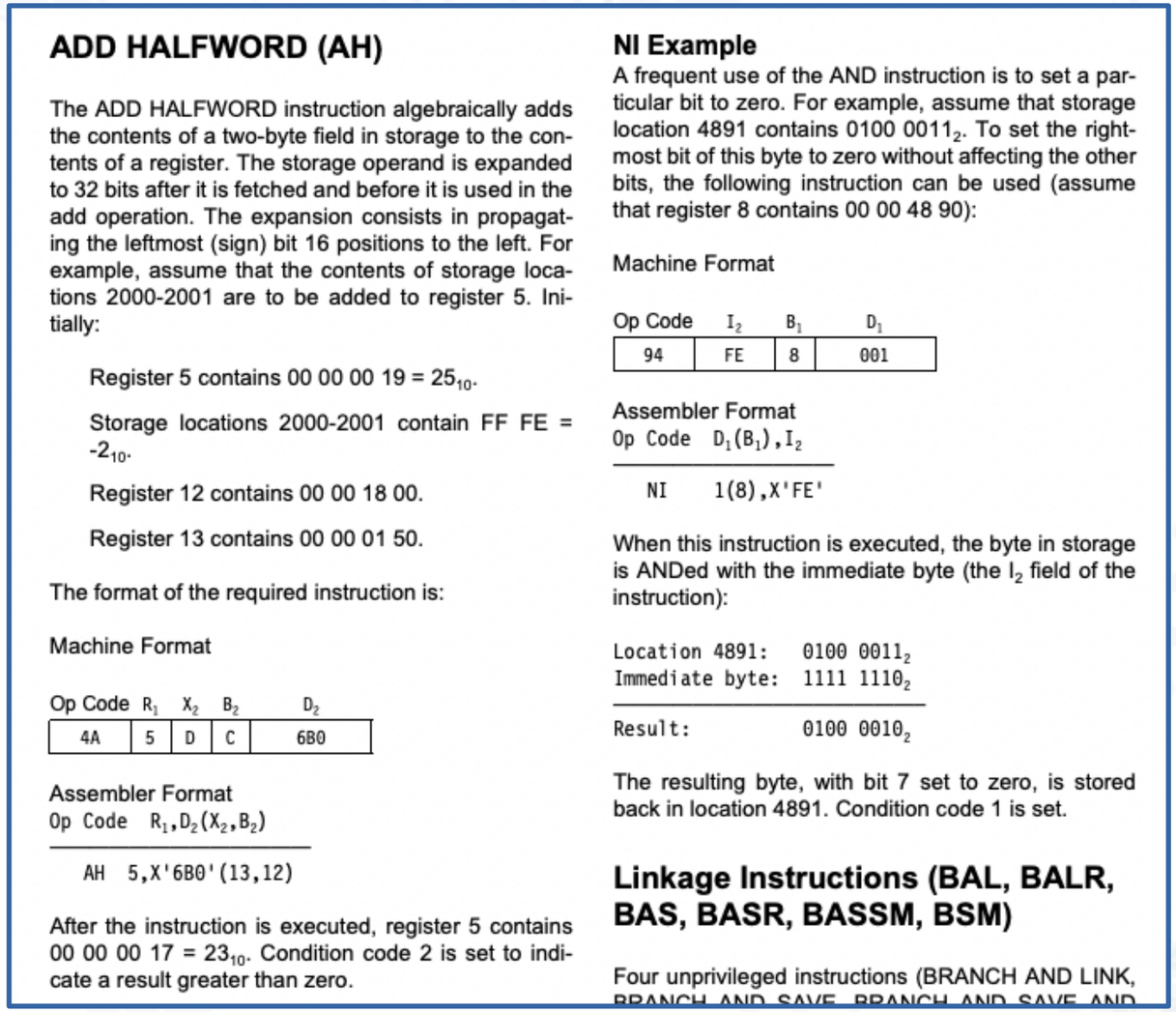
IBM z instructions

The instruction set for the system includes hundreds of instructions. In the 1960s, the instructions, registers, and all the rest of it were implemented as hardware circuitry.

For the past two decades or more, the machine architecture has been implemented as software.

On an IBM z today, there are several layers of virtualization between your program and the hardware.

That enables the system to run several operating systems and support multiple, disparate workloads simultaneously.



ADD HALFWORD (AH)

The ADD HALFWORD instruction algebraically adds the contents of a two-byte field in storage to the contents of a register. The storage operand is expanded to 32 bits after it is fetched and before it is used in the add operation. The expansion consists in propagating the leftmost (sign) bit 16 positions to the left. For example, assume that the contents of storage locations 2000-2001 are to be added to register 5. Initially:

Register 5 contains 00 00 00 19 = 25_{10}.

Storage locations 2000-2001 contain FF FE = -2_{10}.

Register 12 contains 00 00 18 00.

Register 13 contains 00 00 01 50.

The format of the required instruction is:

Machine Format

Op Code	R_1	X_2	B_2	D_2
4A	5	D	C	6B0

Assembler Format

```
Op Code  R1,D2(X2,B2)
AH   5,X'6B0'(13,12)
```

After the instruction is executed, register 5 contains 00 00 00 17 = 23_{10}. Condition code 2 is set to indicate a result greater than zero.

NI Example

A frequent use of the AND instruction is to set a particular bit to zero. For example, assume that storage location 4891 contains 0100 0011_2. To set the rightmost bit of this byte to zero without affecting the other bits, the following instruction can be used (assume that register 8 contains 00 00 48 90):

Machine Format

Op Code	I_2	B_1	D_1
94	FE	8	001

Assembler Format

```
Op Code  D1(B1),I2
   NI    1(8),X'FE'
```

When this instruction is executed, the byte in storage is ANDed with the immediate byte (the I_2 field of the instruction):

```
Location 4891:    0100 0011
Immediate byte:   1111 1110
Result:           0100 0010
```

The resulting byte, with bit 7 set to zero, is stored back in location 4891. Condition code 1 is set.

Linkage Instructions (BAL, BALR, BAS, BASR, BASSM, BSM)

Four unprivileged instructions (BRANCH AND LINK, BRANCH AND SAVE, BRANCH AND SAVE AND

from *IBM z Principles of Operation*

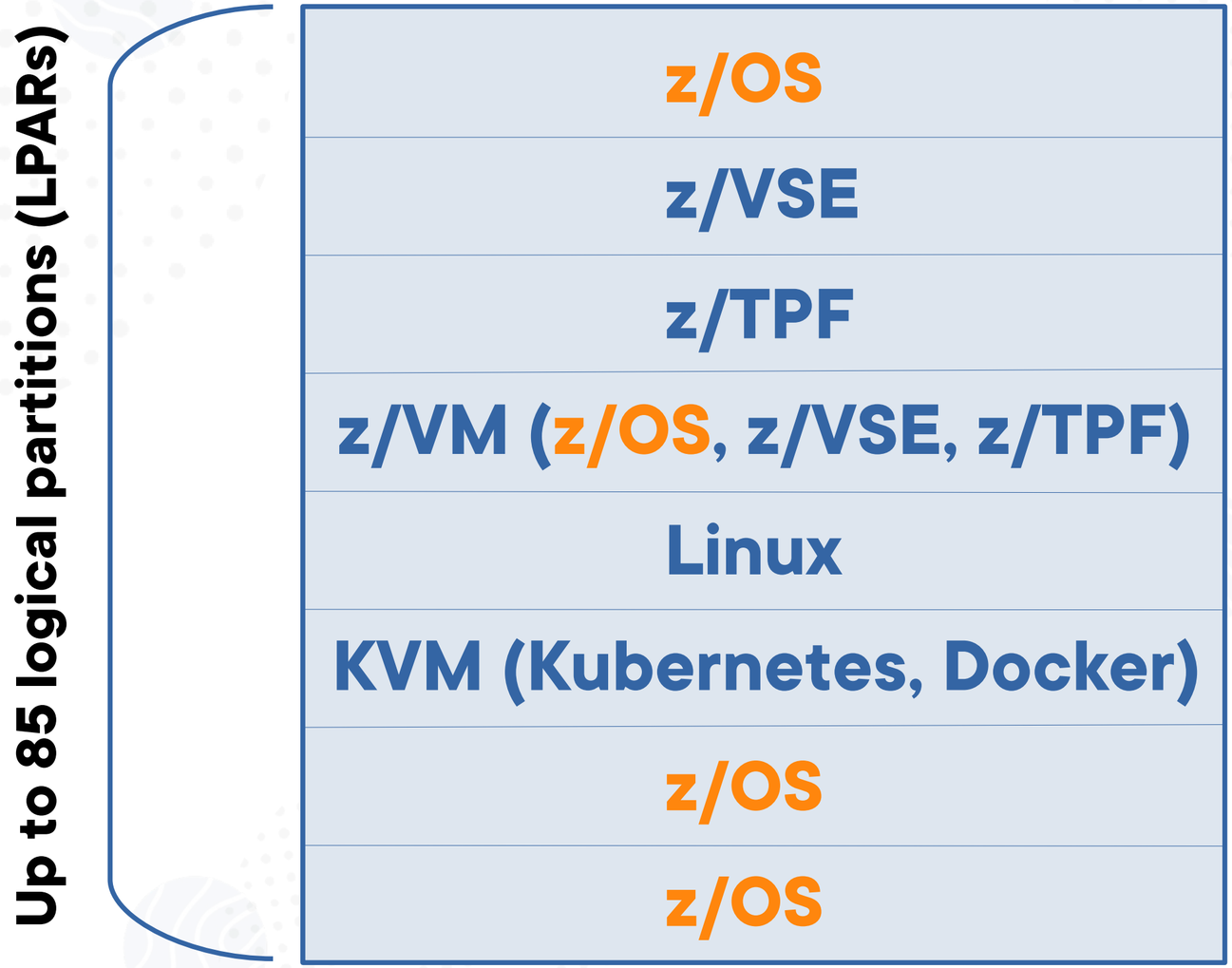
z/OS is a brain in a jar

To support IBM's commitment to backward compatibility – code compiled in 1964 will run unchanged on a modern z system – while adding support for emerging technologies over the years, the heart of the operating system still thinks it's talking to physical core memory, IBM 3390-style direct access storage devices, physical I/O channels, physical registers, and instructions built as circuitry.

All those things are virtualized and supported through software interfaces that enable z/OS to work as it did in the Long Ago while actually controlling modern hardware.

It's like a brain in a jar whose original body is long dead and buried, still perceiving the world as it once was through the five senses that it can process. When you work with z/OS, you will encounter throwbacks to the past. Backward compatibility is the reason for them.

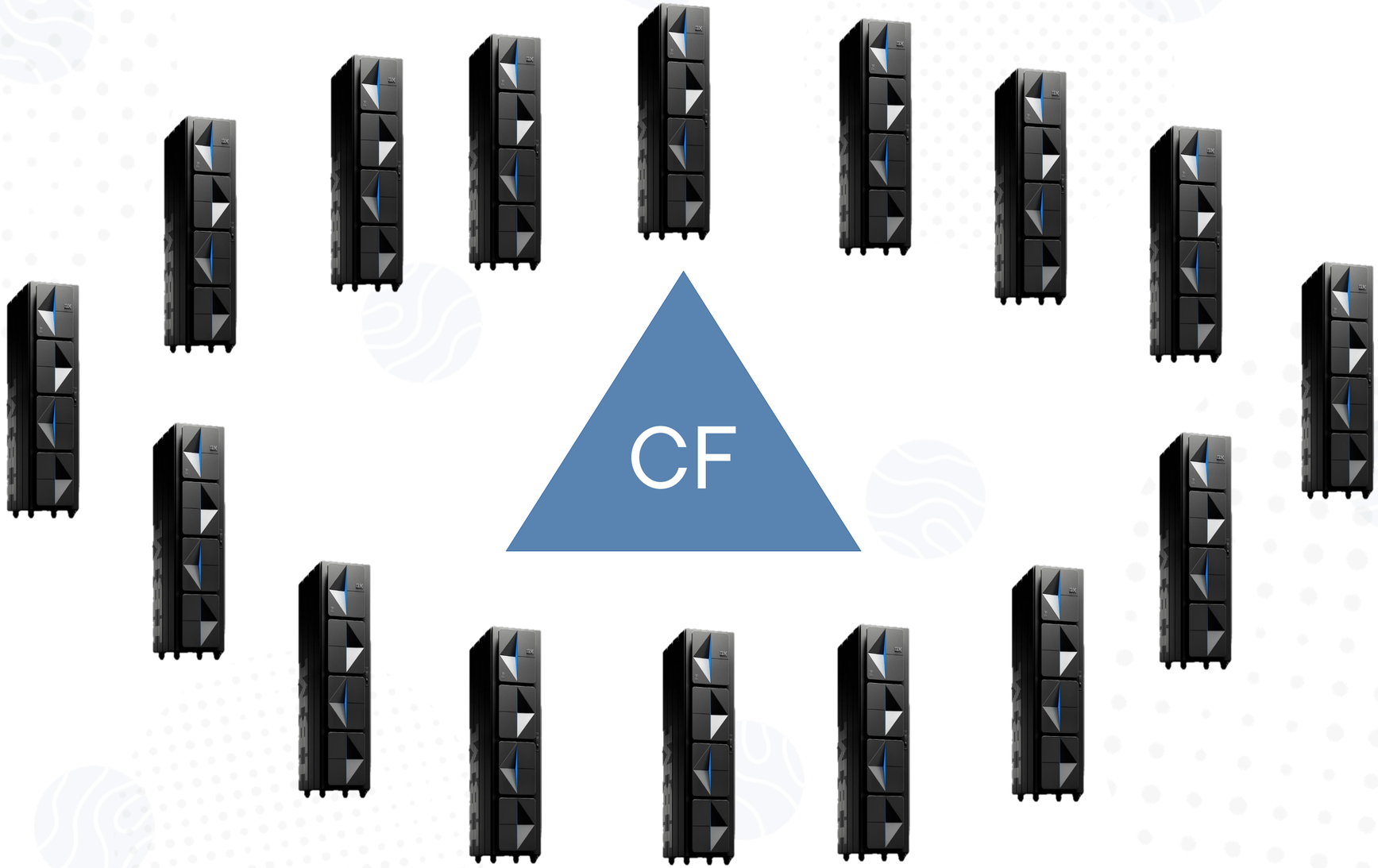
Operating systems supported on IBM z



Each LPAR can run any of

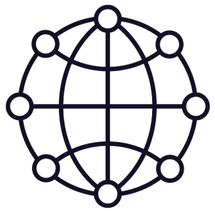
- z/OS
- z/VSE
- z/TPF
- z/VM (Type 1 hypervisor)
- RHEL
- SUSE
- Ubuntu Enterprise
- KVM (Type 1 hypervisor)

Parallel Sysplex – mainframe clustering



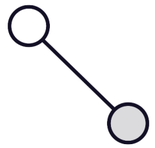
What does an operating system do?

Memory management



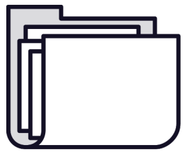
Network management

Process management



Hardware I/O interface

Security

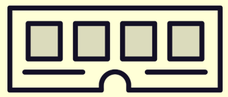


File management

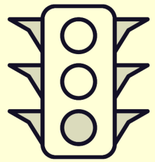
Secondary storage mgmt

User interface

Core functions of an operating system



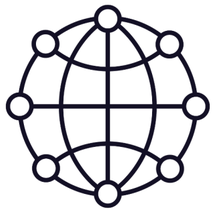
Memory management



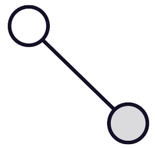
Process management

Security

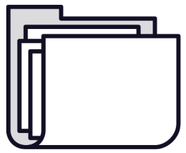
Secondary storage mgmt



Network management



Hardware I/O interface



File management

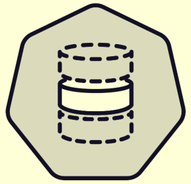
User interface

Functions usually handled by services

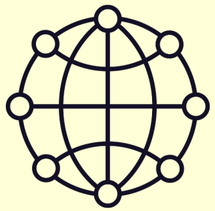
Memory management

Process management

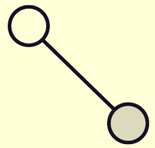
Security



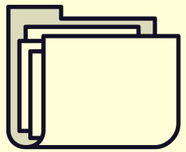
Secondary storage mgmt



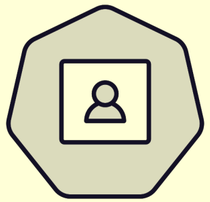
Network management



Hardware I/O interface



File management

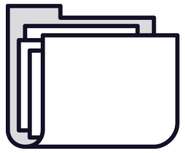


User interface

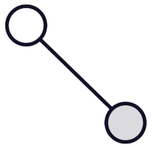
Services are called *subsystems* on z/OS

Security — RACF (or SAF-compliant 3rd party)

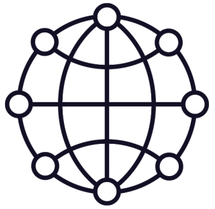
Secondary storage mgmt — DFSMSdss, DFSMSsms



File management — DFSMSdfp



Hardware I/O interface — I/O Supervisor (IOS)



Network management — Communications Server

User interface — Time-Sharing Option (TSO)

Two "sides" of z/OS

USS (Unix System Services)

Unix-like file organization and functionality

MVS (Multiple Virtual Systems)

The "traditional" or "legacy" functionality of z/OS

This class focuses on MVS

USS (Unix System Services)

Unix-like file organization and functionality

MVS (Multiple Virtual Systems)

The "traditional" or "legacy" functionality of z/OS

Terminology differences – general vs. MVS

Main memory	Main storage
Real memory	Real storage
Virtual memory	Virtual storage
Core, Kernel	Nucleus
Service, Daemon	Subsystem
Kernel space	Supervisor state
User space	Problem space
Process	Task
Thread	Subtask
Address space	Address space, Region
Filesystem	n/a
File	Data Set
Directory	n/a
n/a	Catalog
Create a file	Allocate a data set
Path, pathname	Fully-qualified data set name
Pathname segment	Data set qualifier
Filename suffix	n/a

A quick checkpoint

IBM mainframes...

...haven't changed much since the 1990s

...were completely redesigned from the ground up between 2000 and 2006

...have added support for emerging technologies while maintaining backward compatibility

...now run Unix instead of traditional mainframe operating systems

A quick checkpoint

IBM mainframes...

...haven't changed much since the 1990s

...were completely redesigned from the ground up between 2000 and 2006

...have added support for emerging technologies while maintaining backward compatibility

...now run Unix instead of traditional mainframe operating systems

A quick checkpoint

z/OS...

...is the flagship operating system for the IBM z

...is the only operating system that runs on the IBM z

...supports traditional mainframe languages like COBOL and PL/I, but not newer languages

...is based on Unix

A quick checkpoint

z/OS...

...is the flagship operating system for the IBM z

...is the only operating system that runs on the IBM z

...supports traditional mainframe languages like COBOL and PL/I, but not newer languages

...is based on Unix

A quick checkpoint

z/OS...

...is completely different from all other operating systems

...is exactly the same as all other operating systems

...is similar in some ways and different in some ways from other operating systems

...combines operating system functionality and application functionality in one

A quick checkpoint

z/OS...

...is completely different from all other operating systems

...is exactly the same as all other operating systems

...is similar in some ways and different in some ways from other operating systems

...combines operating system functionality and application functionality in one

Data Sets, Catalogs, DASD

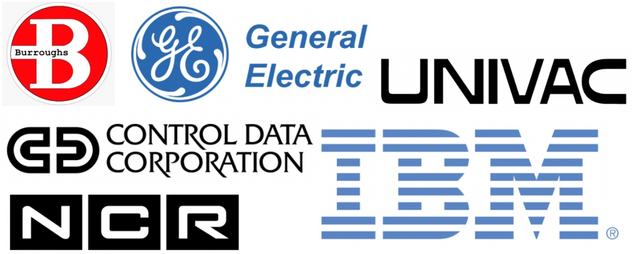
Dave Nicolette
Programmer

neopragma.com

Persistent data storage: Two approaches

Filesystem
Directories
Files
Pathnames
inodes

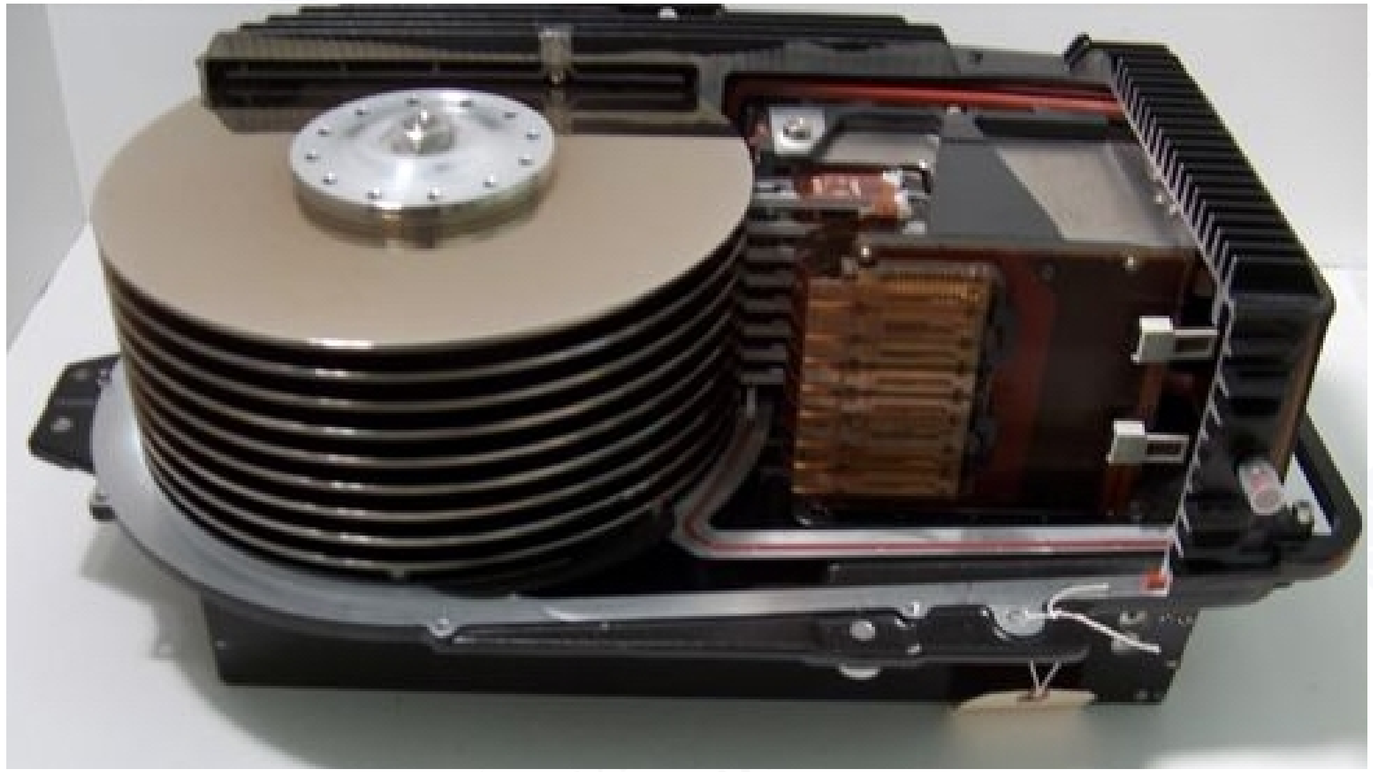
```
/u/alpha/dir2/file1.txt
\Users\alpha\dir2\file1.txt
```



Catalog
Data Sets
Qualified names
VTOC (Volume Table of Contents)
Access methods

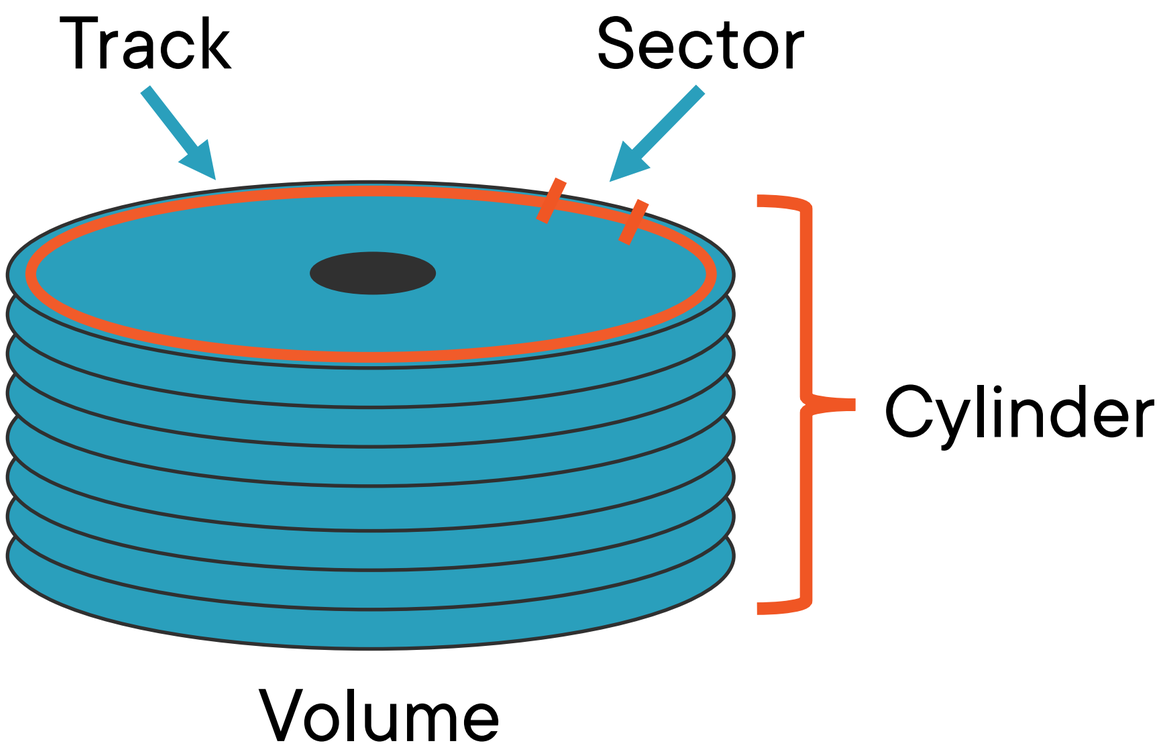
```
ALPHA.MLQ2.FILE1
```

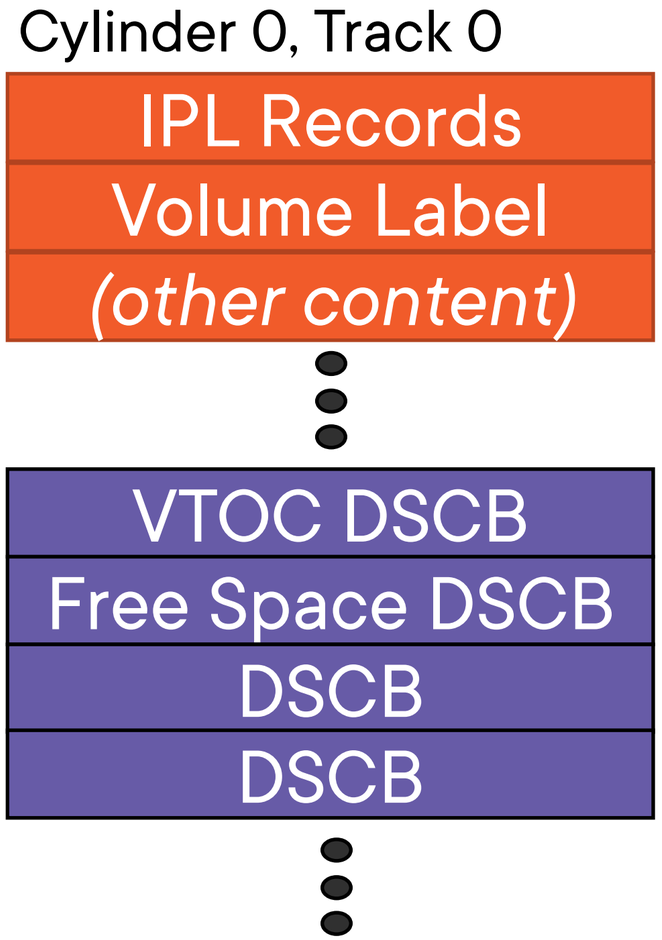
Direct Access Storage Device (DASD)



The guts of an IBM 3390 DASD *circa* 1980s-90s

Direct Access Storage Device (DASD)

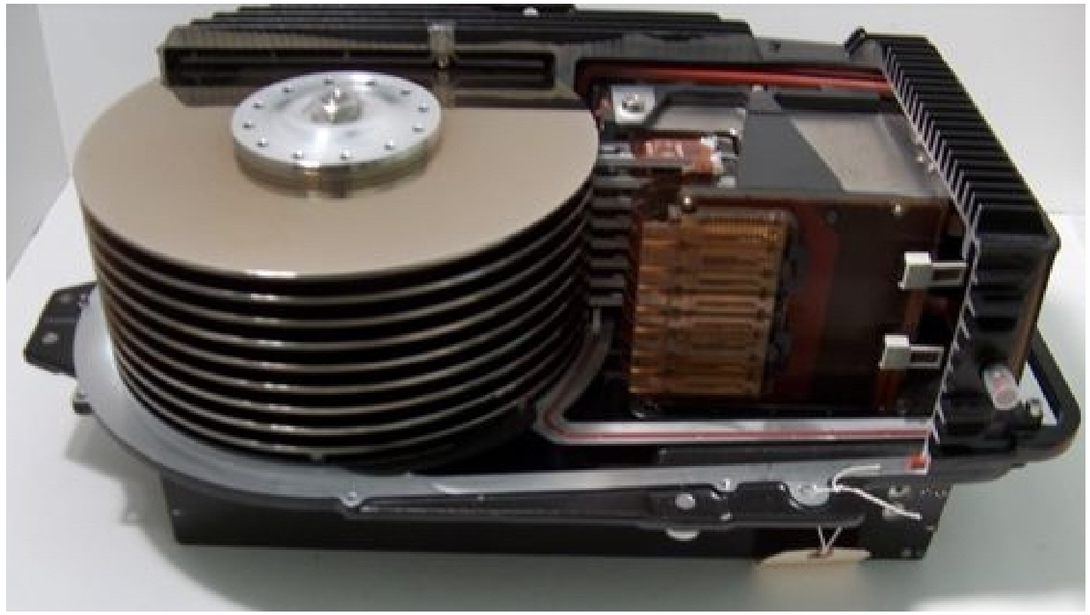




Direct Access Storage Device (DASD)

```
//FILE1 DD DSN=HIGH.MEDIUM.LOW,
//       DISP=(MOD,DELETE,DELETE),
//       SPACE=(TRK,1)


//FILE2 DD DSN=SOME.THING.HERE,
//       DISP=(NEW,CATLG,DELETE),
//       SPACE=(CYL,20)
```



Solid State Storage Device (DASD)

```
//FILE1 DD DSN=HIGH.MEDIUM.LOW,
//       DISP=(MOD,DELETE,DELETE),
//       SPACE=(TRK,1)


//FILE2 DD DSN=SOME.THING.HERE,
//       DISP=(NEW,CATLG,DELETE),
//       SPACE=(CYL,20)
```

Where are the tracks?

Where are the cylinders?

Brain in a jar

Think of z/OS as a brain in a jar. It was removed from its original body decades ago and it can work with all manner of modern peripheral devices, but its neural pathways and external connections are the same as they always were.

Inputs have to be translated into terms the brain is equipped to deal with – sight, smell, touch, taste, hearing. Outputs have to look like arms, legs, hands, and feet. This is for the sake of backward compatibility.

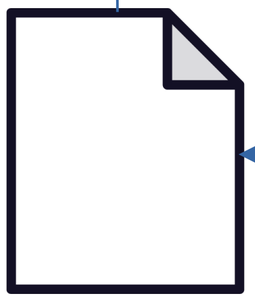
The I/O Subsystem presents external data storage devices to the brain in a jar as if they were still constructed of platters on a spindle with read/write heads, organized into tracks and cylinders. z/OS doesn't know what the actual storage medium is.

Catalog – VTOC – Data Set – Extents

Program requests data set by name:
`ALPHA.MLQ2.FILE1`

System looks up the name in the catalog to find the correct volume:
`DSN: ALPHA.MLQ2.FILE1`
`VOL: PVL332`

System looks up the VTOC entry on volume PVL322 to find the location of the data set on the storage medium:
`VOL: PVL332`
`DSN: ALPHA.MLQ2.FILE1`
`Location: offset from beginning of volume`
`Extents: different parts of the file`



System allocates the data set to the program.

Demo:
Find data sets using ISPF

We'll use ISPF Option 3.4 to find data sets with specified high-level qualifiers (HLQs).

Lab 5:
Find your data sets using ISPF

Follow the instructions for this lab in your copy of the course Github repository.

Queued Sequential Access Method (QSAM)

Dave Nicolette
Programmer

neopragma.com

MVS data set types and access methods

	BSAM	QSAM	BDAM	BPAM	VSAM ESDS	VSAM RRDS	VSAM KSDS	VSAM LDS
Seq forward	X	X		X	X	X	X	X
Seq backward						X	X	
Skip seq					X	X	X	
Direct by offset			X		X	X	X	
Direct by number						X		
Direct by key							X	
Byte stream								X

Legacy – backward compatibility only

Libraries

LDS used by system facilities, rarely by applications

Access rarely done this way (relative byte address or RBA)

Data set types you will use in your JCL & COBOL labs

	BSAM	QSAM	BDAM	BPAM	VSAM ESDS	VSAM RRDS	VSAM KSDS	VSAM LDS
Seq forward	X	X		X	X	X	X	X
Seq backward						X	X	
Skip seq					X	X	X	
Direct by offset			X		X	X	X	
Direct by number						X		
Direct by key							X	
Byte stream								X

- Legacy – backward compatibility only
- Libraries
- LDS used by system facilities, rarely by applications
- Access rarely done this way (relative byte address or RBA)

Logical records and blocks

```
DSN: MARY.LITTLE.LAMB    DSORG: PS   RECFM: FB   LRECL: 30   BLKSIZE: 60
```

```
                 1         2         3
       01234567890123456789012345678 90
Record 1 Mary had a little lamb
Record 2 whose fleece was white as snow,
Record 3 and everywhere that Mary went
Record 4 the lamb was sure to eat grass.
```

Application programs process *logical records* one at a time.

The system reads and writes data in *blocks*.

```
                 1         2         3         4         5         6
       012345678901234567890123456789012345678901234567890123456789 0
Block 1 Mary had a little lamb          whose fleece was white as snow,
Block 2 and everywhere that Mary went the lamb was sure to eat grass.
```

QSAM – Queued Sequential Access Method

```
. . .
    PERFORM Next-Record
        UNTIL Lamb-End
. . .
Next-Record.
    READ Lamb-File
        INTO Lamb-Work-Area
        AT END
            SET Lamb-End TO TRUE
        NOT AT END
            PERFORM Lamb-Manipulation
    END-READ
    .
```

```
Read # System retrieves       Program sees

  1    [Mary had a little lamb        whose fleece was white as snow,]
                                [Mary had a little lamb          ]

  2    (block is already in memory)
                                [whose fleece was white as snow,]

  3    [and everywhere that Mary went the lamb was sure to eat grass.]
                                [and everywhere that Mary went]

  4    (block is already in memory)
                                [the lamb was sure to eat grass.]

  5    (system signals end of file)
                                The AT END clause is triggered.
```

QSAM pre-reads blocks and feeds individual logical records to the requesting application program one by one.

Demo:
Allocate data sets using ISPF

We'll use ISPF Option 3.2 to allocate, catalog, and delete QSAM data sets.

Lab 6:
Allocate QSAM data set using ISPF

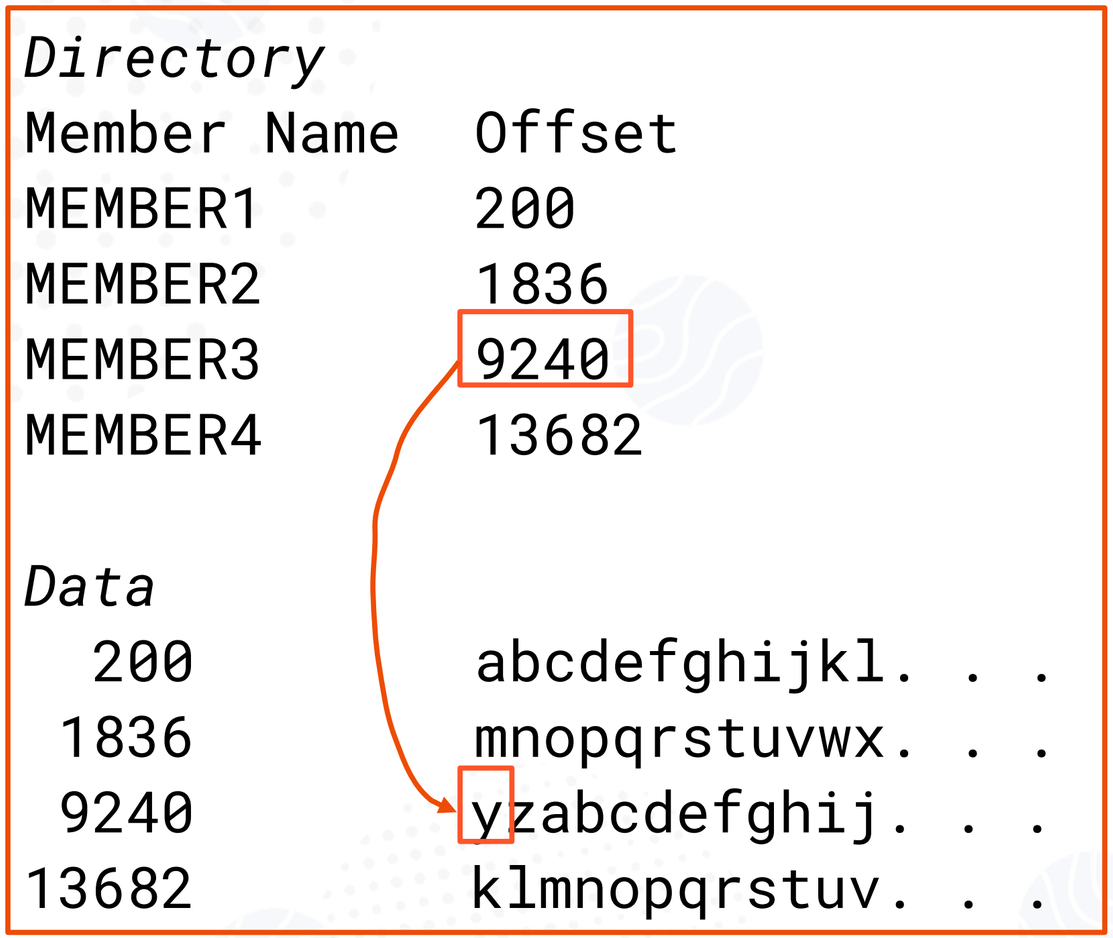
Follow the instructions for this lab in your copy of the course Github repository.

Basic Partitioned Access Method (BPAM)

Dave Nicolette
Programmer

neopragma.com

Partitioned Data Sets (Libraries)



A PDS is a collection of data sets that share the same attributes, such as data set organization, logical record length and block size.

The individual data sets in a library are called *members* of the library.

As far as z/OS is concerned, the entire library is a single data set. The BPAM Access Method handles the gory details.

Application programs can treat a member of a library as a separate data set.

Libraries for application developers

Libraries might be defined for production, test, staging, bug fixing, operations, or other purposes. Each organization has naming conventions that simplify security setup and help people understand the purpose of each data set.

Application developers have their own libraries. The high-level qualfier (HLQ) for a developer's libraries is the same as their TSO prefix. Usually, security is set up such that a developer can create, delete, and modify data sets whose HLQ matches their own TSO prefix.

An application developer might organize their data sets into libraries like this:

```
<prefix>.DEV.JCL
<prefix>.DEV.COBOL
<prefix>.DEV.COPYLIB
<prefix>.DEV.REXX
<prefix>.DEV.SQL
<prefix>.DEV.SNIPPETS
<prefix>.DEV.PROGLIB
```

```
<prefix>.TST.JCL
<prefix>.TST.COBOL
<prefix>.TST.COPYLIB
<prefix>.TST.REXX
<prefix>.TST.SQL
<prefix>.TST.PROGLIB
```

PDS and PDSE

Old

New

Less old

PDS

For backward compatibility

Fixed amount of
directory space

"Load library" holds object
modules that need
tweaking at load time

PDSE

Use for new work

Directory space managed
more efficiently

"Program Library" holds
ready-to-load executables

Demo:
Allocate source libraries using ISPF

We'll use ISPF Option 3.2 to allocate, catalog, and delete partitioned data sets.

Lab 7:
Allocate a JCL library for yourself

Follow the instructions for this lab in your copy of the course Github repository.

The ISPF Editor

Dave Nicolette
Programmer

neopragma.com

About the ISPF Editor

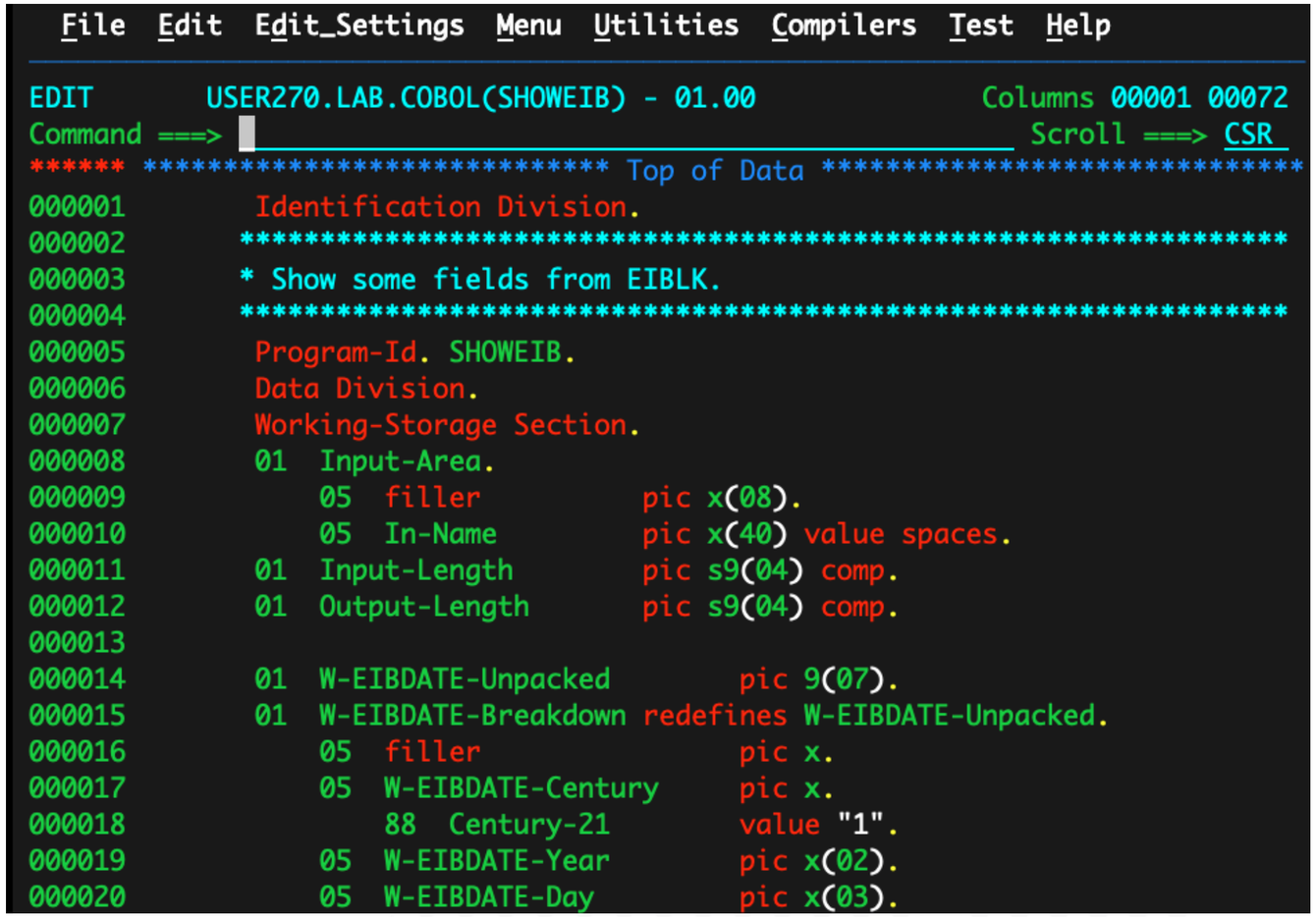
A text editor, not an IDE

Designed to work with IBM 3270 terminals – a 24x80 grid of characters

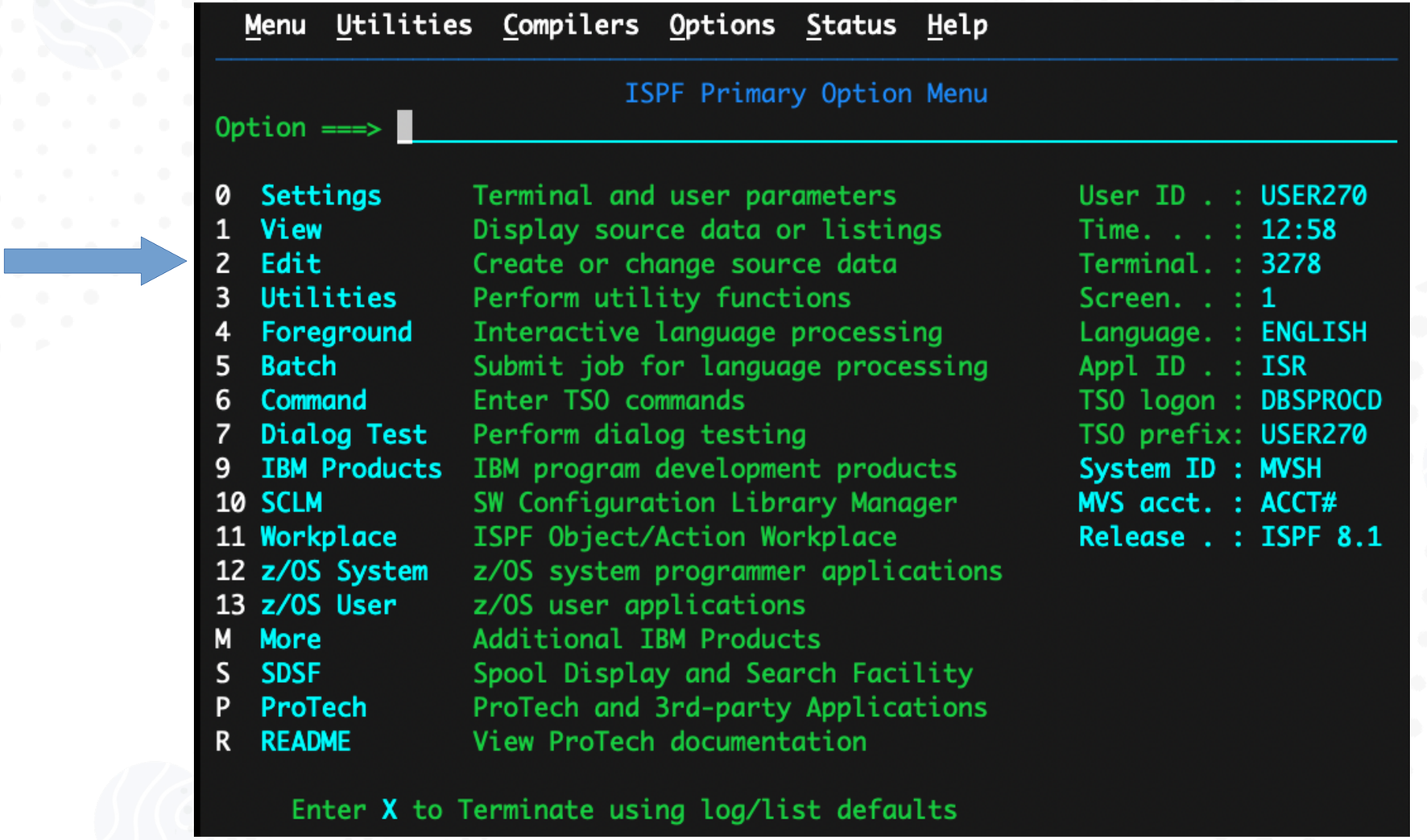
Keyboard-only input – a mouse can position the cursor, but nothing more

Offers some configuration options

Customizable by writing macros



Accessing the ISPF Editor

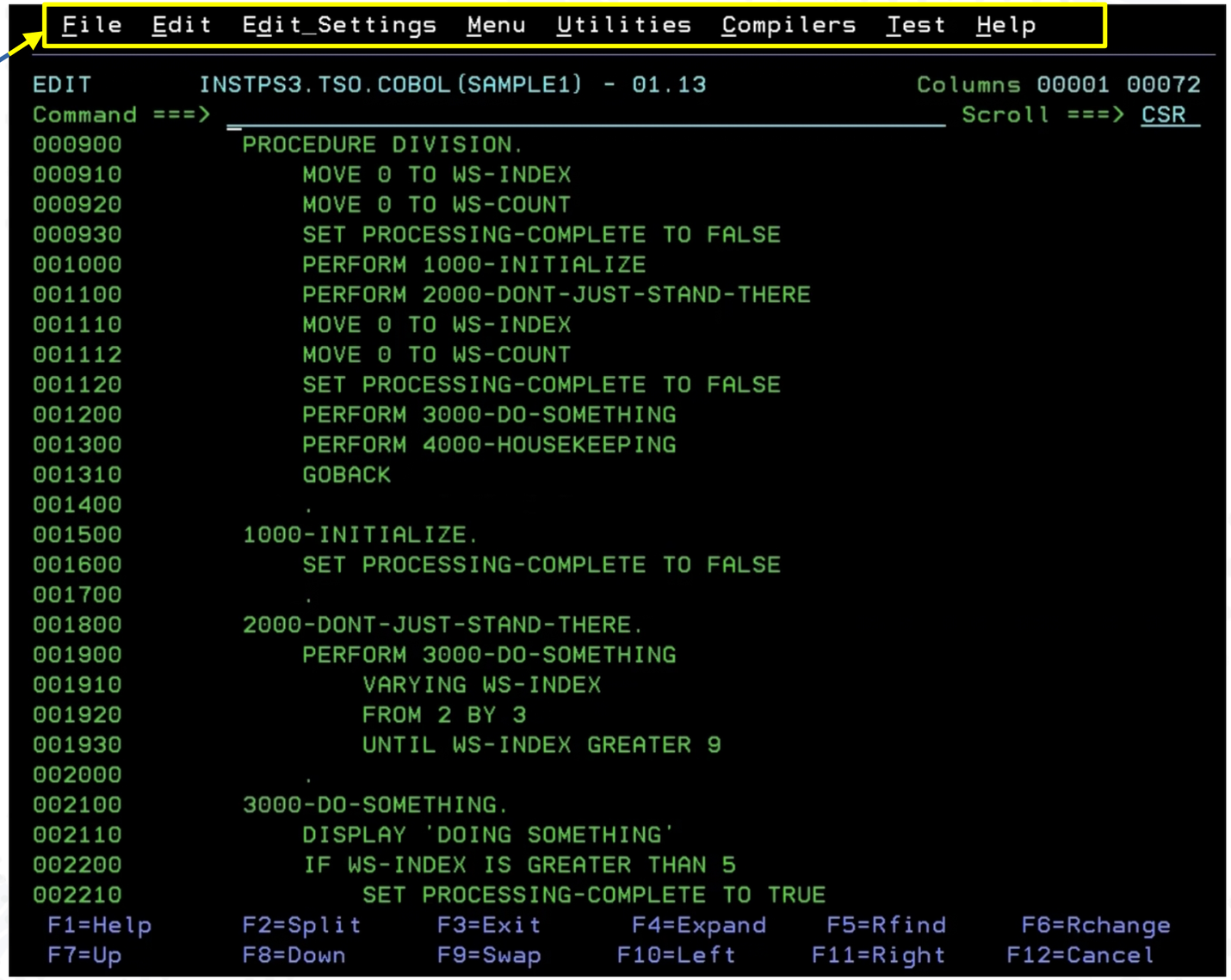


ISPF Editor panel layout

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
EDIT       INSTPS3.TSO.COBOL(SAMPLE1) - 01.13            Columns 00001 00072
Command ===> _____________________________________________ Scroll ===> CSR
000900         PROCEDURE DIVISION.
000910             MOVE 0 TO WS-INDEX
000920             MOVE 0 TO WS-COUNT
000930             SET PROCESSING-COMPLETE TO FALSE
001000             PERFORM 1000-INITIALIZE
001100             PERFORM 2000-DONT-JUST-STAND-THERE
001110             MOVE 0 TO WS-INDEX
001112             MOVE 0 TO WS-COUNT
001120             SET PROCESSING-COMPLETE TO FALSE
001200             PERFORM 3000-DO-SOMETHING
001300             PERFORM 4000-HOUSEKEEPING
001310             GOBACK
001400             .
001500         1000-INITIALIZE.
001600             SET PROCESSING-COMPLETE TO FALSE
001700             .
001800         2000-DONT-JUST-STAND-THERE.
001900             PERFORM 3000-DO-SOMETHING
001910                 VARYING WS-INDEX
001920                 FROM 2 BY 3
001930                 UNTIL WS-INDEX GREATER 9
002000             .
002100         3000-DO-SOMETHING.
002110             DISPLAY 'DOING SOMETHING'
002200             IF WS-INDEX IS GREATER THAN 5
002210                 SET PROCESSING-COMPLETE TO TRUE
 F1=Help      F2=Split     F3=Exit      F4=Expand    F5=Rfind     F6=Rchange
 F7=Up        F8=Down      F9=Swap      F10=Left     F11=Right    F12=Cancel
```

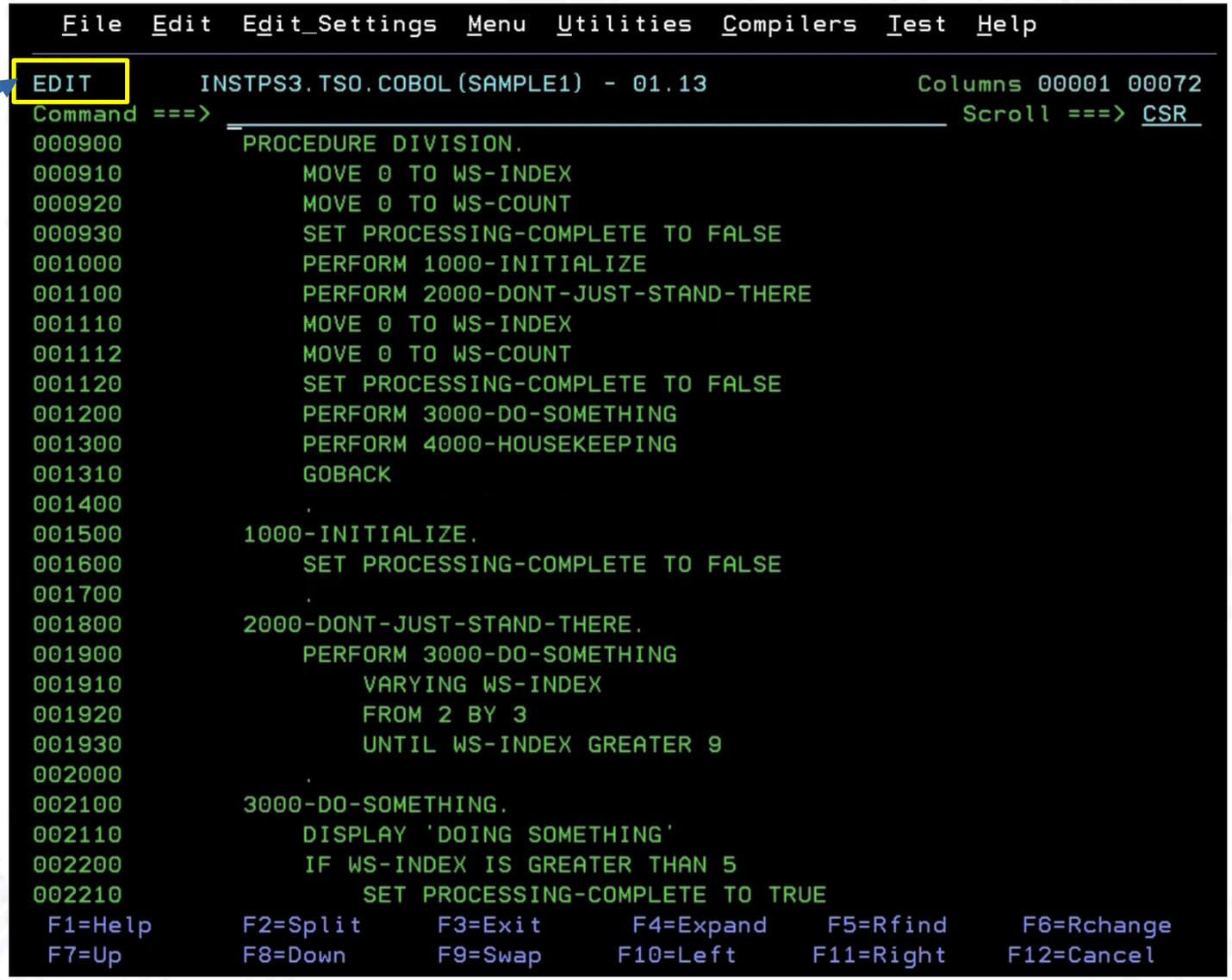
ISPF Editor panel layout

Action bar



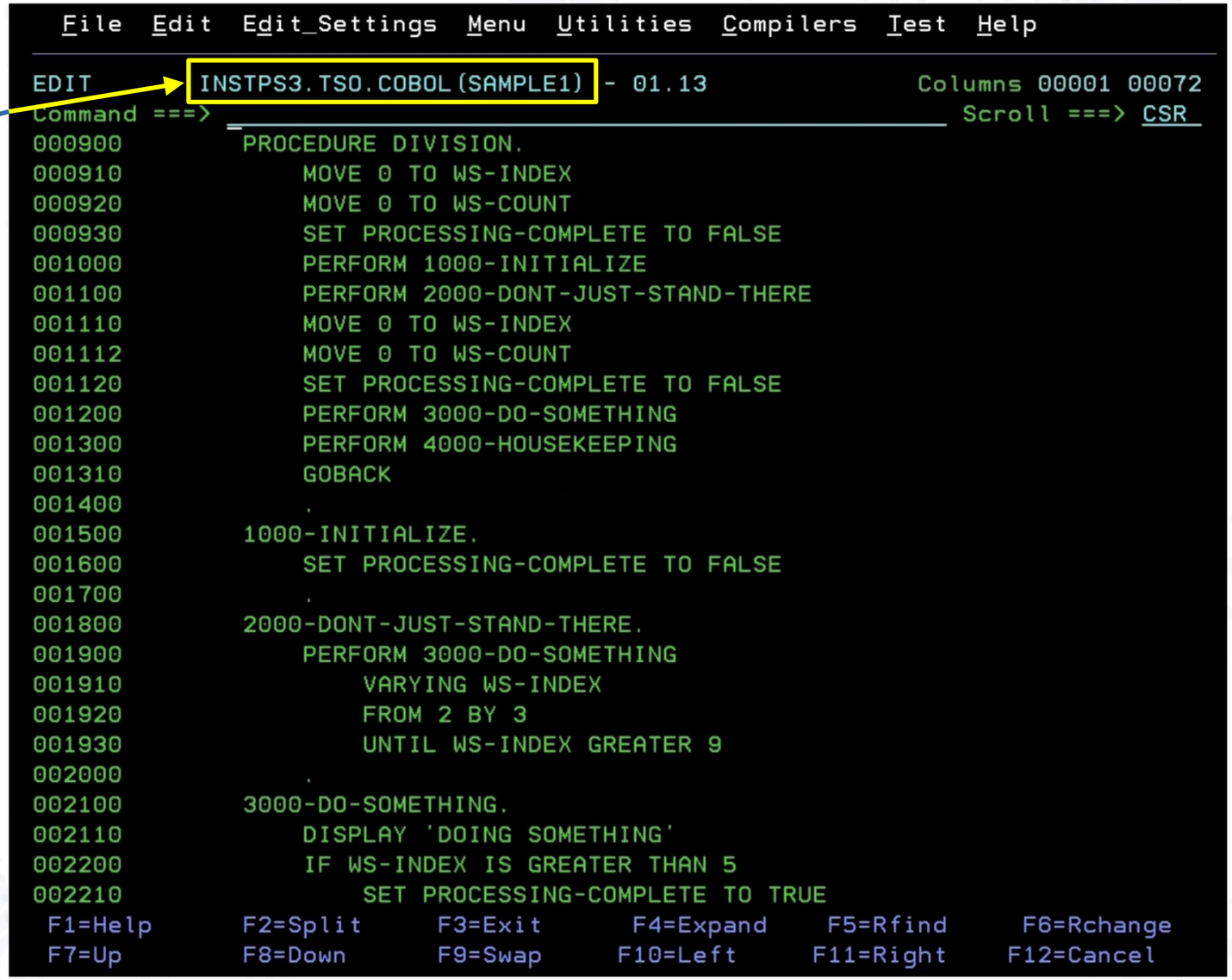
ISPF Editor panel layout

"View" or
"Edit"

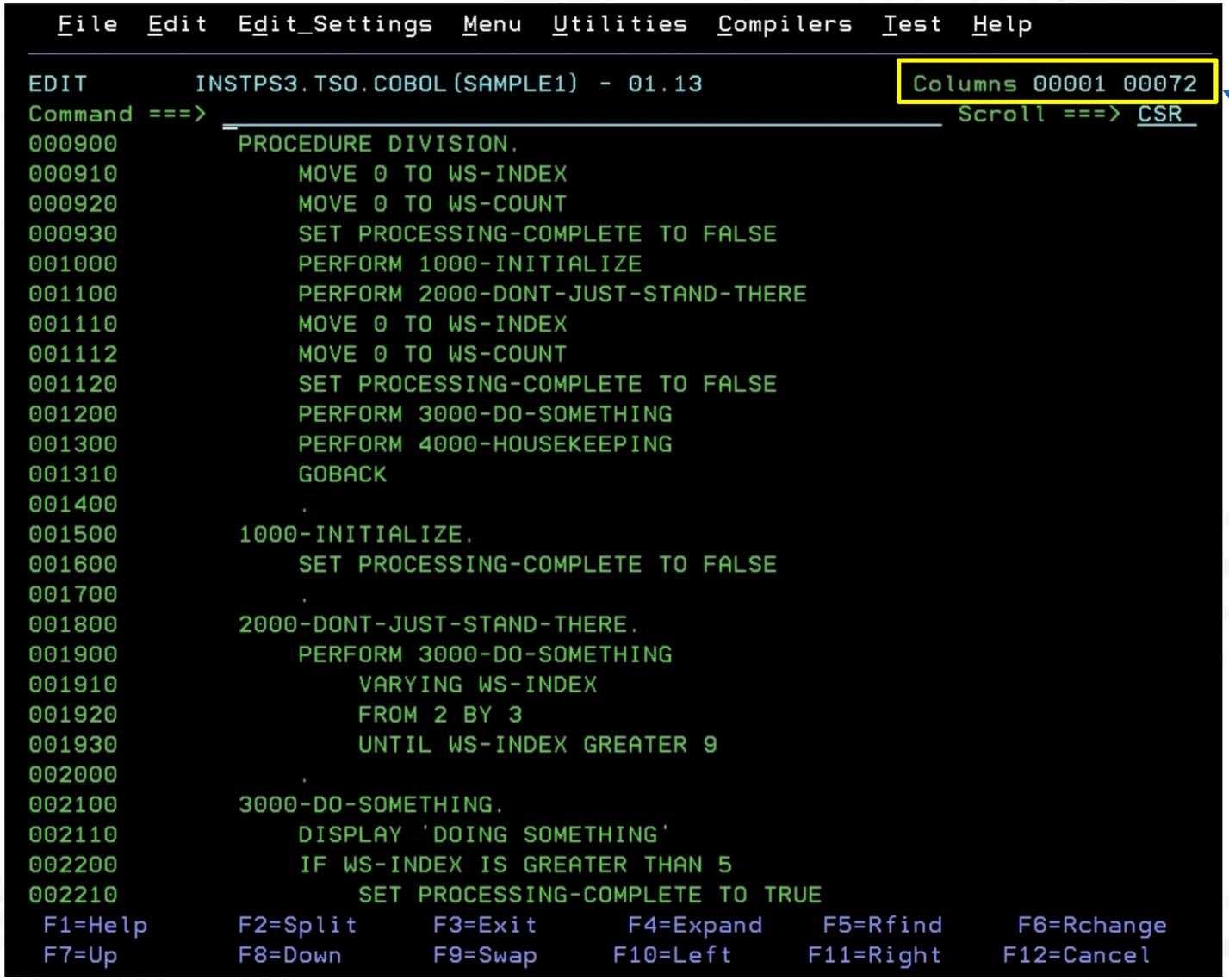


ISPF Editor panel layout

Name of data set



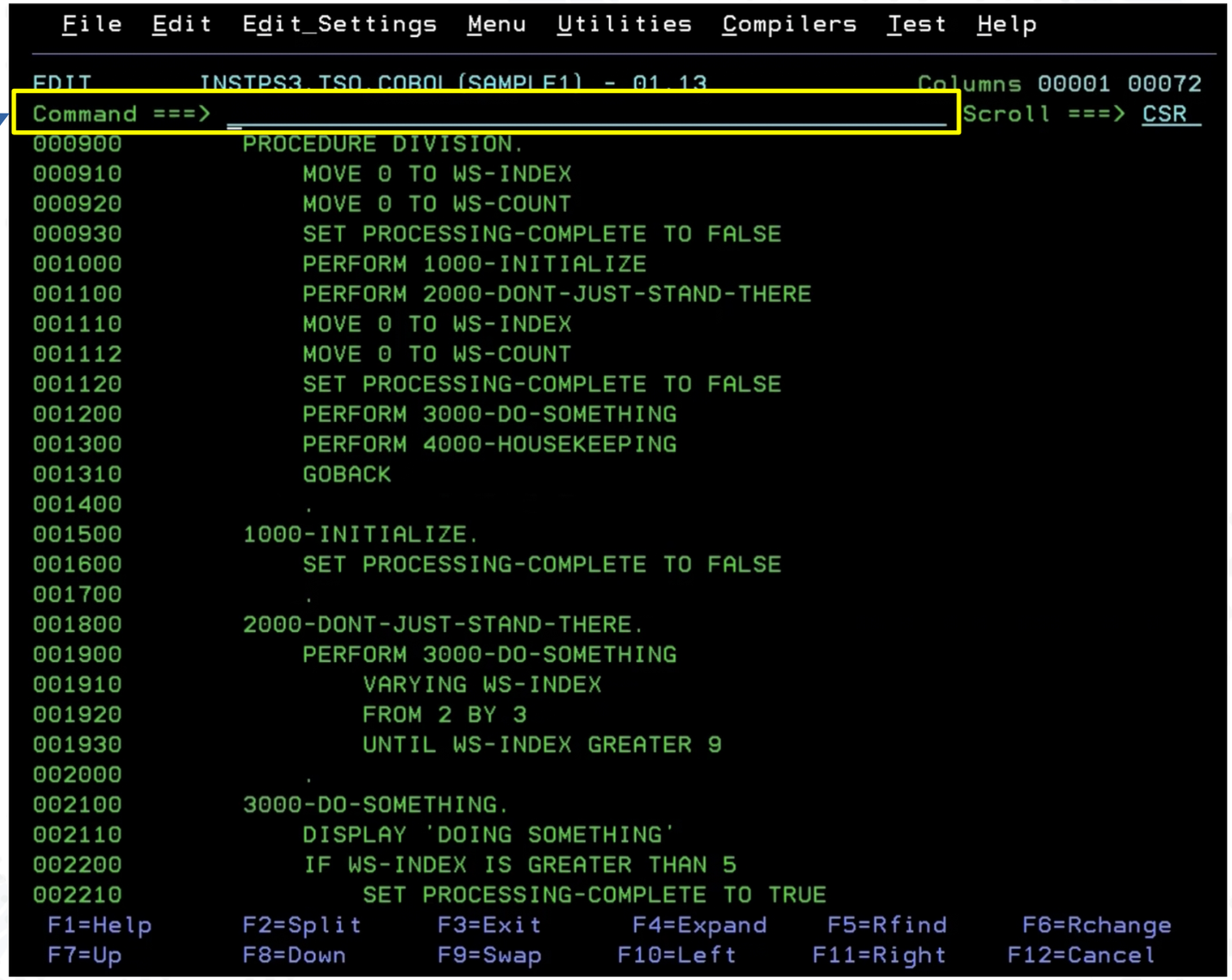
ISPF Editor panel layout



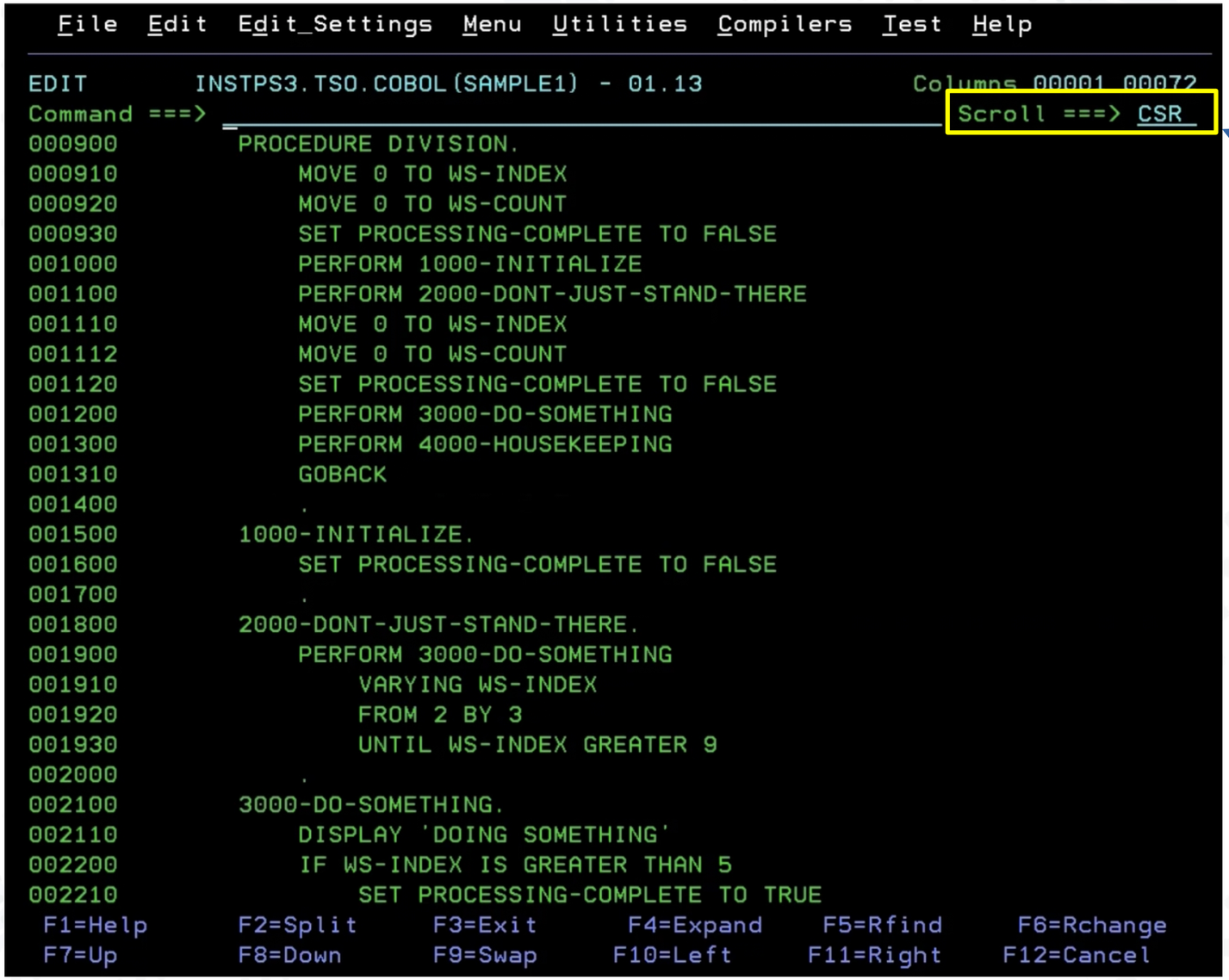
Which columns are visible in the editor pane

ISPF Editor panel layout

Command field



ISPF Editor panel layout

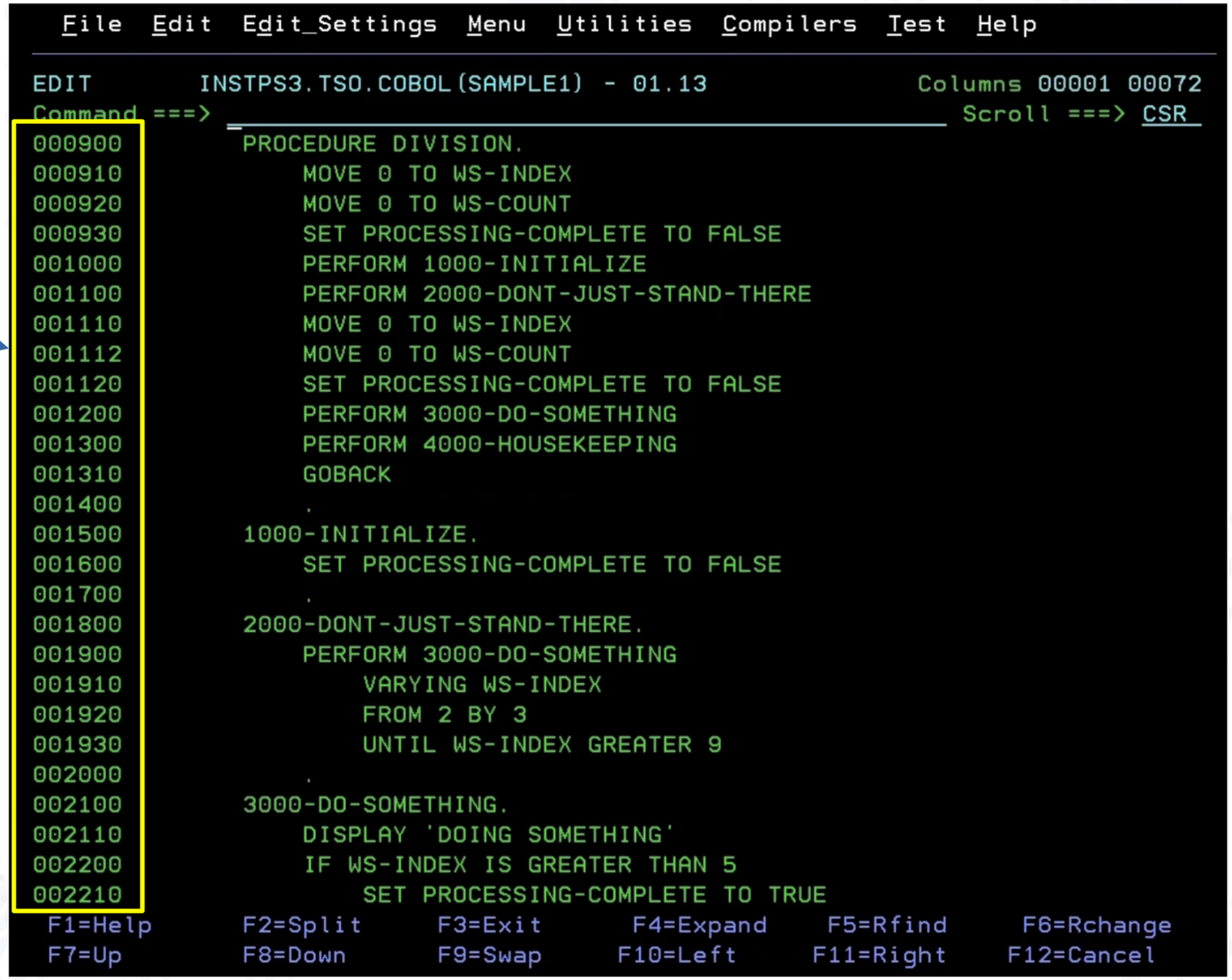


Scroll amount. PAGE, CSR, or a number. Vertical: Lines. Horizontal: Columns

ISPF Editor panel layout

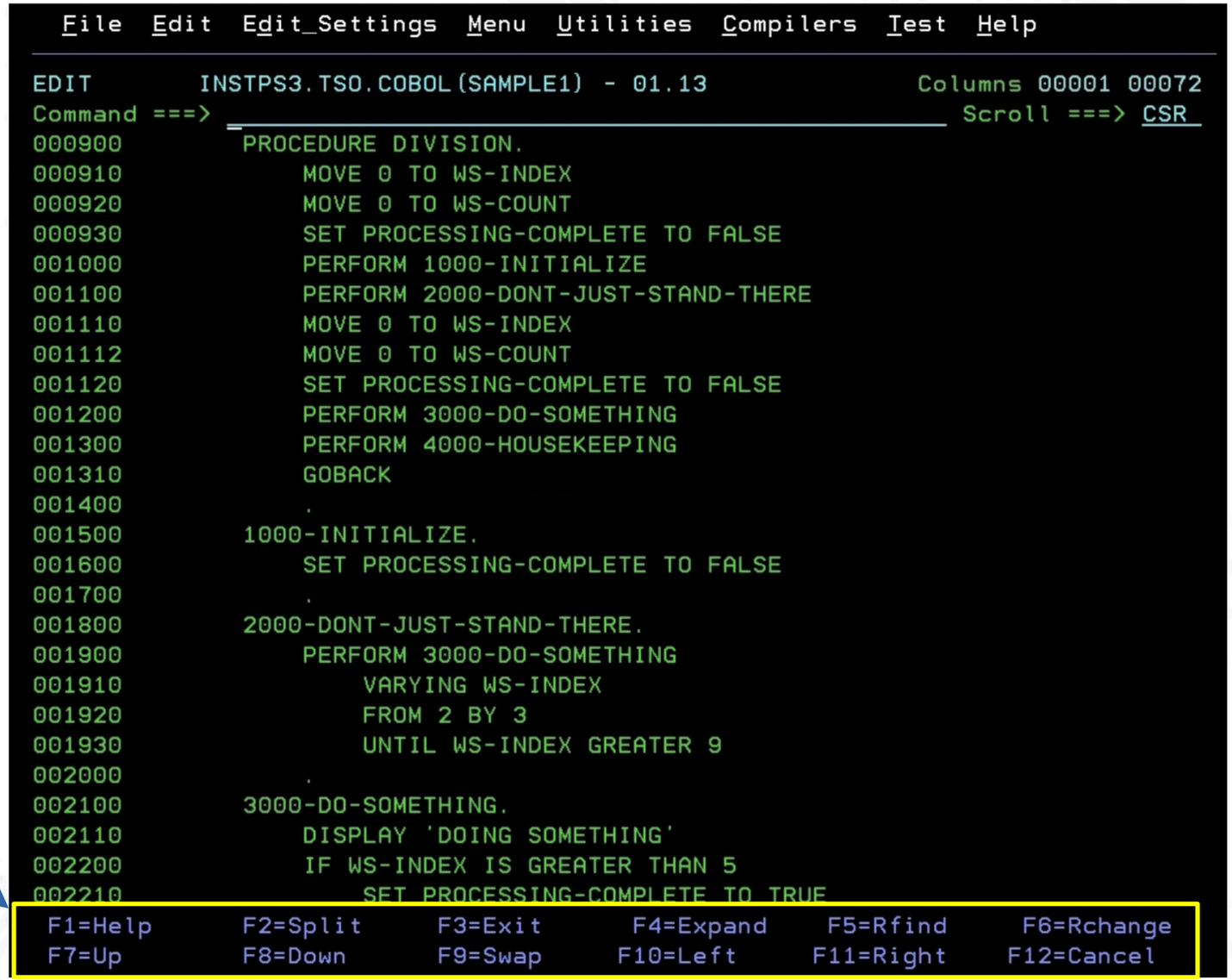
Line numbers (not part of the data).

Line command entry area.



ISPF Editor panel layout

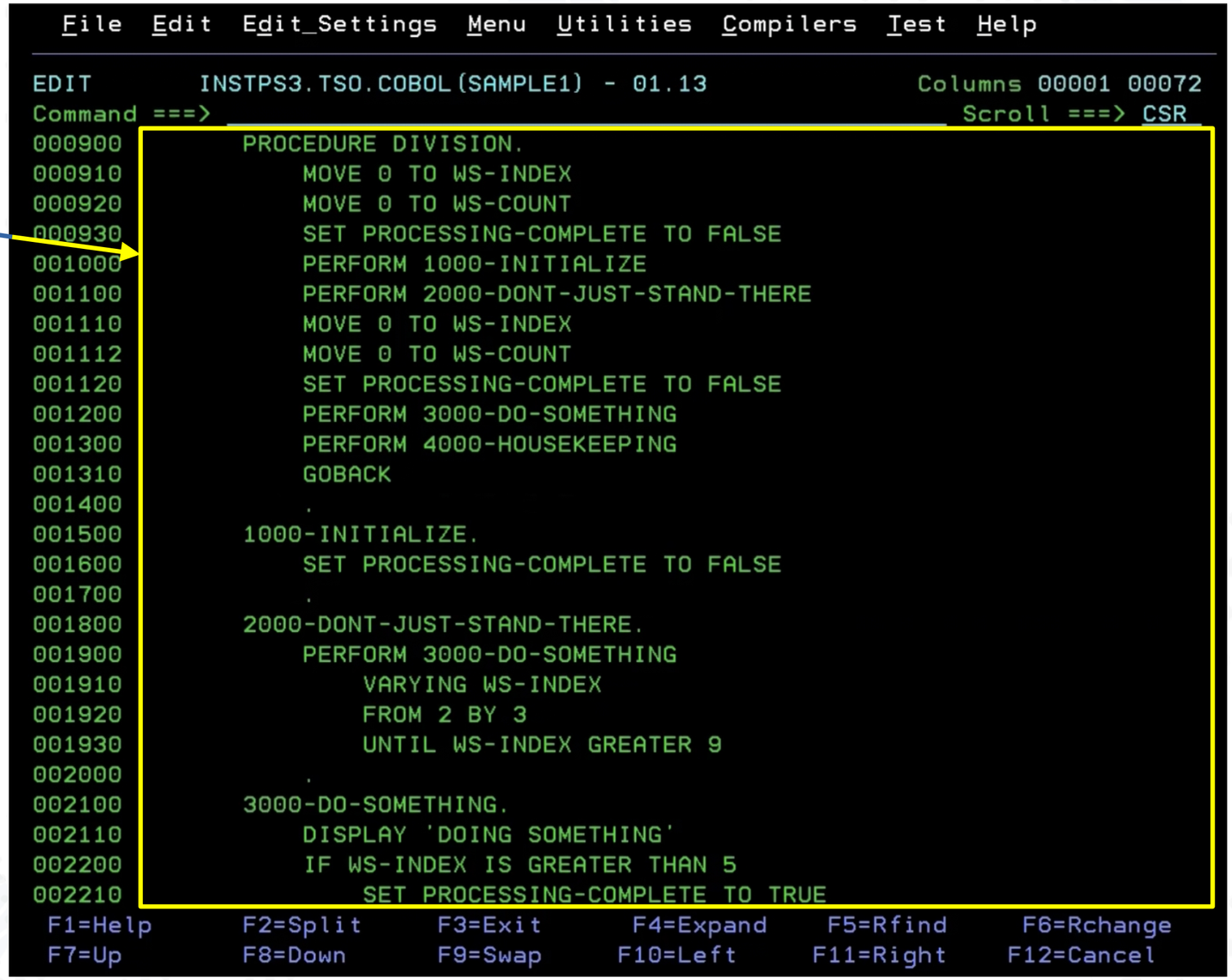
Reminder of how function keys are mapped to actions.



ISPF Editor panel layout

Editable text from the data Set.

A small view into the data set that will fit on a 3270 display.



Demo:
Basic text editing with the ISPF Editor

We'll show how to perform basic text editing operations with the ISPF Editor.

Lab 8:
Practice using the ISPF Editor

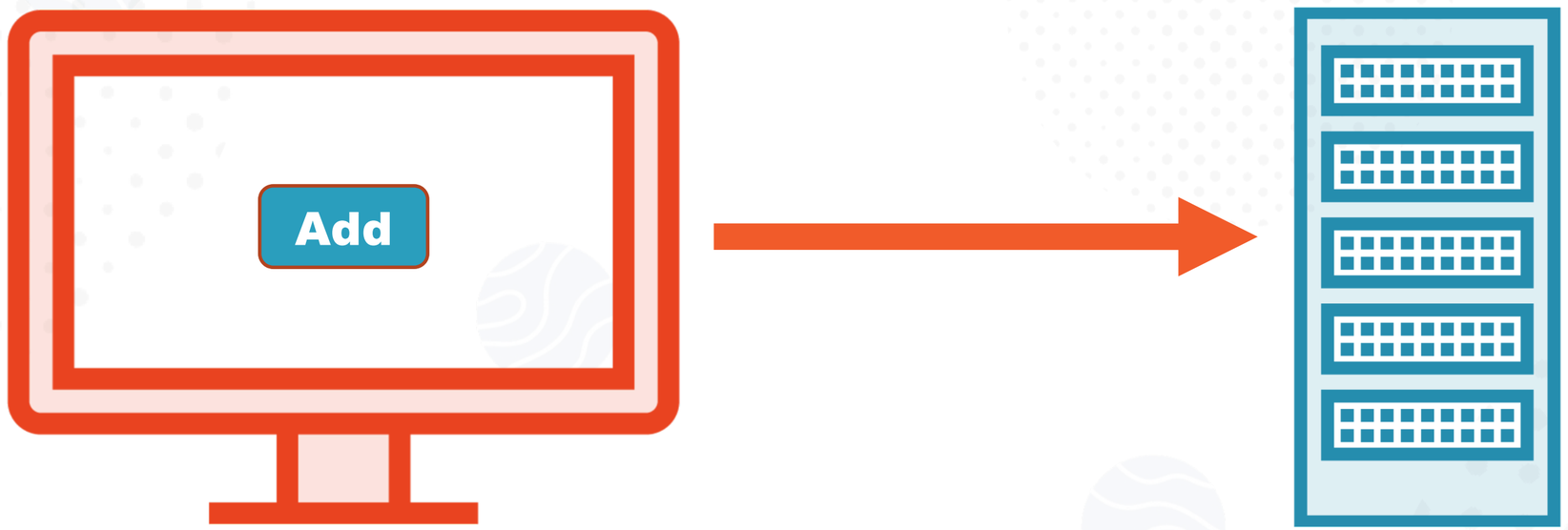
Follow the instructions for this lab in your copy of the course Github repository.

Batch and interactive processing

Dave Nicolette
Programmer

neopragma.com

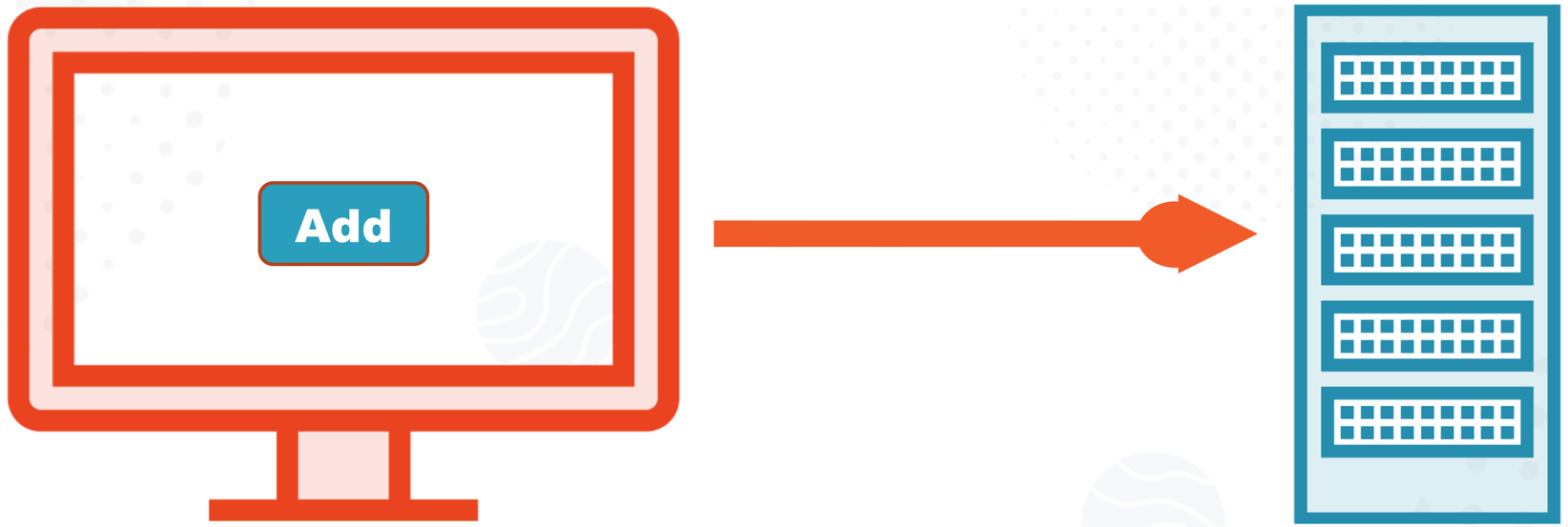
Interactive processing – what we're used to today



User Sends Request

User initiates a single transaction with the server.

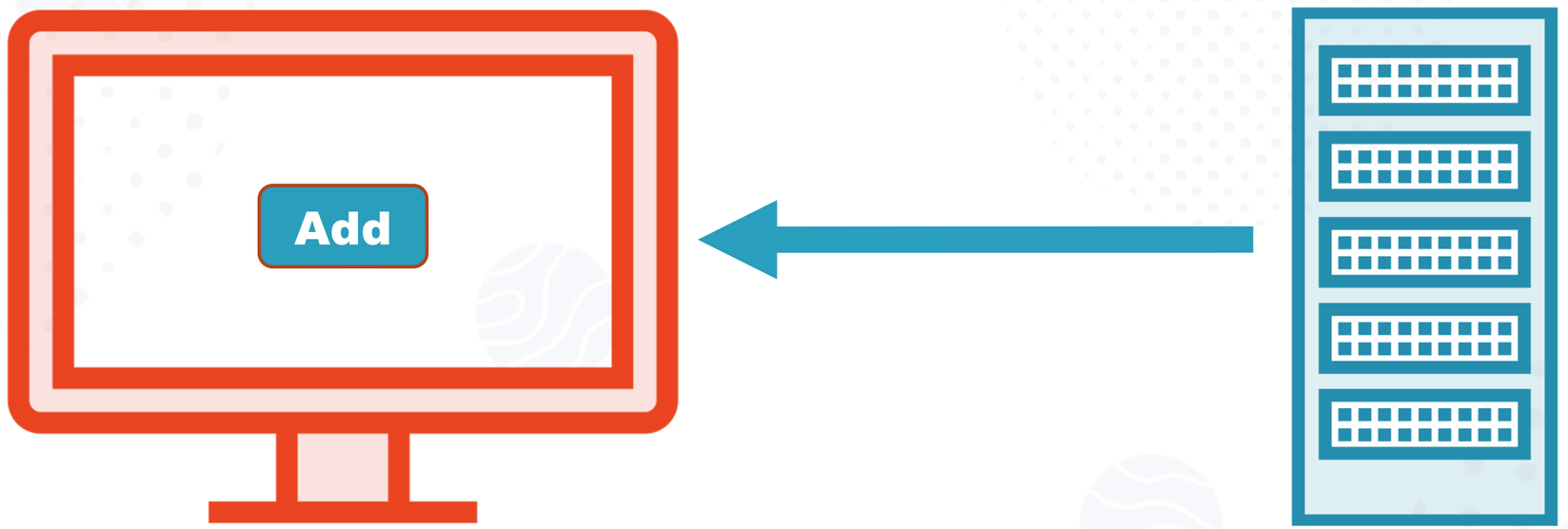
Interactive processing – what we're used to today



User Sends Request
User initiates a single transaction with the server.

Server Processes Request
Server receives the request and carries out the transaction.

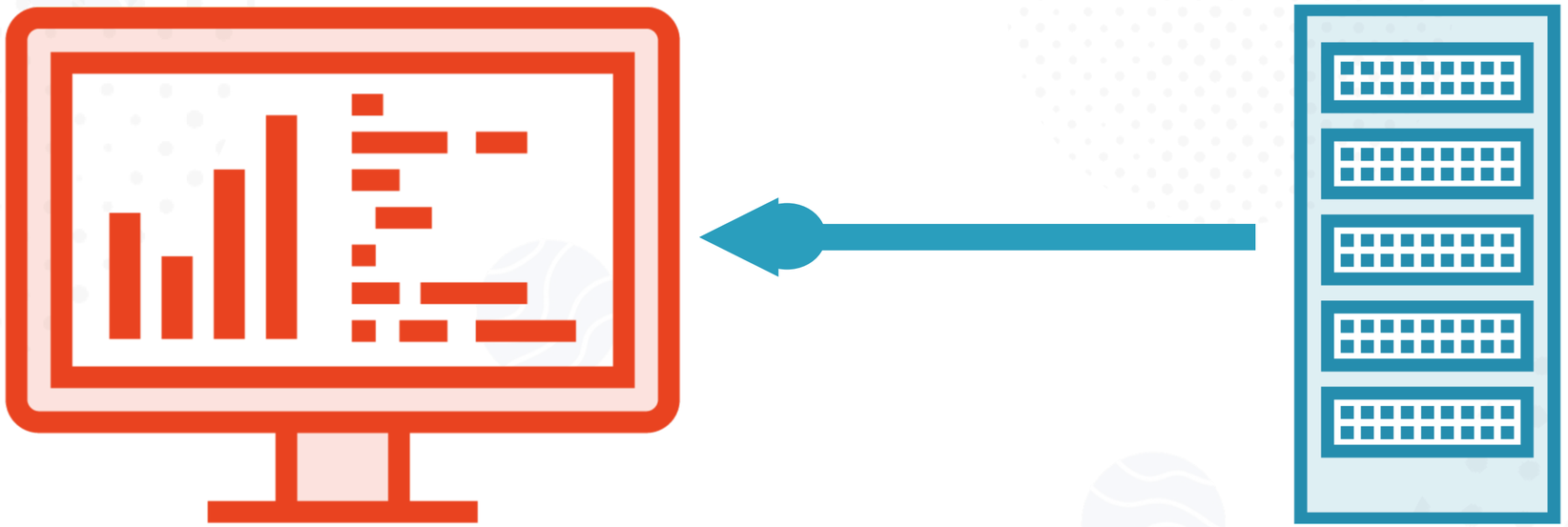
Interactive processing – what we're used to today



Server Sends Response

Server sends the result of the transaction to the user.

Interactive processing – what we're used to today



User Receives Response
User receives the result of a single transaction.

Server Sends Response
Server sends the result of the transaction to the user.

Interactive processing – command line

```
mt-cobol-labs % df -h
```

```
mt-cobol-labs % df -h
Filesystem       Size   Used  Avail Capacity iused      ifree %iused  Mounted on
/dev/disk3s1s1  1.8Ti   20Gi  1.5Ti     2%  502379 4291458807    0%   /
devfs           217Ki  217Ki    0Bi   100%     752          0  100%   /dev
/dev/disk3s6    1.8Ti   20Ki  1.5Ti     1%       0 15895084360    0%   /System/Volumes/VM
/dev/disk3s2    1.8Ti  668Mi  1.5Ti     1%    2384 15895084360    0%   /System/Volumes/Preboot
/dev/disk3s4    1.8Ti  921Mi  1.5Ti     1%     292 15895084360    0%   /System/Volumes/Update
/dev/disk1s2    500Mi  6.0Mi  481Mi     2%       1     4924960    0%   /System/Volumes/xarts
/dev/disk1s1    500Mi  6.3Mi  481Mi     2%      57     4924960    0%   /System/Volumes/iSCPreboot
/dev/disk1s3    500Mi  2.0Mi  481Mi     1%      50     4924960    0%   /System/Volumes/Hardware
/dev/disk3s5    1.8Ti  319Gi  1.5Ti    18% 3767777 15895084360    0%   /System/Volumes/Data
map auto_home     0Bi    0Bi    0Bi   100%       0          0  100%   /System/Volumes/Data/home
/dev/disk3s1    1.8Ti   20Gi  1.5Ti     2%  502390 4288556160    0%   /System/Volumes/Update/mnt1
/dev/disk4s1    1.8Ti  1.6Ti  219Gi    89%  213051  230346257    0%   /Volumes/NeoPragma
mt-cobol-labs %
```

Batch processing

System Processes Many Requests

Transactions are collected into a set, or *batch*, for processing in a single run.

Console

The original mainframes had a console comprising a modified typewriter keyboard and a line printer. The operator could enter a few commands there, to pause or shut down the system, but the console was not as capable as a Unix or Windows command line interface.

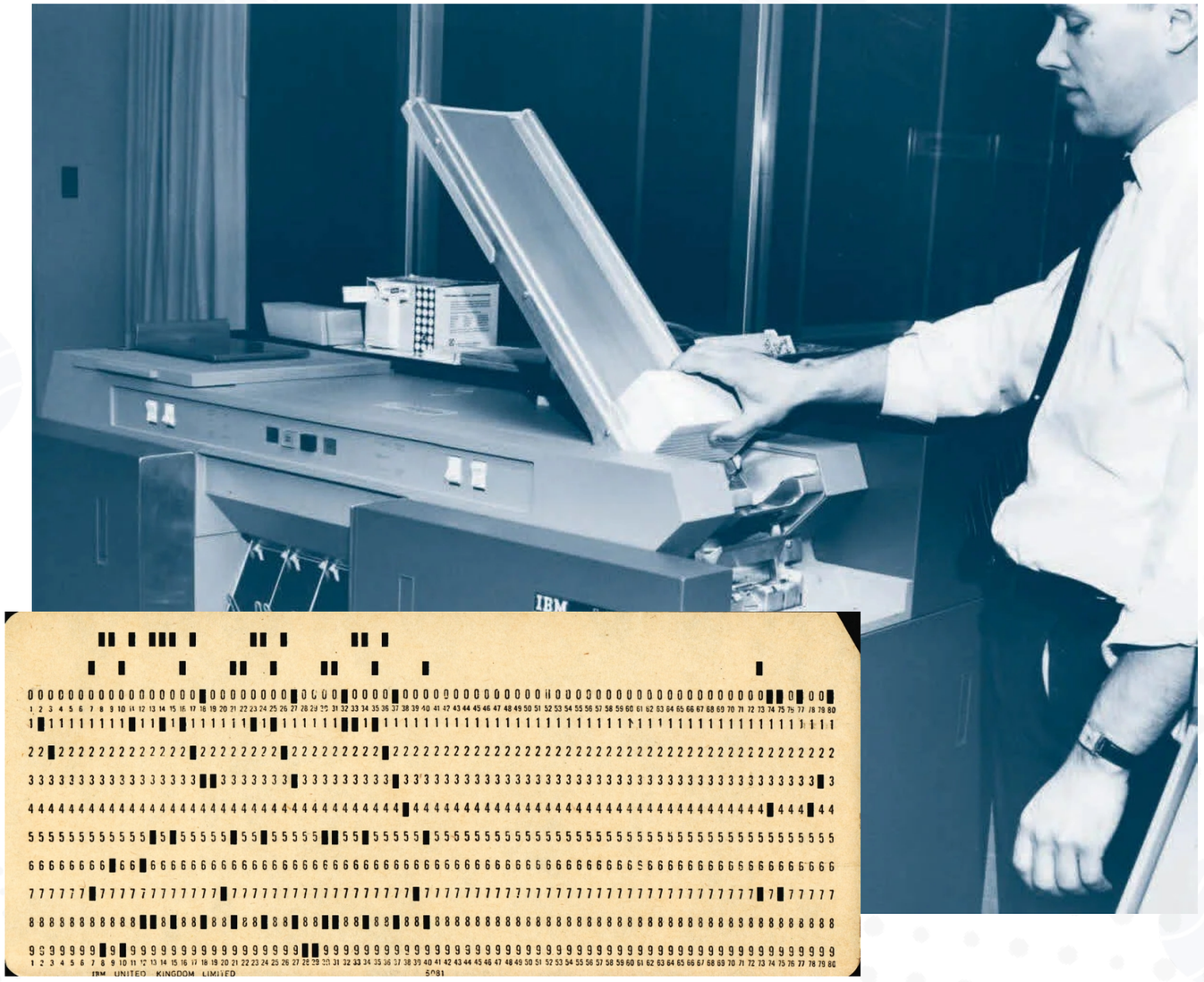
Punched cards

The primary way to load work into the system was via *punched cards*.

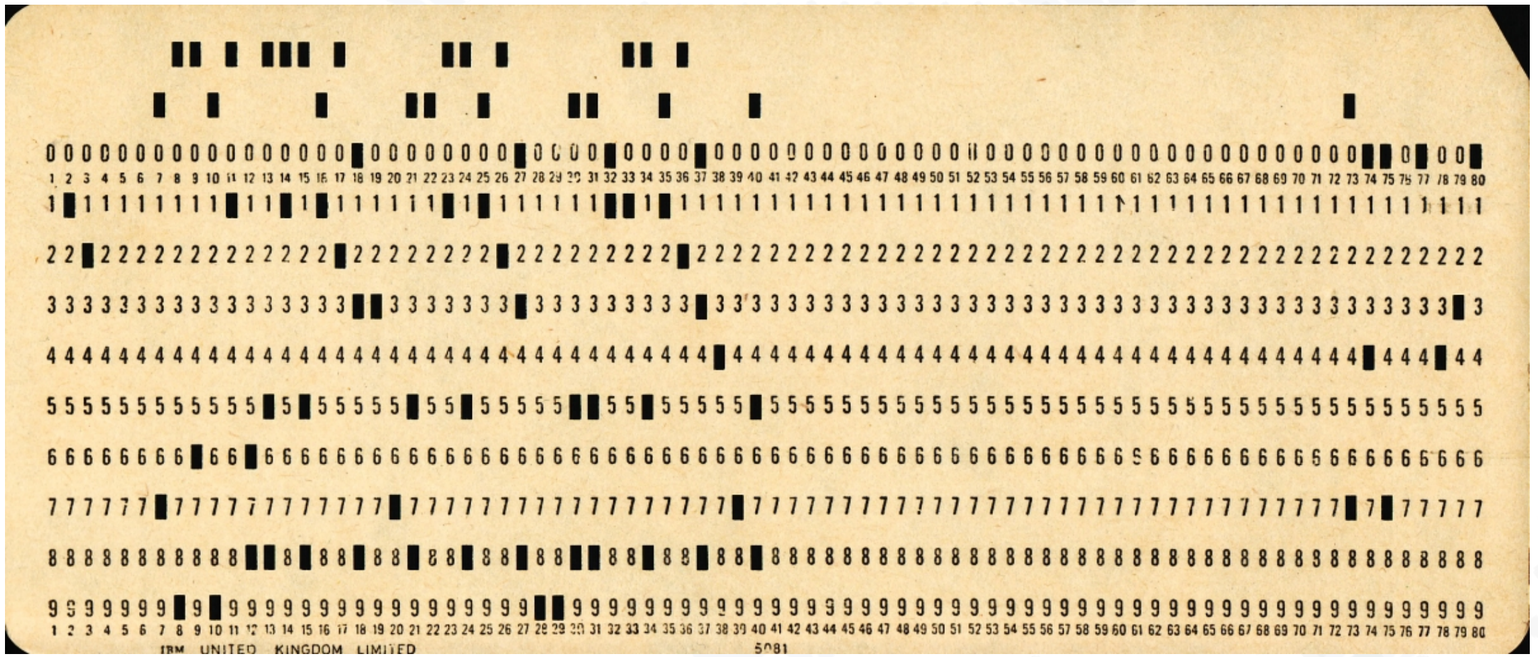
Holes punched through the cards in specific patterns represented letters, numbers, and other displayable characters.

Each card represented a single line of source code or data.

Each vertical column of holes represented one character. (We still call the character positions in a data set "columns.")



The ubiquitous 80-byte fixed-length format



Each mainframe manufacturer defined their own card size and layout. IBM cards had 80 columns. IBM won the mainframe wars, so their card size and layout prevails. The brain-in-a-jar still expects Job Control statements, program source code, configuration files, control statements for system utilities, and many other types of data sets to consist of 80-byte plain text records. We call them "card image records."

Every workload is a batch job...sort of

Mainframes were originally designed to support batch processing. Batch jobs were and are defined by *Job Control Language*, or JCL.

Other kinds of workloads are supported on top of the foundation of batch processing. Therefore, all the z/OS subsystems are started using JCL, including TSO, TCP/IP, and everything else. Long-running services like DB2 and CICS are also started using JCL. Your lab work in the class will depend heavily on JCL.

z/OS has one primary subsystem and an arbitrary number of secondary subsystems. The primary subsystem always has to be software that knows how to interpret and act on JCL. Otherwise, there would be no way to tell the system to do anything.

Therefore, the primary subsystem is always a *Job Entry Subsystem*. Currently, IBM is supporting just one product in that category: *JES2*.

Introduction to JCL

Dave Nicolette
Programmer

neopragma.com

JCL controls...

Which batch job should be run?

What program or programs should be executed?

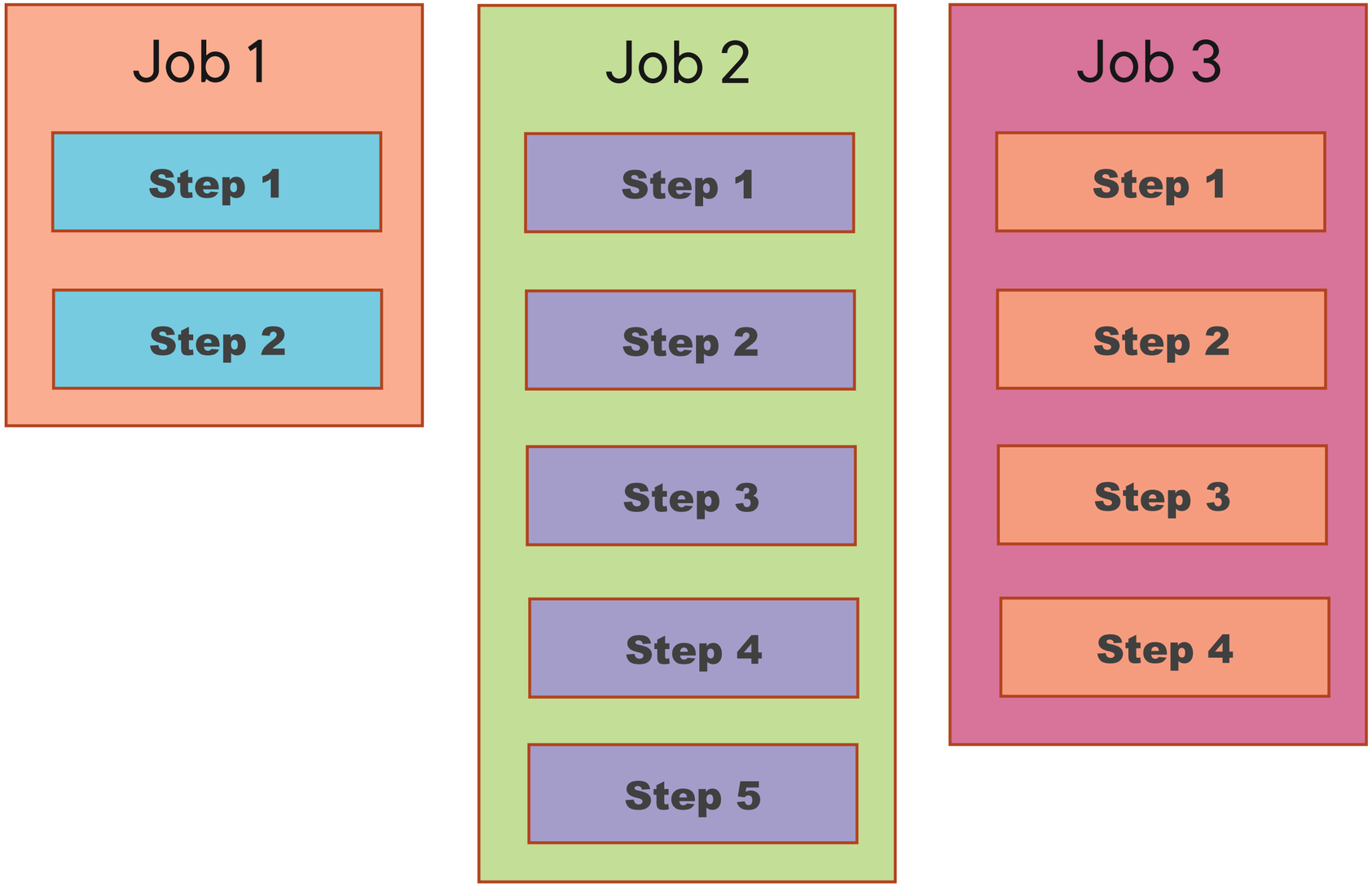
Which files or other resources do these programs need?

Structure of a job

A *job* contains one or more *steps*.

A *step* executes a *program*.

A *program* accesses data sets and/or database resources.



All the JCL statements (yes, there are only four)

Job Control Language reflects the structure of a job.

Each job in a jobstream is defined by a JOB statement.

Each step in a job is defined by an EXEC statement.

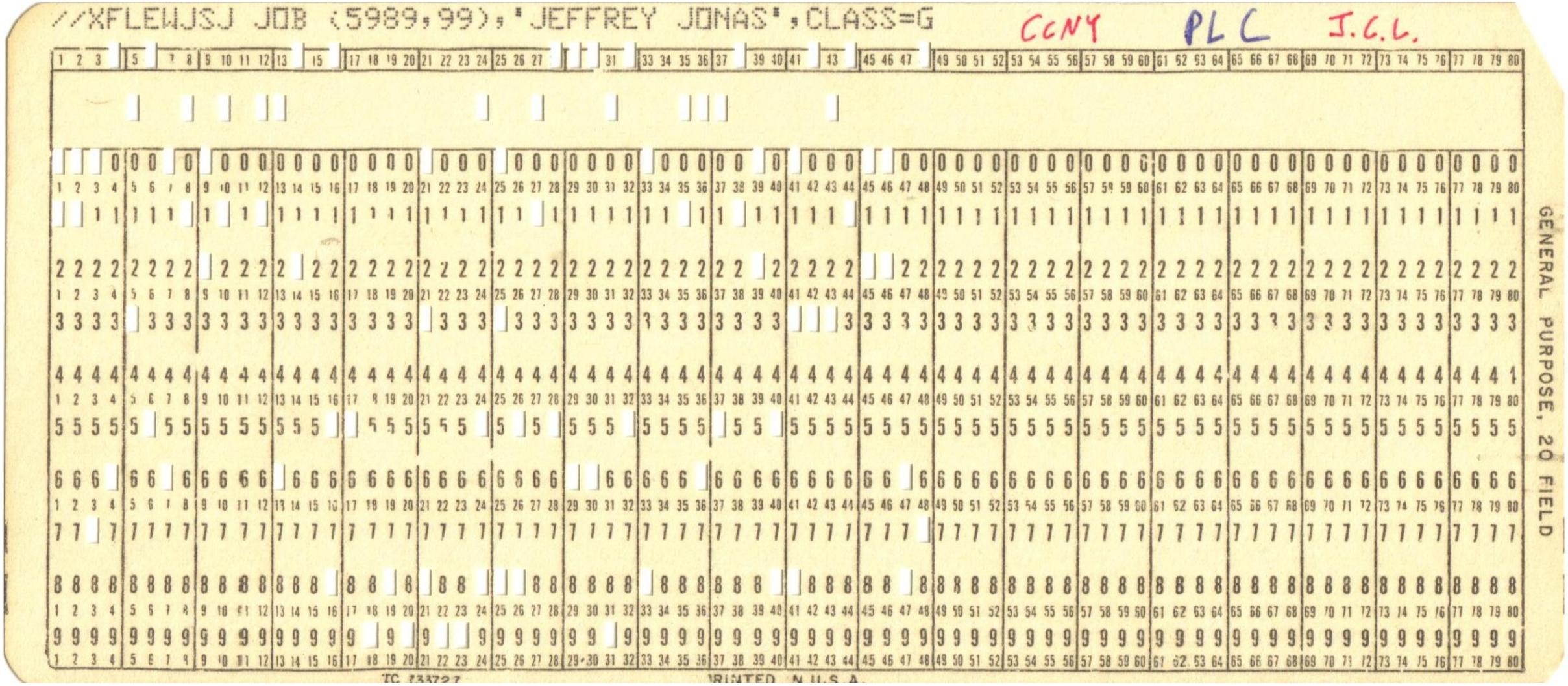
Each data set accessed in a job step is defined by a DD statement.

And you can include comments to make the purpose of things clear to other people.

```
//jobname  JOB . . .
//stepname EXEC . . .
//ddname1  DD . . .
//ddname2  DD . . .

//* This step reformats
//* the input file
//REFMT EXEC . . .
//INFILE  DD . . .
//OUTFILE DD . . .
```

A JOB statement on a punched card



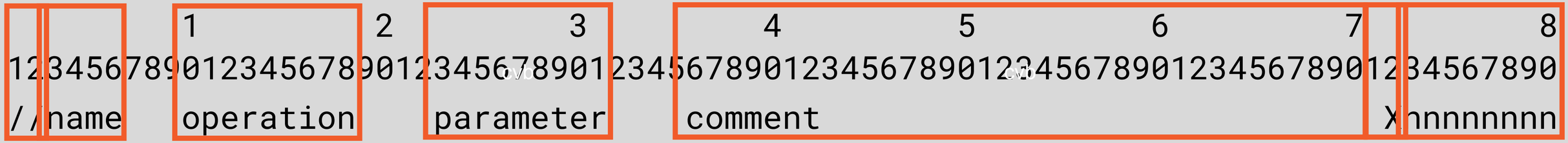
JCL statement format

Dave Nicolette
Programmer

neopragma.com

```
//CICSRUN  JOB accounting info,name,CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR, X
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

- **The first few lines of JCL to start a CICS system**
- **Each JCL statement is 80 characters long**
- **Columns 73-80 are reserved for a sequence number**
- **Certain things have to be coded in particular columns on the JCL statement**
- **Column 72 is for a continuation character**
- **Why does JCL look this way?**



JCL Statement – Fields

Identifier field, name field, operation field, parameter or operand field, comment field.

When a statement must be continued on the next line, column 72 holds the continuation character.

Columns 73 through 80 are reserved for a sequence number – not usually used anymore.

```
      1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//x
//
```

JCL Statement – Identifier Field

Two slashes followed by a space or any character *other than asterisk* (*) identifies this as a JCL statement or the continuation of a JCL statement from the previous line.

```
12345678901234567890123456789012345678901234567890123456789012345678901234567890
```

```
//*************************************************************************
//* COMMENTS IN A BOX
//*************************************************************************
```

Space or asterisk in column 4!

JCL Statement – Identifier Field

Two slashes in columns 1 and 2, asterisk (*) in column 3, *and* space or asterisk in column 4 identify this as a JCL comment line.

```
//CICSRUN  JOB accounting info,name,CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

- **Slash slash character other than asterisk: JCL statement with a name**
- **Slash, asterisk, non-space in 1-3: It's a JES2 statement**
- **No slash in column 1: It's in-stream data**
- **Slash, asterisk, space in 1-3: End of in-stream data**

```
//CICSRUN  JOB accounting info,name,CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

- **Job name: CICSRUN**
- **Step name: CICS**
- **DD name: SYSIN**

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//name      operation    parameter    comment
```

Rules for Names

The name must begin in column 3, immediately after the two slashes, with no intervening spaces.

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//A         operation    parameter    comment
//ABCDEFGH  operation    parameter    comment
```

Rules for Names

The name must consist of between 1 and 8 characters.

```
      1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//A         operation    parameter    comment
//ABCDEFGH  operation    parameter    comment
//$JNAME    operation    parameter    comment
//#JB@23    operation    parameter    comment
```

Rules for Names

You can use characters from ISO-8859-1 including letters, numbers, and the characters $, #, and @.

```
      1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//STEPLIB  DD     parameter     comment
//SYSPRINT DD     parameter     comment
//SYSUDUMP DD     parameter     comment
//etc.     DD     parameter     comment
```

Rules for Names

Some DD names are have special meaning to the system. Don't use them for your own datasets.

Reserved DD names are documented here: https://www.ibm.com/docs/en/zos-basic-skills?topic=jdsausn-jcl-dd-statement-ddnames-that-are-reserved-specific-uses

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//MYFILE   DD    DSN=first.in.concat,DISP=SHR
//         DD    DSN=second.in.concat,DISP=SHR
//         DD    DSN=third.in.concat,DISP=SHR
//         DD   DSN=fourth.in.concat,DISP=SHR
```

Rules for Names

When concatenating datasets under the same DDNAME, code the name only on the 1st DD statement.

```
//CICSRUN  JOB accounting info,name,CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS     EXEC PGM=DFHSIP,REGION=240M,
//              PARM=('SIT=6$',
//              'DSALIM=6M,EDSALIM=120M',
//              'RENTPGM=PROTECT,STGPROT=YES',
//              'START=AUTO,SI')
//SYSIN     DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF   DD SYSOUT=*
//LOGUSR    DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR    DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT      DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG    DD SYSOUT=A
//CEEOUT    DD SYSOUT=A
//DFHLCD    DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//             AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF   DD SYSOUT=A
etc.
```

- Operation field: JOB
- Operation field: EXEC
- Operation field: DD

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

◀ **JOB Parameters**

◀ **Step Parameters**

◀ **DD Parameters**

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',

//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

I HAVE A FEW COMMENTS ABOUT THIS JOB.

◂ Comments

Continuing a JCL statement

Dave Nicolette
Programmer

neopragma.com

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

◂Parameters on the EXEC statement

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

- **Break the line after at least one parameter value – not before the parameters begin**

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

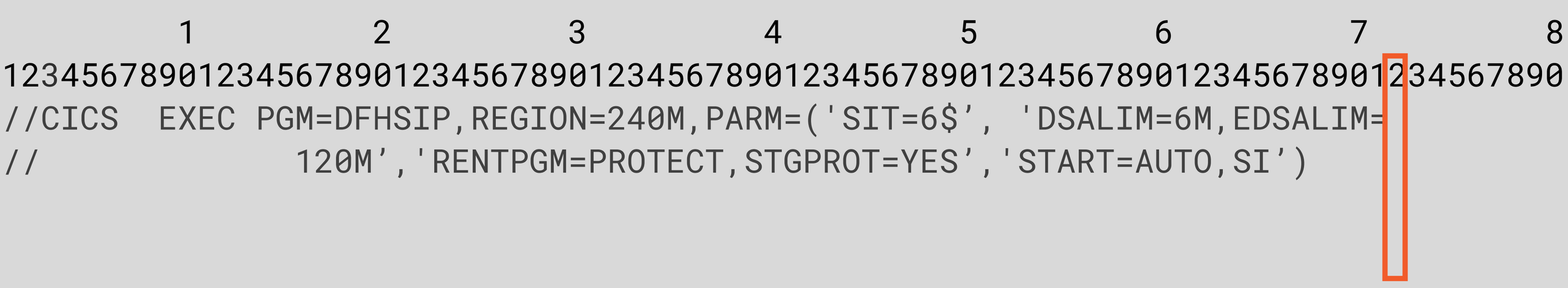
- **Break the line after the comma that separates one parameter from the next**

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

- **Start each continuation line with two slashes followed by at least one space.**

```
//CICSRUN JOB accounting,'NAME',CLASS=A,
//            MSGCLASS=A,MSGLEVEL=(1,1),NOTIFY=userid
/*JOBPARM SYSAFF=sysid
//CICS    EXEC PGM=DFHSIP,REGION=240M,
//             PARM=('SIT=6$',
//             'DSALIM=6M,EDSALIM=120M',
//             'RENTPGM=PROTECT,STGPROT=YES',
//             'START=AUTO,SI')
//SYSIN    DD *
GRPLIST=(DFHLIST,userlist1,userlist2),
LPA=YES,
APPLID=CICSHTH1,
DFLTUSER=CICSUSER,
MXT=30,
INITPARM=(DFHDBCON='01',DFHD2INI=('MYDB')),
ISC=YES,
IRCSTRT=YES,
.END
/*
//DFHCXRF  DD SYSOUT=*
//LOGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=136)
//MSGUSR   DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=140)
//COUT     DD SYSOUT=*,DCB=(DSORG=PS,RECFM=V,BLKSIZE=137)
//CEEMSG   DD SYSOUT=A
//CEEOUT   DD SYSOUT=A
//DFHLCD   DD DSN=CICSTS55.CICS.CICSHTH1.DFHLCD,DISP=SHR
//DFHGCD  DD DSN=CICSTS55.CICS.CICSHTH1.DFHGCD,DISP=SHR,
//            AMP=('BUFND=33,BUFNI=32,BUFSP=1114112')
//DFHCXRF  DD SYSOUT=A
etc.
```

- **Resume the continued text between columns 4 and 16 – not after column 16**



Continuing a line in the middle of a quoted value

Continue the quoted value through column 71. Do not code anything in column 72.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//CICS  EXEC PGM=DFHSIP,REGION=240M,PARM=('SIT=6$’, 'DSALIM=6M,EDSALIM=
//                120M’,'RENTPGM=PROTECT,STGPROT=YES’,'START=AUTO,SI’)
```

Continuing a line in the middle of a quoted value

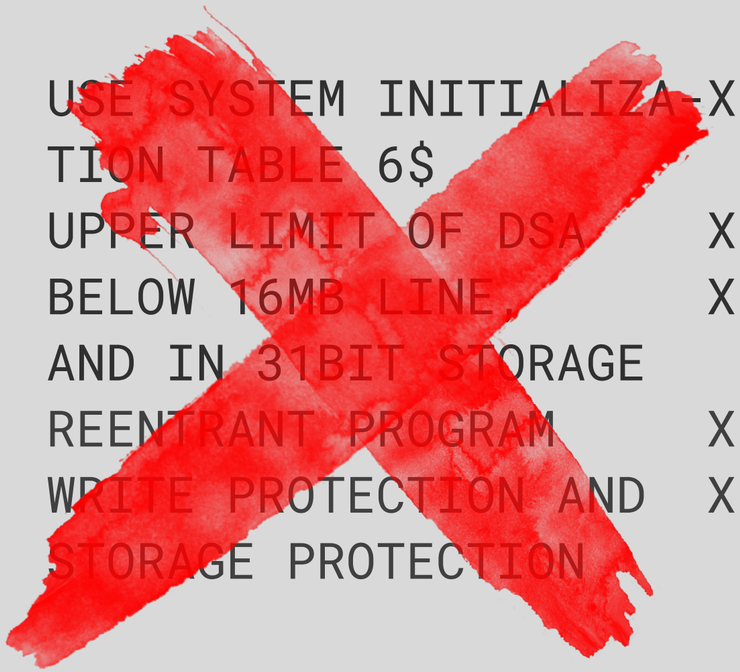
Start the continuation line with two slashes in columns 1 and 2.
Resume the quoted value exactly in column 16.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//CICS EXEC PGM=DFHSIP,REGION=240M,
//          PARM=('SIT=6$’,                          USE SYSTEM INIT TBL 6$
//          'DSALIM=6M,EDSALIM=120M’,                DSA MAX 31BIT WAS 800
//          'RENTPGM=PROTECT,STGPROT=YES’,           WRITE/STG PROTECTION
//          'START=COLD')                 AUTO,SI’)  COLD START BECAUSE OF X
//                                                   WHAT HAPPENED YESTERDAY
//                                                   RANDALL SAYS HIS DOG  X
//                                                   ATE THE RECOVERY MANAGX
//                                                   ER CONTROL RECORD.    X
//                                                   THIRD TIME THIS WEEK.
```

Continuing a comment that follows parameter values

Continue the quoted value through column 71.
Code a non-blank character in column 72.
Start subsequent lines with two slashes.
Leave column 3 blank.
Continue the comment text anywhere after column 3, through column 71

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//CICS EXEC PGM=DFHSIP,REGION=240M,
//          PARM=('SIT=6$’,                    USE SYSTEM INITIALIZA-X
//                                             TION TABLE 6$
//          'DSALIM=6M,EDSALIM=120M’,          UPPER LIMIT OF DSA     X
//                                             BELOW 16MB LINE,       X
//                                             AND IN 31BIT STORAGE
//          'RENTPGM=PROTECT,STGPROT=YES’,     REENTRANT PROGRAM      X
//                                             WRITE PROTECTION AND   X
//                                             STORAGE PROTECTION
//          'START=AUTO,SI’)
```



Don't continue both the parameter list and the comments

The system will honor the parameter continuation and ignore the comment continuation.
You will probably get a JCL error.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//*************************************************************
//* USING SIT TABLE 6$.
//* UPPER LIMIT FOR DSA ABOVE THE LINE WAS ORIGINALLY 800M.
//* REENTRANT PROGRAM WRITE PROTECTION YES.
//* STORAGE PROTECTION YES.
//*************************************************************
//CICS EXEC PGM=DFHSIP,REGION=240M,
//          PARM=('SIT=6$’,
//          'DSALIM=6M,EDSALIM=120M’,
//          'RENTPGM=PROTECT,STGPROT=YES’,
//          'START=AUTO,SI’) AUTO START NOTWITHSTANDING RANDALL'S DOG
```

If you need lots of comments, code them on separate lines.

It's more readable. Comments on the right of each line are okay if they are very short.

The JOB statement

Dave Nicolette
Programmer

neopragma.com

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//AYT289X  JOB 599,'A. YAN',CLASS=A,MSGCLASS=A,MSGLEVEL=(2,1),TIME=1440,
//             NOTIFY=&SYSUID
// . . .
```

JOB Statement with Single-Value Job Accounting Parameter

Job accounting parameter has a single value.
Programmer name parameter is used.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//GMT443N JOB (599,'T/HK'),'GENNA',CLASS=A,MSGCLASS=A,MSGLEVEL=(1,1),
//             NOTIFY=&SYSUID
// . . .
```

JOB Statement with Two-Value Job Accounting Parameter

Job accounting parameter has two comma-delimited fields, all enclosed in parentheses.
Programmer name parameter is used.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//GMT443N JOB (599,'T/HK'),'GENNA',CLASS=A,MSGCLASS=A,MSGLEVEL=(1,1),
//             NOTIFY=&SYSUID
// . . .
```

Programmer Name Parameter

The programmer name parameter is optional as far as the system is concerned, but may be required in your organization.
The length of the value is limited to 20 characters.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//PO117A JOB ,'ORLOVSKY',CLASS=A,MSGCLASS=A,MSGLEVEL=(1,1),
//             NOTIFY=&SYSUID
// . . .
```

JOB Statement with No Job Accounting Parameter

Placeholder is coded for the omitted job accounting parameter.
Programmer name parameter is used.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//JKT308B JOB CLASS=A,MSGCLASS=Q,MSGLEVEL=(1,1),NOTIFY=&SYSUID
// . . .
```

JOB Statement with Neither Job Accounting Nor Programmer Name

No placeholders are coded for the omitted positional parameters.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//JKT308B JOB CLASS=A,MSGCLASS=Q,MSGLEVEL=(1,1),NOTIFY=&SYSUID
// . . .
```

NOTIFY Parameter Specifying the &SYSUID Symbol

JES will replace &SYSUID with the user id of the account that submitted the job.
When the job starts and when it ends, JES will send a notification message *via* TSO to that account.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//JKT308B JOB CLASS=A,MSGCLASS=Q,MSGLEVEL=(1,1),NOTIFY=&SYSUID
// . . .
```

CLASS Parameter Specifying Predefined Value "A"

JES will handle job initiation according to the configuration defined for job class A.
This is a system configuration setting not under control of application developers.
You will be told which job classes to use for each type of job you work with.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//JKT308B JOB CLASS=A,MSGCLASS=Q,MSGLEVEL=(1,1),NOTIFY=&SYSUID
// . . .
```

MSGLEVEL Parameter

Controls which output datasets you want to be generated in the job log.

`MSGLEVEL=(`*`statements`*`,`*`messages`*`)`

statements

0 = Only the JOB statement

1 = JCL & procedure statements

2 = Only JCL statements

messages

0 = Only JCL messages

1 = JCL, JES, and operator messages

```
//xxx JOB ... MSGLEVEL=(0,0)
```

- **Output the JOB statement and JCL messages**

```
//xxx JOB ... MSGLEVEL=(1,1)
```

- **Output all JCL and procedure statements and all JCL, JES, and operator message (typical)**

```
//xxx JOB ... MSGLEVEL=(2,0)
```

- **Output only JCL statements and only JCL messages**

```
       1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//JKT308B JOB CLASS=A,MSGCLASS=Q,MSGLEVEL=(1,1),NOTIFY=&SYSUID
// . . .
```

MSGCLASS Parameter Specifying Predefined Value "Q"

JES will handle job log output according to the configuration defined for message class A.
This is a system configuration setting not under control of application developers.
You will be told which message classes to use for each type of job you work with.

Demo:
Coding a JOB statement

We'll show how to code a JOB statement that complies with the organization's standards.

Then we'll run the job and see it fail. We'll locate the job output on the system and examine it.

Lab 9:
A simple batch job (that won't run)

Follow the instructions for this lab in your copy of the course Github repository.

The EXEC statement

Dave Nicolette
Programmer

neopragma.com

```
//xxx          JOB  ...
```

◂**A job with no steps isn't very useful**

```
//xxx       JOB ...
//STEP1     EXEC ...
//STEP2     EXEC ...
//STEP3     EXEC ...
//STEP4     EXEC ...
//STEP5     EXEC ...
//STEP6     EXEC ...
//STEP7     EXEC ...
//STEP8     EXEC ...
//STEP9     EXEC ...
//STEP10    EXEC ...
//STEP11    EXEC ...
//STEP12    EXEC ...
//STEP251   EXEC ...
//STEP252   EXEC ...
//STEP253   EXEC ...
//STEP254   EXEC ...
//STEP255   EXEC ...
```

◂**A job must have at least one step…**

◂**…and *can* have up to 255 steps.**

```
//STEP1 EXEC PGM=program
```

- **PGM= names an executable to be run.**

```
//STEP2 EXEC PROC=procedure
```

- **PROC= names a catalogued procedure to be inserted into the jobstream at this point. The procedure may contain multiple steps.**

```
//STEP3 EXEC procedure
```

- **PROC= is the default, so you can omit the keyword.**

Demo:
The do-nothing utility IEFBR14

We'll show how to code an EXEC statement that runs the IBM utility IEFBR14. This will be enough to make our simple job run.

What is the point of a do-nothing utility?

The Job Entry Subsystem acts on DD statements even if the program named in the EXEC statement doesn't refer to those ddnames.

We must have an EXEC statement even if we don't need the program to do any work. That's the purpose of IEFBR14.

Jobs like the one at right are very common. The example deletes a data set and then allocates it again, to make sure it's empty.

You'll soon learn about the DISP parameter on the DD statement, which controls what happens to the data set at the end of the job step.

```
//RESET   JOB ,'BOB',CLASS=A,MSGCLASS=A,
//            MSGLEVEL=(1,1),NOTIFY=&SYSUID
//DELETE EXEC PGM=IEFBR14
//SYSOUT   DD SYSOUT=*
//DD1      DD DSN=HLQ1.MLQ1.FILE1,
//            DISP=(MOD,DELETE,DELETE),
//            SPACE=xxx,UNIT=xxx
//CREATE EXEC PGM=IEFBR14
//DD1      DD DSN=HLQ.MLQ1.FILE1,
//            DISP=(NEW,CATLG,DELETE),
//            SPACE=xxx,UNIT=xxx
```

Lab 10:
The minimum batch job that will run

Follow the instructions for this lab in your copy of the course Github repository.

The DD statement

Dave Nicolette
Programmer

neopragma.com

Data Definition (DD) JCL statement

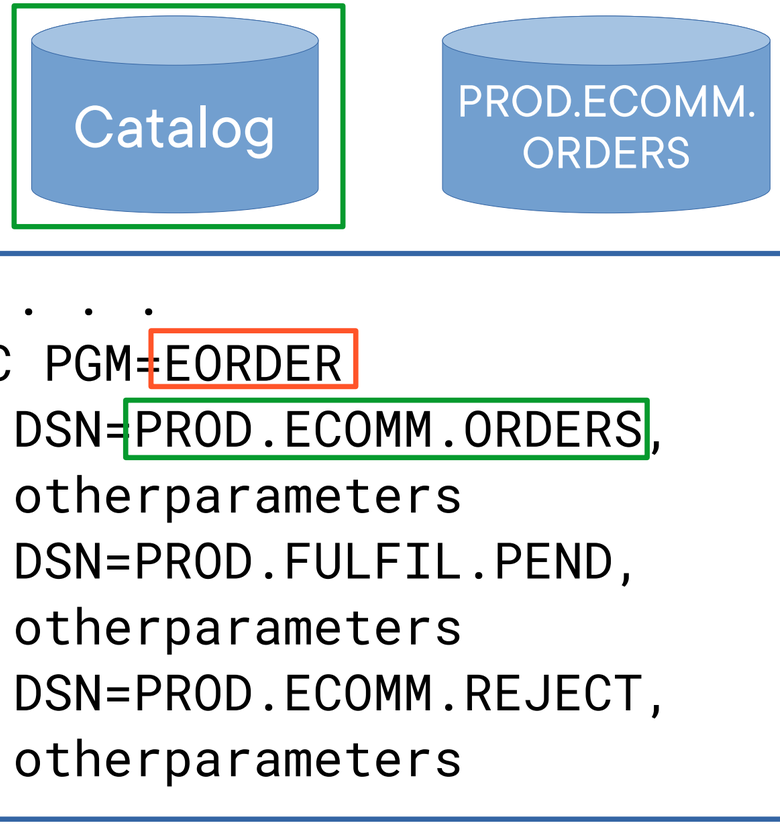
A DD statement describes a data set that is used in the current job step. It provides the connection between the *catalog* and the *program* that runs in the step.

Typically, application programs don't know the actual data set name. They only know the name coded on the DD statement in the JCL.

The catalog locates the data set based on its data set name. It knows nothing about DD statements.

The DD statement contains both the DDNAME (the name on the DD statement and in the program) and the DSNAME (the data set name stored in the catalog.

The scope of a DD statement is the job step where it is specified (the EXEC statement). Therefore, DDNAMEs must be unique within a job step, but not necessarily throughout the entire job.



- Program EORDER knows the DDNAME, "ORDERS".
- The catalog knows the DSNAME, "PROD.ECOMM.ORDERS".
- The DD statement knows both names.

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD1    DD    DSN=ALPHA
//DD2    DD    DSNAME=$FRED
//DD3    DD    DSN=AMBER#45
//DD4    DD    DSN=EM-PHATC
//DD5    DD    DSN=##TG@313
//  . . .
```

Unqualified Data Set Name

Between 1 and 8 characters selected from alphanumeric, national ($, #, @), the hyphen (-) and the character represented by X'C0'.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD1   DD   DSN=ALPHA.ONE                                 9 CHARS, OK
//DD2   DD   DSNAME=EVERYONE.LIKES.CHEESE                  21 CHARS, OK
//DD3   DD   DSN=AMBER#45.TEST.COBOL.SOURCE                26 CHARS, OK
//DD4   DD   DSN=ACCT.MONTHLY.REPORTS.JANUARY.2022(0)  GDG 33 CHARS, OK
//DD5   DD   DSN=##TG@313.OBVIOUSLY.WILL.NOT.WORK      "OBVIOUSLY" > 8
//* Next DD statement - name is longer than 44 characters
//DD6   DD   DSN=THIS.NAME.IS.DEFIN.ITELY.TOO.LONG.TO.WORK.PROPER.LY
//  . . .
```

Qualified Data Set Name

Multiple unqualified names connected by periods.
Length must not exceed 44 characters, including periods

The Disposition (DISP) parameter

```
{DISP=[status]}
{DISP=[status][,normal-disp][,abnormal-disp]}

DISP=( [NEW]          [,DELETE]      [,DELETE] )
       [OLD]          [,KEEP]        [,KEEP]
       [SHR|SHARE]    [,PASS]        [,CATLG]
       [MOD]          [,CATLG]       [,UNCATLG]
       [,   ]         [,UNCATLG]
                      [,     ]
```

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD1 DD DSN=...,DISP=NEW
//DD1 DD DSN=...,DISP=(NEW,DELETE,DELETE)         <= same
//*
//DD2 DD DSN=...,DISP=(NEW,DELETE)
//DD2 DD DSN=...,DISP=(NEW,DELETE,DELETE)         <= same
//*
//DD3 DD DSN=...,DISP=(NEW,KEEP)
//DD3 DD DSN=...,DISP=(NEW,KEEP,KEEP)             <= same
```

DD DISP Examples (1)

For status NEW, in most cases the default abnormal disposition is the same as the normal disposition.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD4 DD DSN=...,DISP=(NEW,CATLG)
//DD4 DD DSN=...,DISP=(NEW,CATLG,CATLG)          <= same
//*
//DD5 DD DSN=...,DISP=(NEW,PASS)
//DD5 DD DSN=...,DISP=(NEW,PASS)                 <= same
//*
//DD6 DD DSN=...,DISP=(NEW,KEEP)
//DD6 DD DSN=...,DISP=(NEW,KEEP,KEEP)            <= same
```

DD DISP Examples (2)

For status NEW, in most cases the default abnormal disposition is the same as the normal disposition.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD7 DD DSN=...,DISP=(NEW,PASS,DELETE)
//DD8 DD DSN=...,DISP=(NEW,PASS,KEEP)
//DD9 DD DSN=...,DISP=(NEW,PASS,CATLG)
//DD10 DD DSN=...,DISP=(NEW,PASS,UNCATLG)
```

DD DISP Examples (3)

For status NEW and normal disposition PASS, if all job steps terminate normally (condition code zero) the data set is deleted at the end of the last job step.
If any steps fail, the value of the third subparameter is honored.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD11 DD DSN=...,DISP=OLD|SHR|MOD
//DD11 DD DSN=...,DISP=(OLD|SHR|MOD,KEEP,KEEP)        <= same
//*
//DD12 DD DSN=...,DISP=(OLD|SHR|MOD,KEEP)
//DD12 DD DSN=...,DISP=(OLD|SHR|MOD,KEEP,KEEP)        <= same
//*
//DD13 DD DSN=...,DISP=(OLD|SHR|MOD,DELETE)
//DD13 DD DSN=...,DISP=(OLD|SHR|MOD,DELETE,DELETE)    <= same
```

DD DISP Examples (4)

For existing datasets, the default disposition is KEEP.
If normal disposition is coded, the default abnormal disposition is the same as the normal disposition.
There are special cases – out of scope for this course.

```
        1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD14 DD DSN=...,DISP=(OLD|SHR|MOD,CATLG)
//DD14 DD DSN=...,DISP=(OLD|SHR|MOD,CATLG,CATLG)       <= same
//*
//DD15 DD DSN=...,DISP=(OLD|SHR|MOD,UNCATLG)
//DD15 DD DSN=...,DISP=(OLD|SHR|MOD,UNCATLG,UNCATLG)   <= same
//*
//DD16 DD DSN=...,DISP=(OLD|SHR|MOD,PASS)
//DD16 DD DSN=...,DISP=(OLD|SHR|MOD,PASS,PASS)         <= same
```

DD DISP Examples (5)

For existing datasets, the default disposition is KEEP.
If normal disposition is coded, the default abnormal disposition is the same as the normal disposition.
There are special cases – out of scope for this course.

```
       1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
//DD17 DD DSN=...
//DD17 DD DSN=...,DISP=(NEW,DELETE,DELETE)       <= same (no DISP at all)
//*
//DD18 DD DSN=...,DISP=(,KEEP,DELETE)
//DD18 DD DSN=...,DISP=(NEW,KEEP,DELETE)         <= same
//*
//DD19 DD DSN=...,DISP=(OLD,,DELETE)
//DD19 DD DSN=...,DISP=(OLD,KEEP,DELETE)         <= same
//*
```

DD DISP Examples (6) – Default Values

DD17. If DISP is omitted altogether, the default is (NEW,DELETE,DELETE).
DD18. If the status subparameter is omitted, the default is NEW.
DD19. For existing datasets, the default disposition is KEEP.

DD statement parameters for QSAM data sets

Dave Nicolette
Programmer

neopragma.com

DD statement parameters for QSAM data sets

SPACE

How to define the size of primary and secondary extents

DISP

How to specify the level of control needed by the step, and what to do with the data set after the step

DSORG

How to specify the organization of the data set

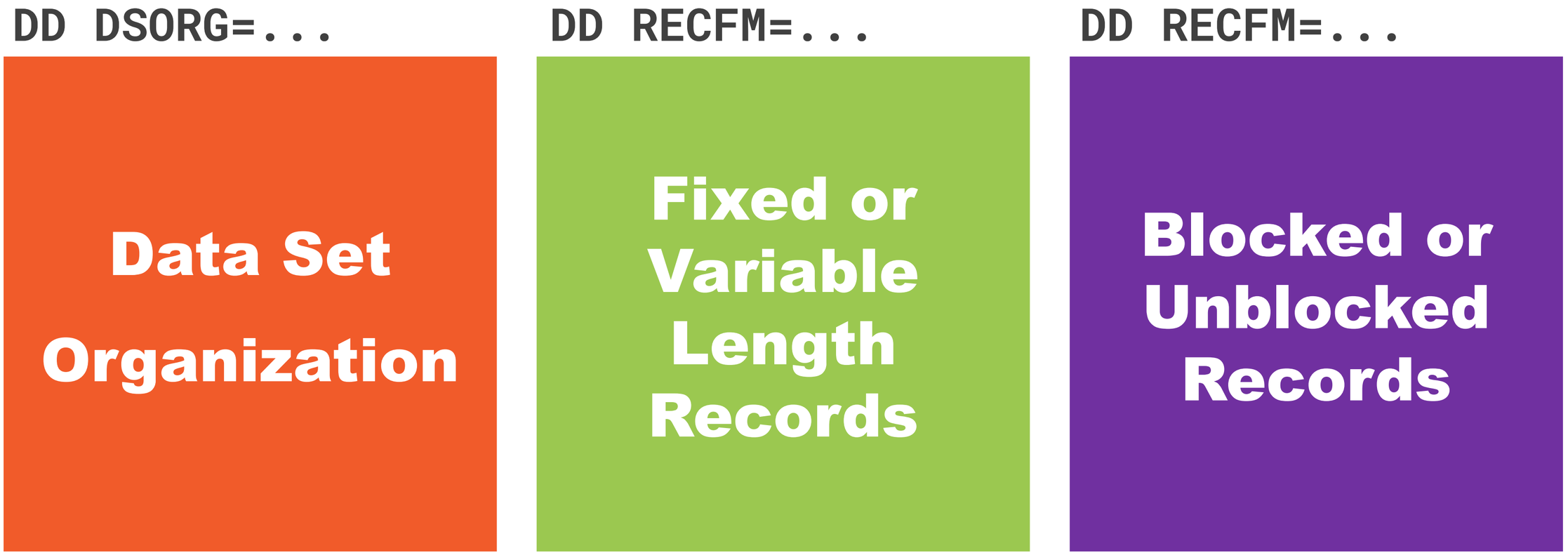
DSNAME

The rules for valid data set names

RECFM

The record format of the data set

Three key characteristics of QSAM data sets



```
//DD1  DD DSN=MY.NEW.DATA.SET,
//        DISP=(NEW,CATLG,DELETE),
//        DSORG=PS,
//        SPACE=(TRK,1)
```

- **PS stands for Physical Sequential. It pertains to QSAM, BSAM, and other types not covered in the course. You will use this a lot in your work.**

```
//DD1  DD DSN=MY.NEW.DATA.SET,
//        DISP=(NEW,CATLG,DELETE),
//        DSORG=PSU,
//        SPACE=(TRK,1)
```

- **PSU stands for Physical Sequential Unmoveable. It pertains to datasets that cannot be relocated from their position on a volume. This is out of scope for the course.**

More about blocks and logical records

Block

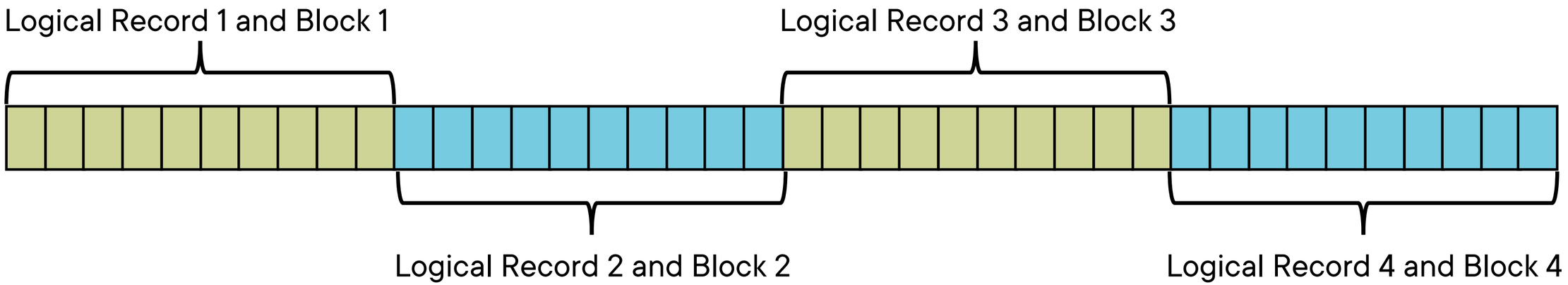
The unit of data transfer between DASD and programs

Logical Record

The logical unit of content in a data set

Fixed-length records, unblocked

```
DD RECFM=F,LRECL=10,BLKSIZE=10
```



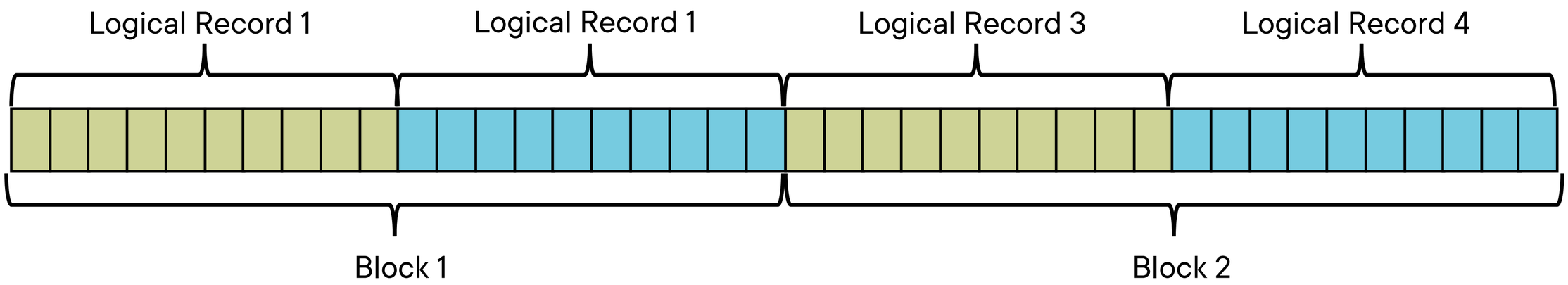
Rule: Block size must equal logical record length

```
//DD1  DD DSN=MY.NEW.DATA.SET,
//         DISP=(NEW,CATLG,DELETE),
//         SPACE=(CYL,30),
//         DSORG=PS,
//         RECFM=F,
//         LRECL=1024,
//         BLKSIZE=1024
```

- **Fixed-length records, unblocked**
- **Logical record length and block size are the same**

Fixed-length records, blocked

```
DD RECFM=FB,LRECL=10,BLKSIZE=20
```



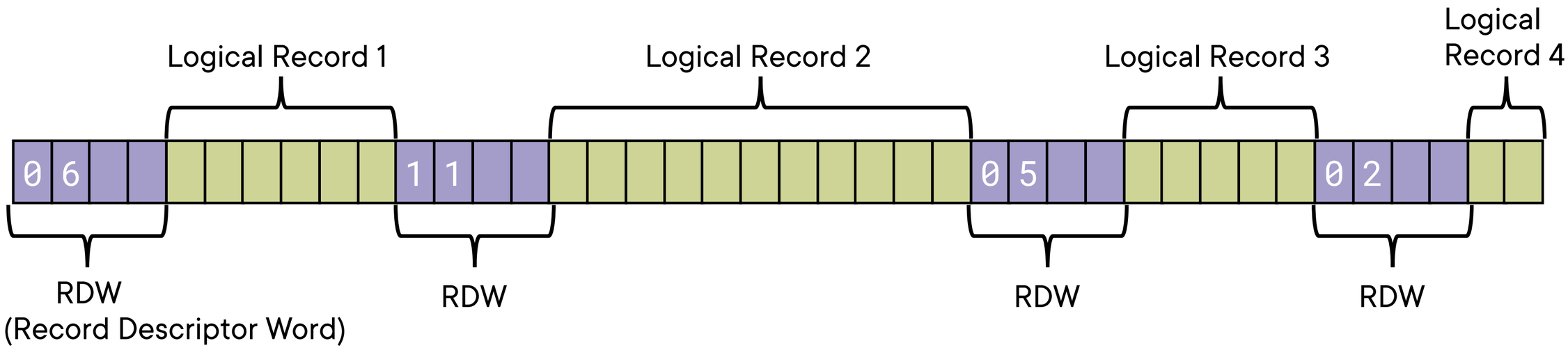
Rule: Block size must be an even multiple of logical record length

```
//DD1 DD DSN=MY.NEW.DATA.SET,
//       DISP=(NEW,CATLG,DELETE),
//       SPACE=(CYL,30),
//       DSORG=PS,
//       RECFM=FB,
//       LRECL=1024,
//       BLKSIZE=8192
```

- **Fixed-length records, blocked**
- **Block size is an even multiple of the logical record length**

Variable-length records, unblocked

```
DD RECFM=V,LRECL=16,BLKSIZE=20
```



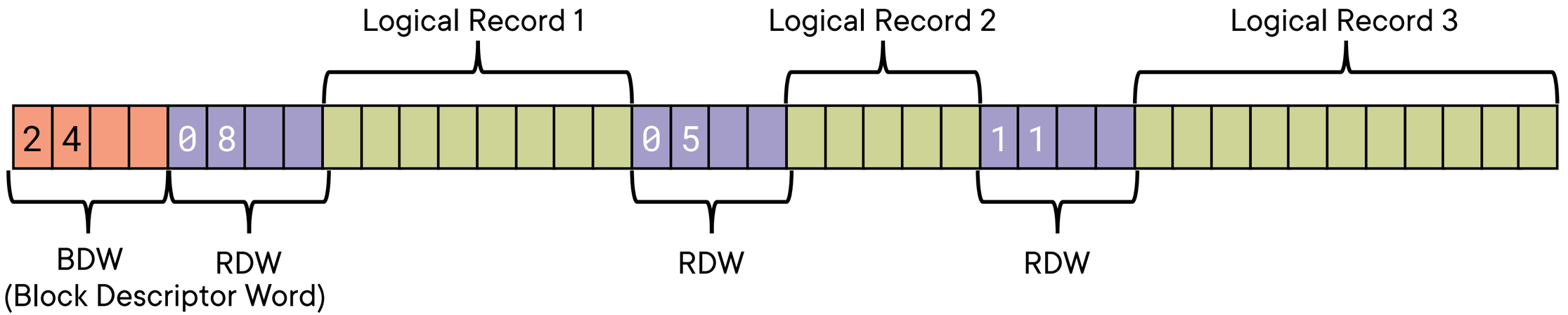
Rule: Block size must be *at least* the max logical record length + 4

```
//DD1 DD DSN=MY.NEW.DATA.SET,
//       DISP=(NEW,CATLG,DELETE),
//       SPACE=(CYL,30),
//       DSORG=PS,
//       RECFM=V,
//       LRECL=1024,
//       BLKSIZE=8196
```

- **Variable-length records, unblocked**
- **Block size is 4 larger than the maximum logical record length to leave room for the Record Descriptor Word (RDW)**

Variable-length records, blocked

```
DD RECFM=VB,LRECL=8,BLKSIZE=20
```



Rule: Block size must be *at least* (average logical record length x number of logical records per block) + 4

```
//DD1 DD DSN=MY.NEW.DATA.SET,
//       DISP=(NEW,CATLG,DELETE),
//       SPACE=(CYL,30),
//       DSORG=PS,
//       RECFM=VB,
//       LRECL=1024,
//       BLKSIZE=8200
```

- **Variable-length records, blocked**
- **Block size is 4 larger than the average logical record length, including RDWs, to leave room for the Block Descriptor Word (BDW)**

Demo:
Allocate a QSAM data set using JCL

We'll add a DD statement to our IEFBR14 job to allocate a QSAM data set. This will have the same outcome as allocating a data set through the ISPF panels.

Demo:
Making the job idempotent

A utility task such as refreshing a data set may be executed many times. No matter how many times we run this job, we want the end result to be the same.

If we run IEFBR14 75 times, we don't want 75 copies of the data set to be allocated. We just want one of them. Here's how.

Lab 11:
A job to delete and re-allocate a QSAM data set

Follow the instructions for this lab in your copy of the course Github repository.

DD statement parameters for partitioned data sets

Dave Nicolette
Programmer

neopragma.com

Two kinds of partitioned data sets

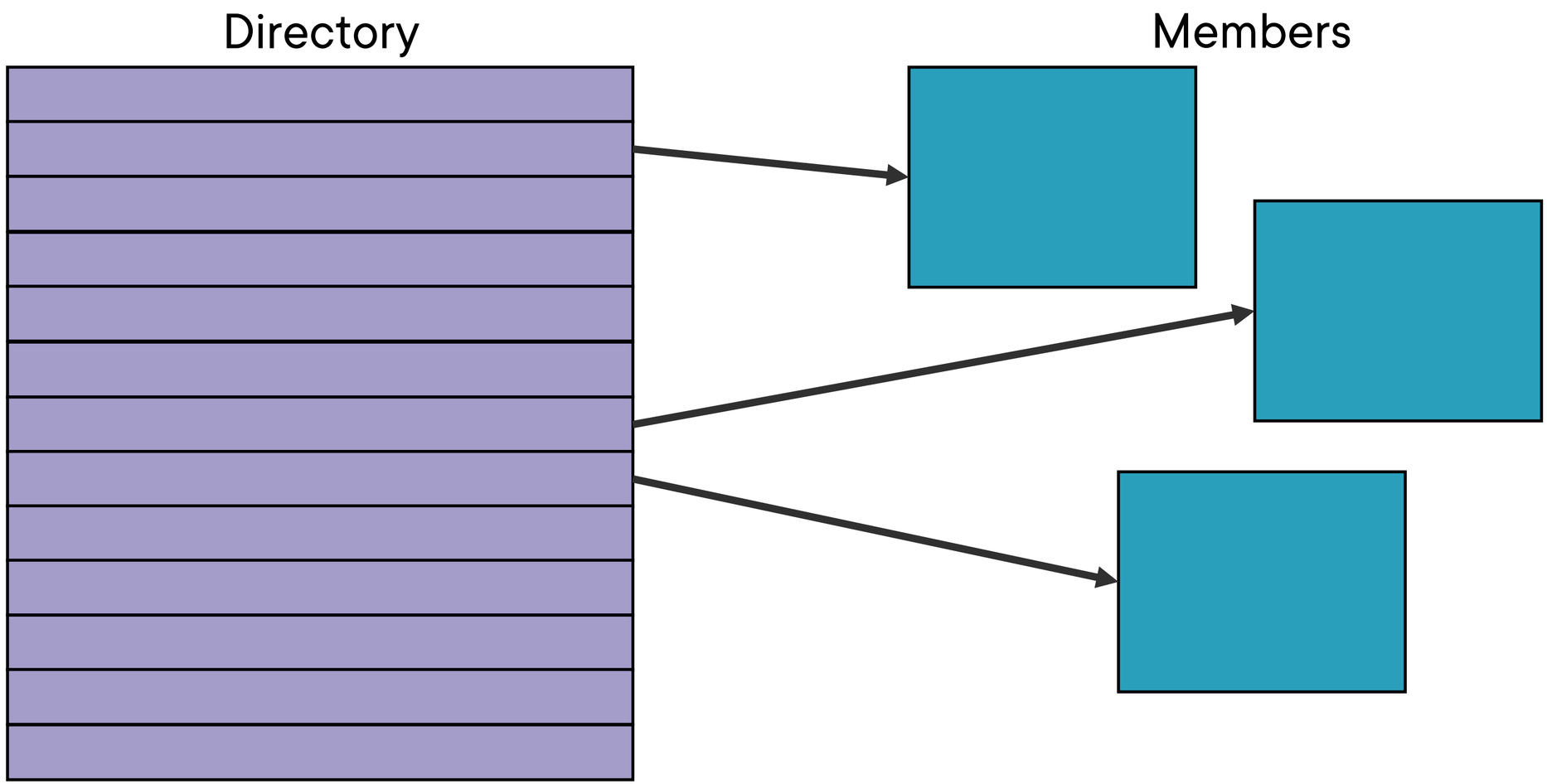
PDS

The original implementation, now supported for backward compatibility

PDSE

The current standard with many improvements and enhanced capabilities

PDSE directory and members



Two kinds of content

Data Members

Text, program source code, configuration files, etc.

Program Objects

Executable binaries

```
//DD1 DD DSN=MY.NEW.PDS,
//       DISP=(NEW,CATLG,DELETE),
//       DSNTYPE=LIBRARY,
//       ...
```

◀ Specifies PDSE

```
//DD1 DD DSN=MY.NEW.PDS,
//       DISP=(NEW,CATLG,DELETE),
//       DSNTYPE=PDS,
//       ...
```

◀ Specifies PDS (legacy)

```
//DD1  DD DSN=MY.NEW.PDS,
//         DISP=(NEW,CATLG,DELETE),
//         DSNTYPE=LIBRARY,
//         DCB=(RECFM=FB,LRECL=100),
//         ...
```

◀PDSE containing data members

```
//DD1  DD DSN=MY.NEW.PDS,
//         DISP=(NEW,CATLG,DELETE),
//         DSNTYPE=LIBRARY,
//         DCB=(RECFM=U)
//         ...
```

◀PDSE containing program objects

```
//DD1  DD DSN=MY.NEW.PDS,
//         DISP=(NEW,CATLG,DELETE),
//         DSNTYPE=LIBRARY,
//         SPACE=(CYL,(10,10,10)),
//         ...
```

- Space allocation in cylinders

```
//DD1  DD DSN=MY.NEW.PDS,
//         DISP=(NEW,CATLG,DELETE),
//         DSNTYPE=LIBRARY,
//         SPACE=(300,(5000,100)),
//         ...
```

- Space allocation in blocks
  - 300 = average block size
  - 5000 = number of blocks in primary extent
  - 100 = number of blocks in secondary extent

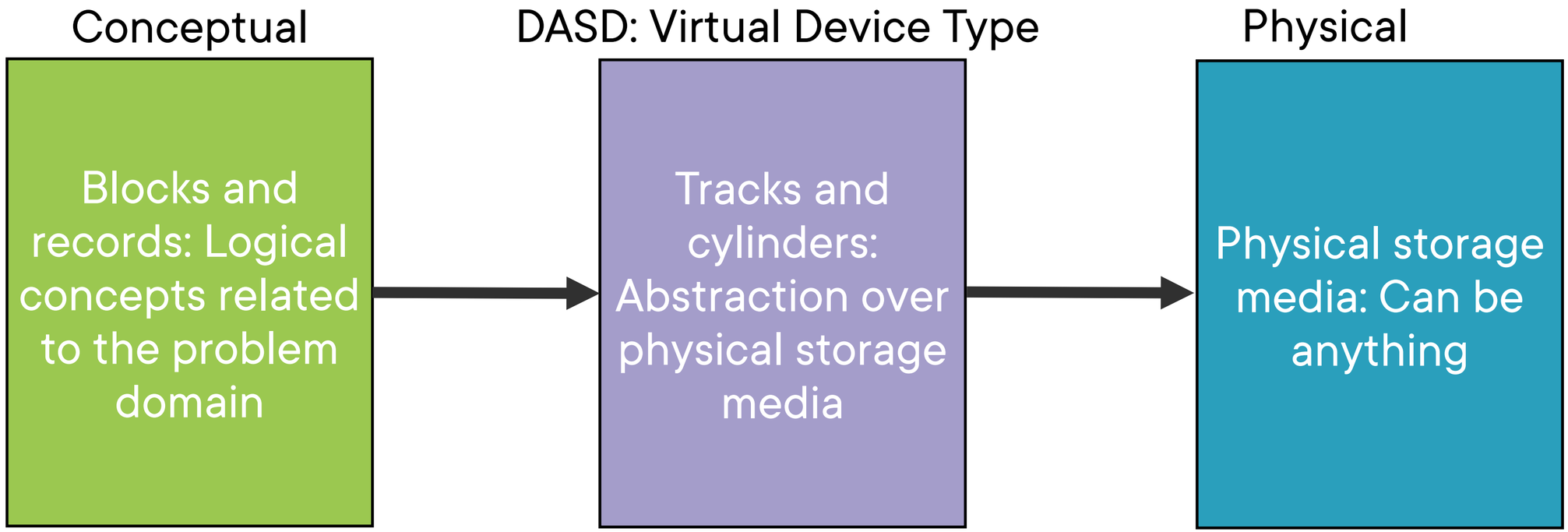
```
//DD1  DD DSN=MY.NEW.PDS,
//        DISP=(NEW,CATLG,DELETE),
//        DSNTYPE=LIBRARY,
//        SPACE=(80,(20,2)),
//        AVGREC=K,
//        ...
```

- Space allocation in records
  - 80 = average record length in bytes
  - AVGREC=K means the values in parentheses are in terms of kilobytes
  - 20 = primary allocation 20K
  - 2 = secondary allocation 2K

```
//DD1  DD DSN=MY.NEW.PDS,
//        DISP=(NEW,CATLG,DELETE),
//        DSNTYPE=LIBRARY,
//        SPACE=(200,(8,1)),
//        AVGREC=M,
//        ...
```

- Space allocation in records
  - 200 = average record length in bytes
  - AVGREC=M means the values in parentheses are in terms of megabytes
  - 8 = primary allocation 8M
  - 1 = secondary allocaiton 1M

Blocks and records related to tracks and cylinders



```
//DD1  DD DSN=MY.NEW.PDS,
//         DISP=(NEW,CATLG,DELETE),
//         DSNTYPE=LIBRARY,
//         SPACE=(CYL,(20,4)),
//         DCB=(DSORG=PO,...),
//         ...
```

◀ **Let the system calculate directory space**

```
//DD1  DD DSN=MY.NEW.PDS,
//         DISP=(NEW,CATLG,DELETE),
//         DSNTYPE=LIBRARY,
//         DATACLAS=name,
//         ...
```

◀ **Reference a data class**

Demo:
Allocating a PDSE using JCL

In an earlier lab, you allocated a source library for your JCL using ISPF panels. Here we show how to do the same thing using JCL.

Lab 12:
A job to delete and re-allocate a source library (PDSE)

Follow the instructions for this lab in your copy of the course Github repository.

Demo:
Allocating a Program Library using JCL

In an earlier lab, you allocated a source library for your JCL using ISPF panels. Here we show how to do the same thing using JCL.

Lab 13:
A job to delete and re-allocate a program library (PDSE)

Follow the instructions for this lab in your copy of the course Github repository.

- z/OS overview
- Data sets
- Access methods
- Basic JCL
- IEFBR14 utility

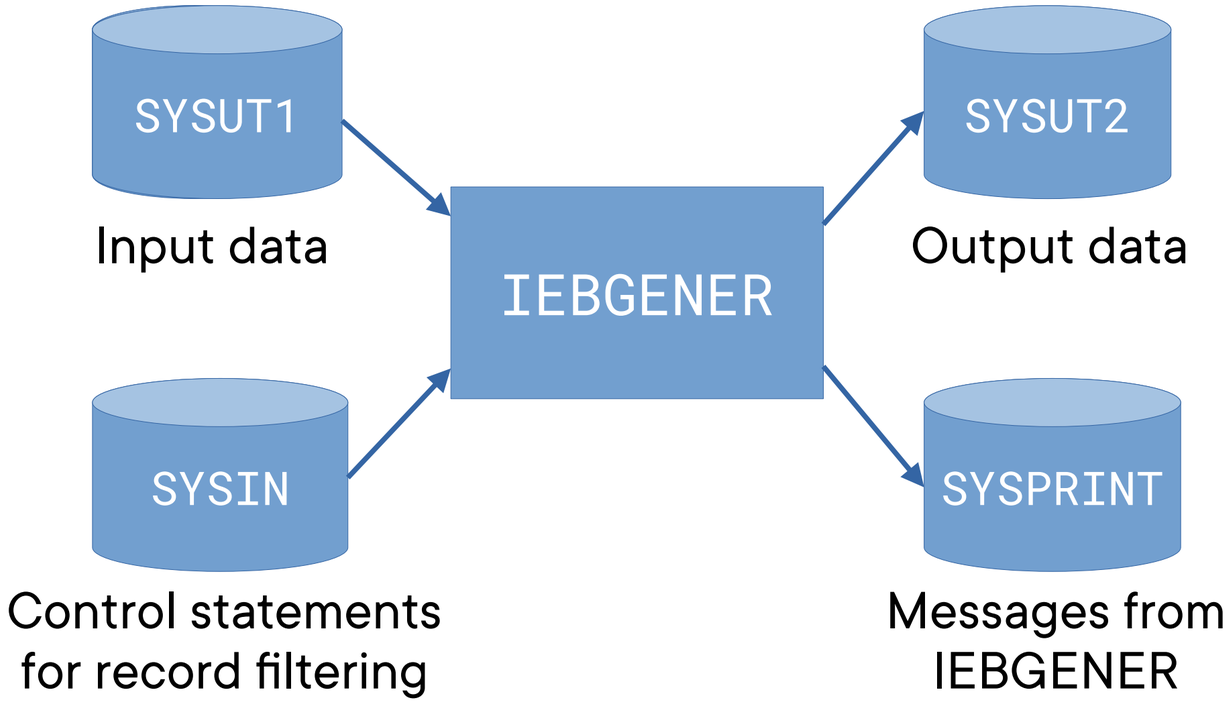
Checkpoint

The IEBGENER utility

Dave Nicolette
Programmer

neopragma.com

Functions of the IEBGENER utility



- **Copy data sets**
- **Filter records**
- **Rearrange fields in records**
- **Change record length, blocksize**

Demo:
Writing JCL to copy a data set using the IEBGENER utility

We'll show how to code a job that copies data from SYSUT1 to SYSUT2 using IEBGENER.

Lab 14:
Use IEBGENER to copy inline data to SYSOUT

Follow the instructions for this lab in your copy of the course Github repository.

Lab 15:
Use IEBGENER to allocate a new sequential dataset and copy records into it

Follow the instructions for this lab in your copy of the course Github repository.

Lab 16:
Use IEBGENER to copy a data set and reformat the records

Follow the instructions for this lab in your copy of the course Github repository.

Getting acquainted with COBOL

Dave Nicolette
Programmer

neopragma.com

COBOL

COmmon
Business
Oriented
Language

Visual characteristics of COBOL source

Syntax

English-like

Structure

Like a business document

COBOL COMPUTE Statement

```
a=(b*c)/(b+c)
```

```
COMPUTE A = (B*C)/(B+C).
```

COBOL Arithmetic Verbs

$$a=(b*c)/(b+c)$$

```
ADD B TO C
    GIVING B-PLUS-C.
MULTIPLY B BY C
    GIVING B-TIMES-C.
DIVIDE B-TIMES-C
    BY B-PLUS-C
    GIVING A.
```

COBOL structure mirrors a project proposal document

Response to RFP	COBOL program	Purpose
Executive Summary	Identification Division	What is this all about?
External Dependencies	Environment Division	Necessary things outside of the program
Parts and Materials	Data Division	Variables used in the program
Steps to Complete the project	Procedure Division	Executable statements

Demo:
A guided tour of a sample COBOL program

We'll walk through a sample COBOL program to see the structure and the look-and-feel of COBOL source code.

Multi-platform and cross-platform languages

Multi-Platform

Source code is similar, but not identical

Conditional compilation or minor changes in the source

Platform-specific compiler produces platform-specific object code

Examples: C, COBOL

Cross-Platform

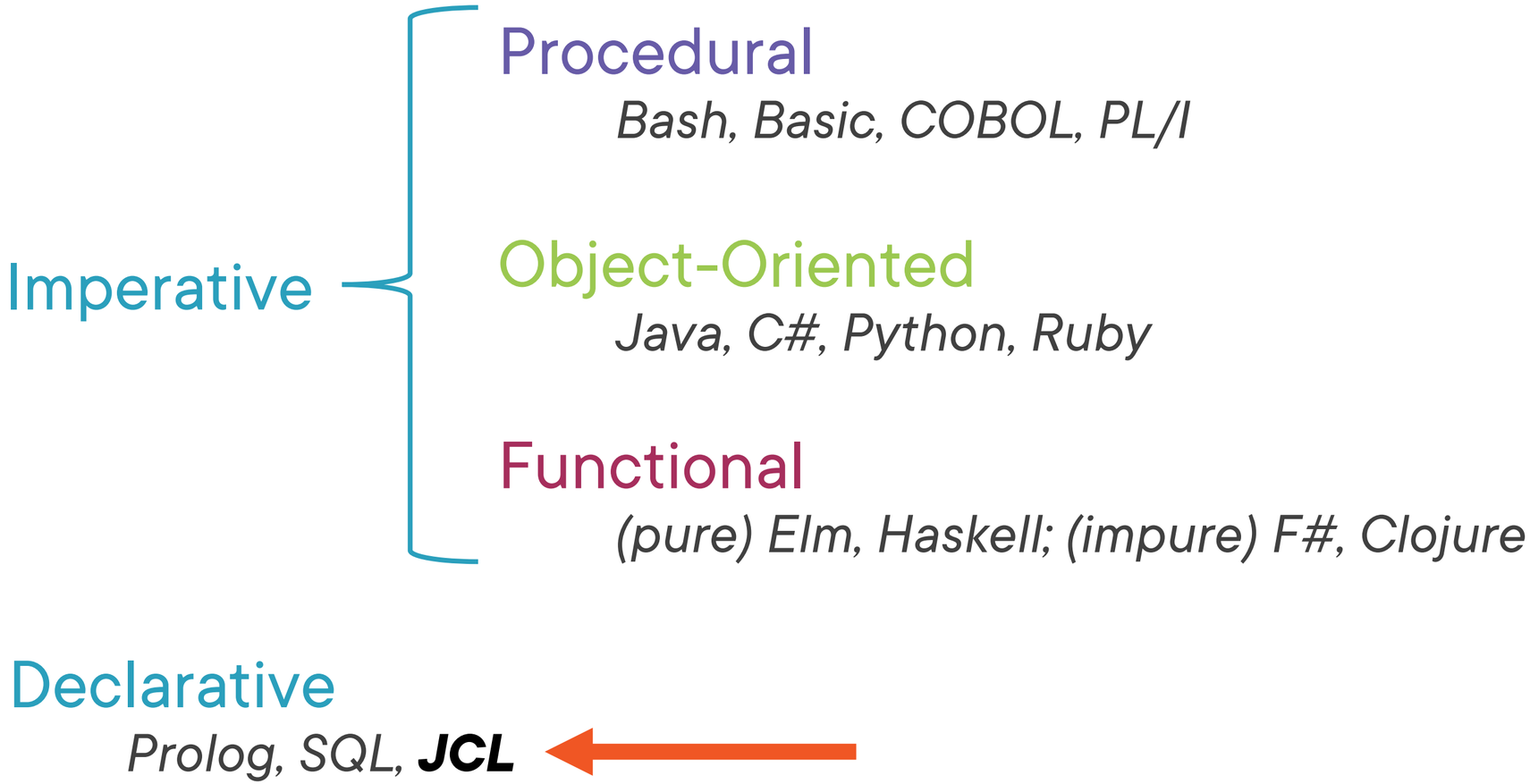
Source code can be identical

Compilers, interpreters, and/or runtimes are written for specific platforms

“Write once, run anywhere” concept

Examples: Java, Ruby

Categories of languages (simplified)



How programs are prepared for execution

Dave Nicolette
Programmer

neopragma.com

Compiled vs. interpreted languages

Interpreted language (BASIC)

An interpreter parses the source code at execution time and runs pre-built routines that correspond to the source statements.

Compiled language (COBOL)

A compiler converts the source statements into executable instructions. Then the executable version of the program can run any number of times.

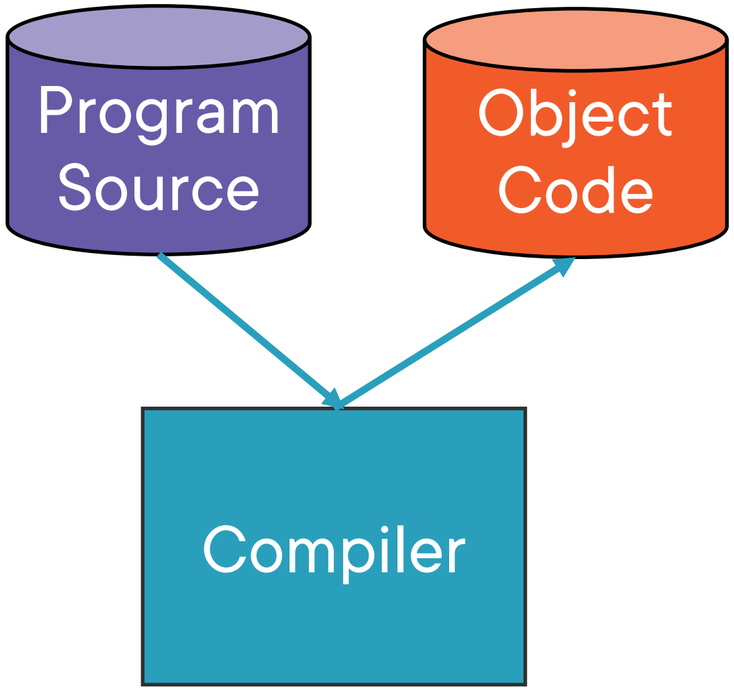
A little of each (Java)

A compiler converts the source statements into an intermediate representation of the logic. A runtime then processes the intermediate code and converts it to executable instructions at runtime.

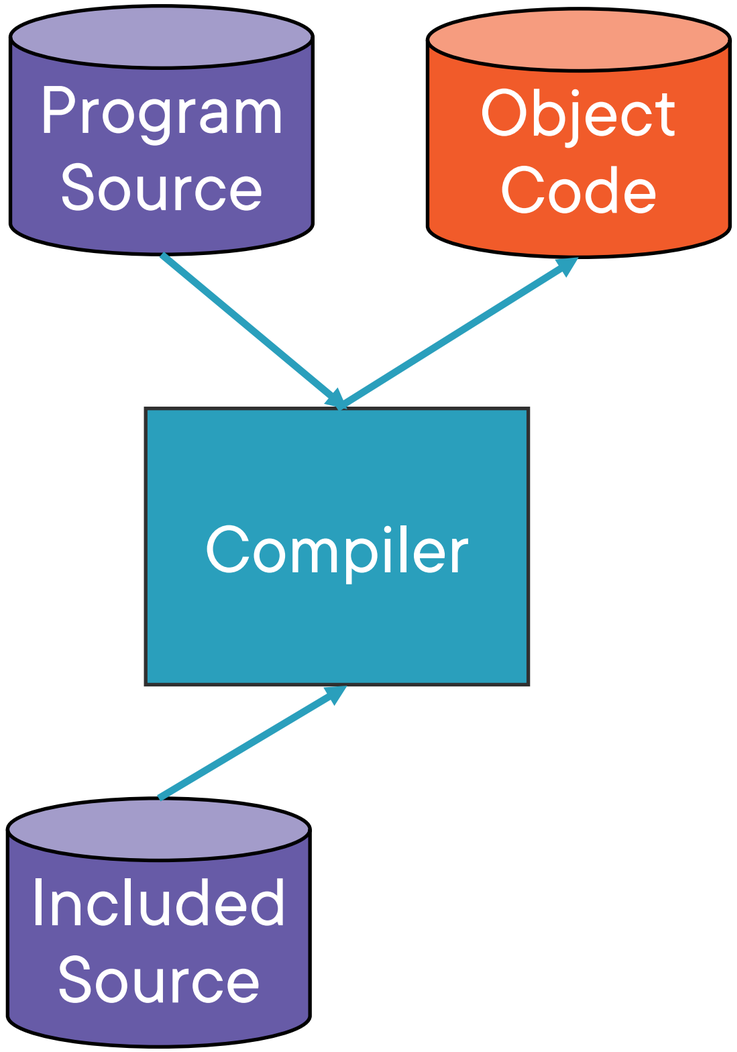
Just-in-Time Compiled language (Python)

The language environment automatically compiles parts of the program that have changed. We don't have to run the compiler separately.

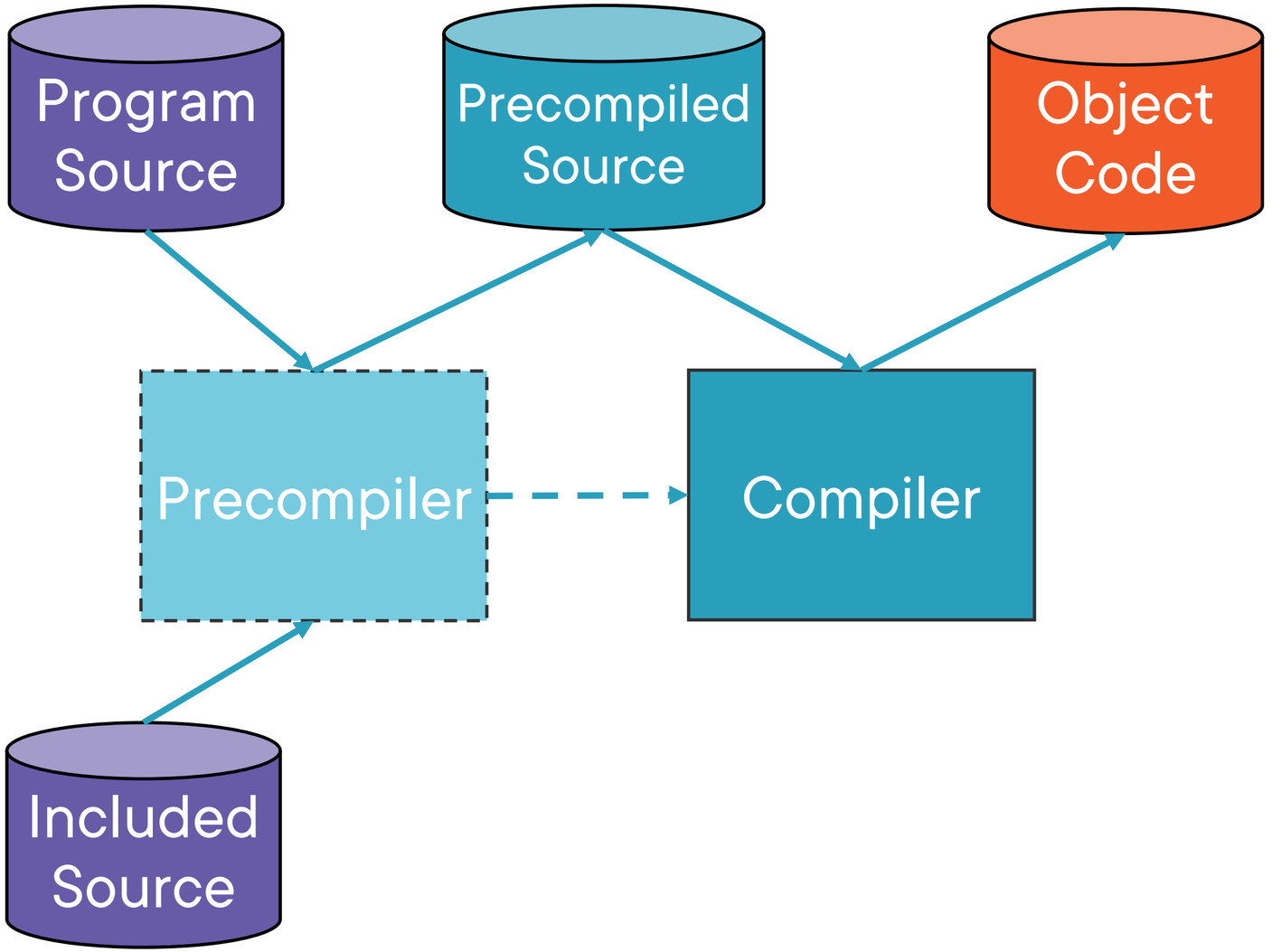
From source to program object



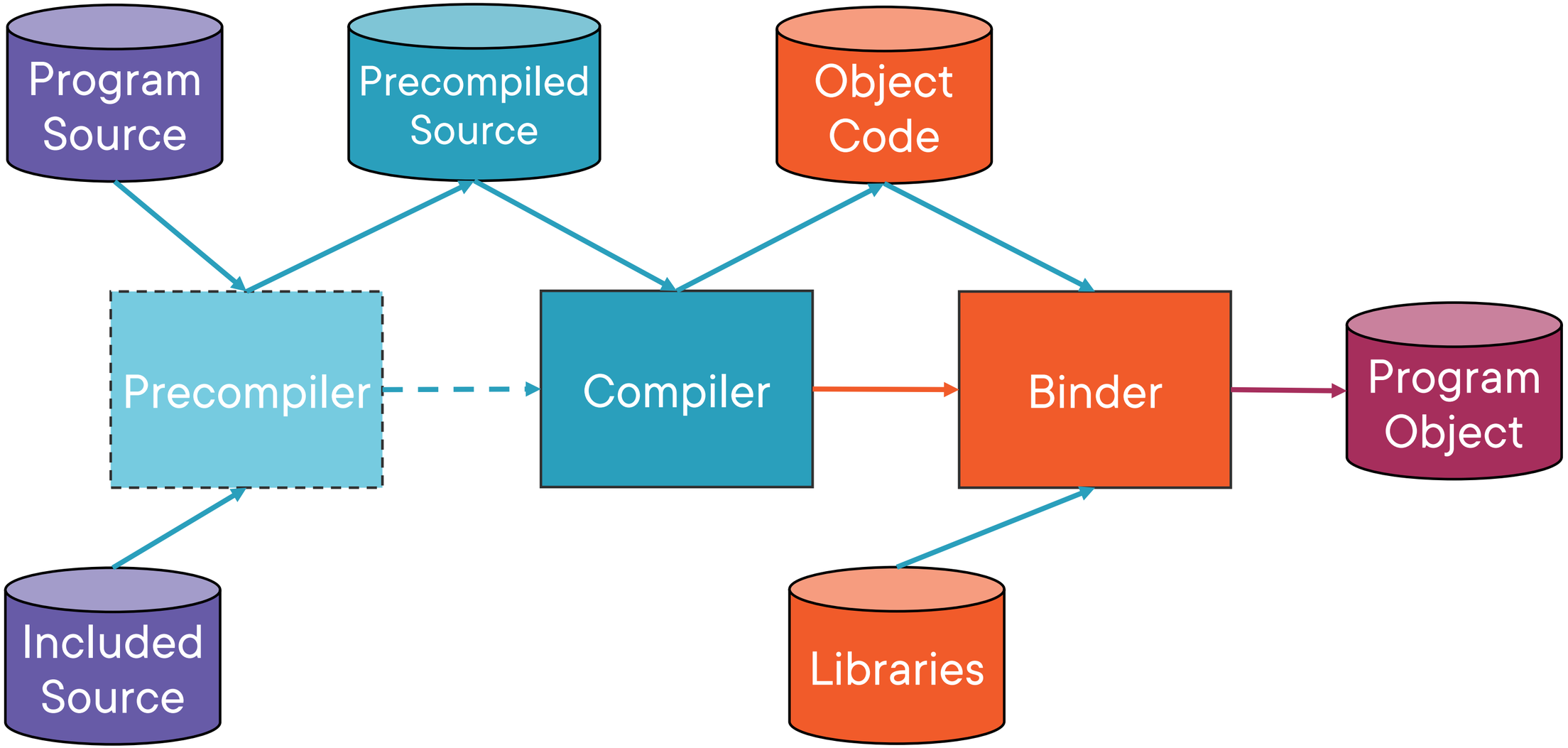
From source to program object



From source to program object



From source to program object



Executables and libraries

Compilers convert *source code* into *object code*.

That object code is not executable because it contains references to other object code, such as subprograms or stubs for invoking system facilities.

Those items have to be *bound* with or *linked* into the object code to produce an executable artifact.

A program that runs after the compiler performs this work.

The current implementation is called the *binder*. The binder produces a *program object* that is stored in a PDSE library. These libraries are often called *program libraries*.

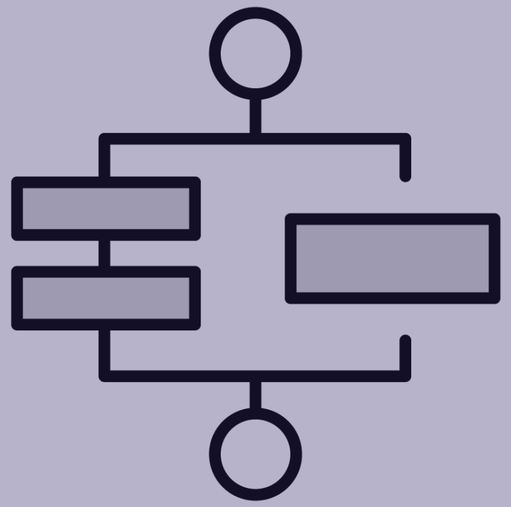
The previous implementation is called the *linkage editor*. The linkage editor is still supported for backward compatibility with older applications. The linkage editor produces a *load module* that is stored in a PDS library. These libraries are often called *load libraries*.

By convention and for cultural reasons, most people refer to program objects as "load modules" and program libraries as "load libraries." That's fine for conversational purposes, but you need to understand the differences.

JCL Procedures

Dave Nicolette
Programmer

neopragma.com



What is a JCL procedure?

A *JCL procedure* is a reusable set of JCL statements that can be included in jobstreams. By convention, people call JCL procedures *PROCs*.

Value:

- Reduces the size and visual complexity of jobstreams
- Provides standard JCL code for common job steps

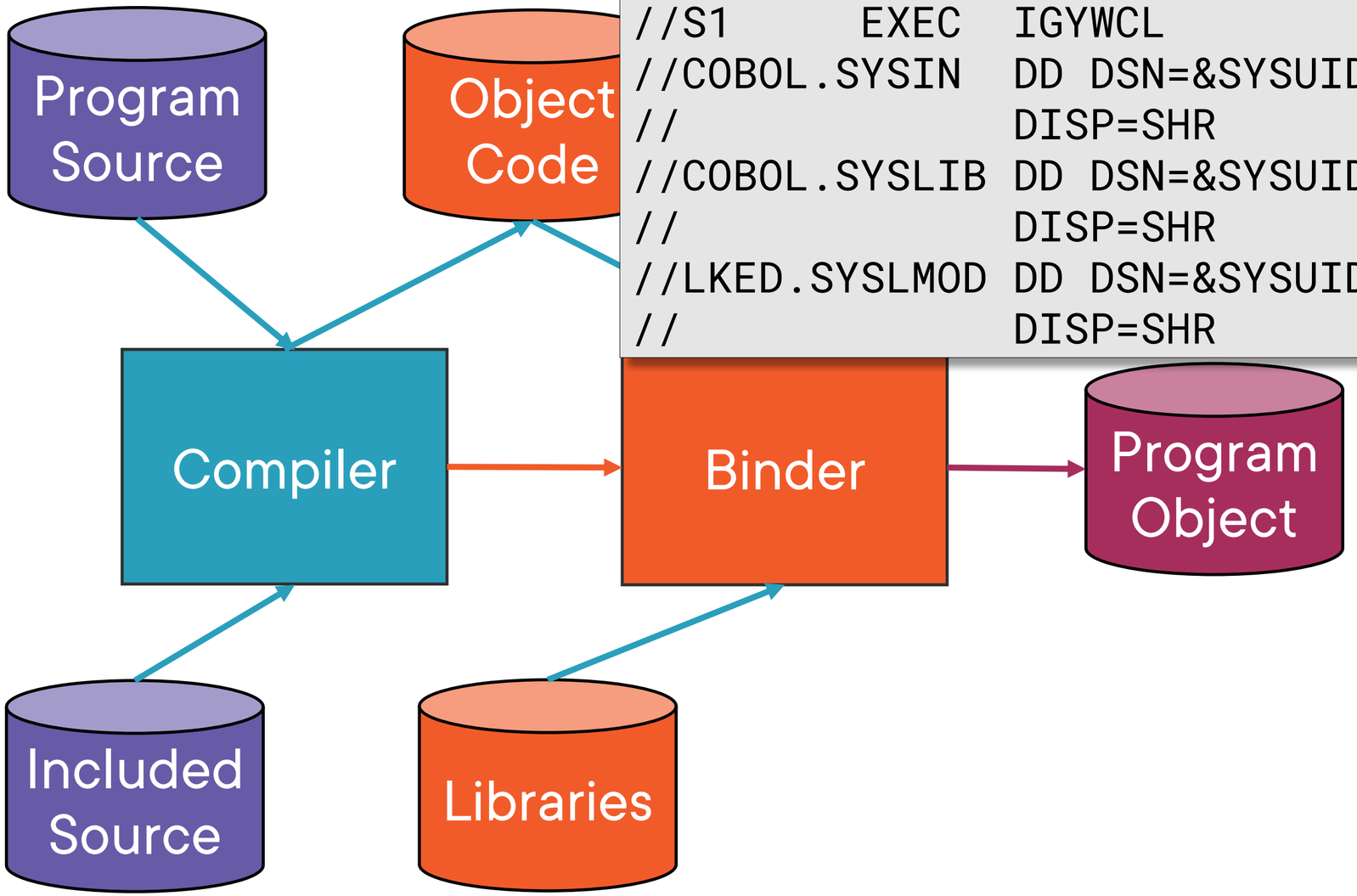
Inline Procedure

Procedure is coded within the jobstream. Steps in that jobstream can call the procedure.

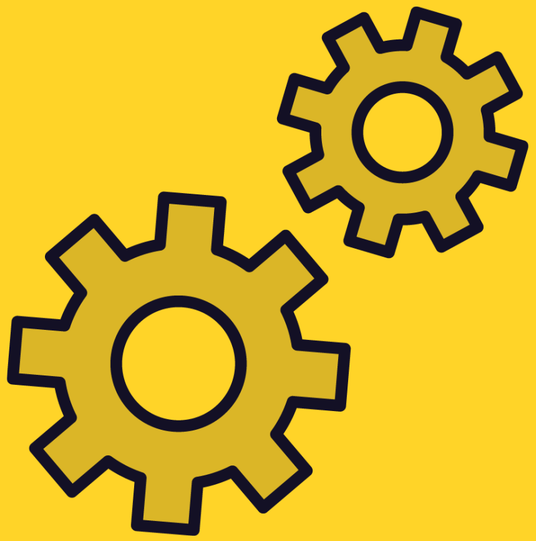
Catalogued Procedure

Procedure lives in a library separate from jobstreams. Steps in any job can call the procedure.

Compile and bind (link) a COBOL program




```
//jobname JOB   blah
//S1      EXEC  IGYWCL
//COBOL.SYSIN   DD DSN=&SYSUID..TSO.SRC(GRCOBFC),
//                DISP=SHR
//COBOL.SYSLIB DD DSN=&SYSUID..TSO.COPY,
//                DISP=SHR
//LKED.SYSLMOD DD DSN=&SYSUID..TSO.LOAD(GRCOBFC),
//                DISP=SHR
```



IBM compile PROCs in SYS1.PROCLIB

Base Names	*Language*
PLIX, PLICK, IEL1	PL/I
IGYW, COB2	COBOL
ASMA, HLASM	Assembler
CEDC	C/C++

Suffix	*Steps*
C	compile
L	link/bind
P	pre-link
G	execute ("go")

Examples	
IGYWCL	COBOL compile and link/bind
HLASMC	High-level assembler compile
IEL1CG	PL/I compile and execute

Demo:
A guided tour of catalogued procedures for compilation

We'll browse the system PROCLIB and see which procedures are available for compiling COBOL programs.

Minimal COBOL program

Dave Nicolette
Programmer

neopragma.com

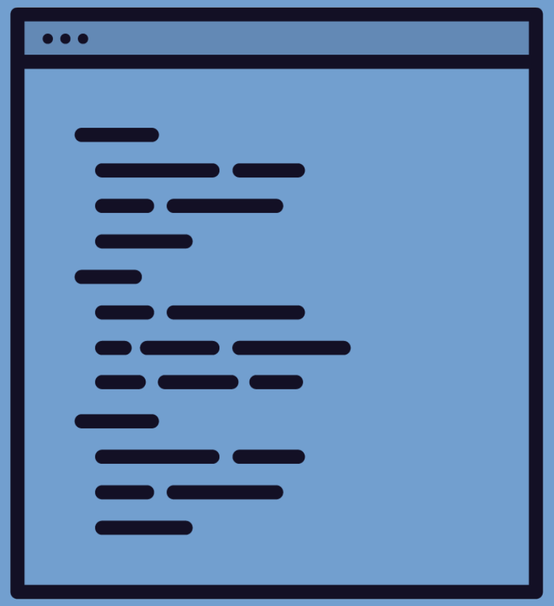
Demo:
A minimal COBOL program

We'll show the minimum amount of code needed to satisfy the compiler, and walk through the steps to compile and bind the program.

New concepts from this demo

- JCL symbols
- Conditional step execution
- Overriding DD statements

COBOL source code formats



The current ANSI COBOL standard supports free format and fixed format COBOL source.

COBOL implementations for particular platforms may require specific formatting rules.

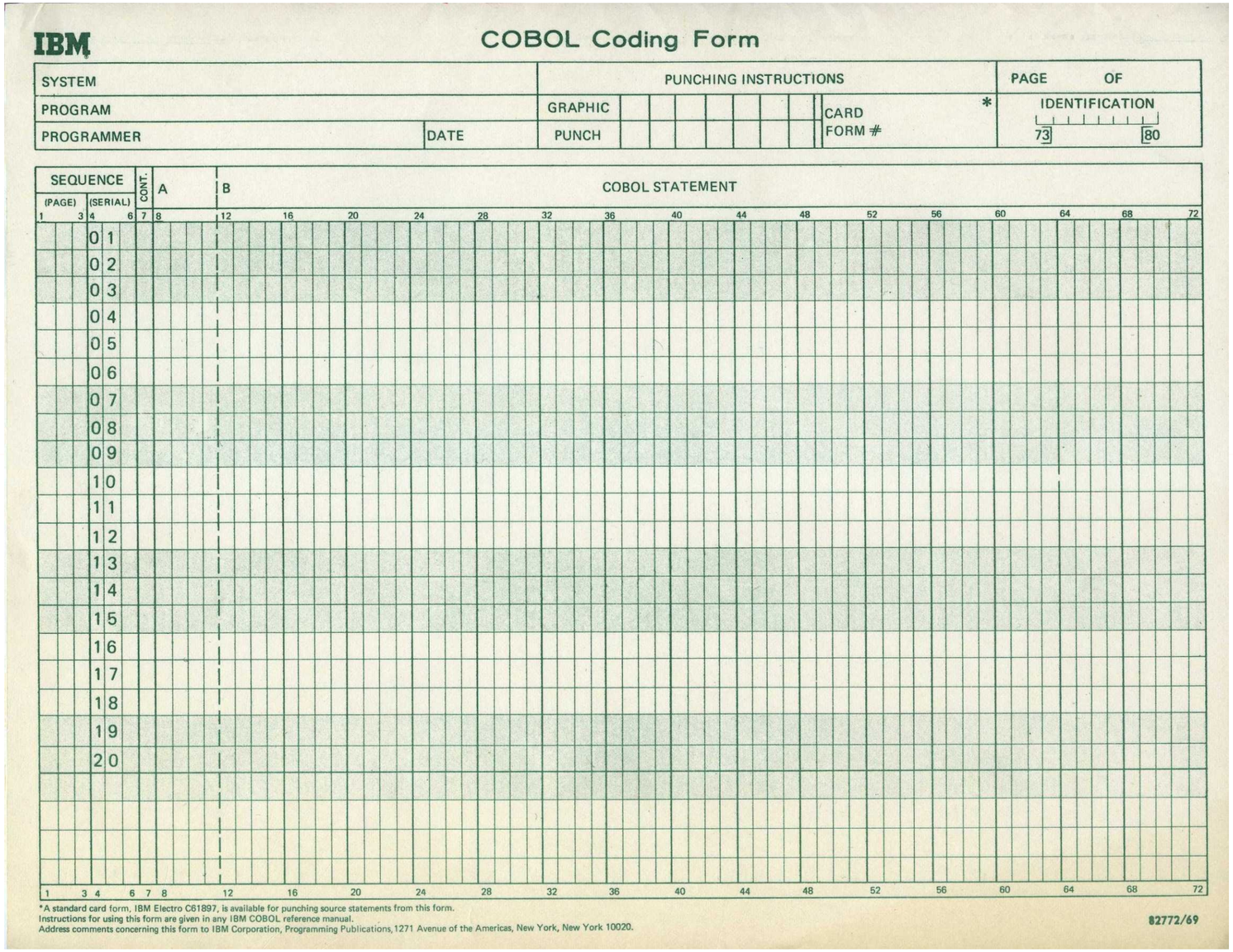
Early COBOL implementations were designed around punched card entry.

The brain-in-a-jar still thinks it's looking at punched cards (80-byte card-image records).

Therefore, IBM Enterprise COBOL doesn't support free format COBOL source.

Why is the source format the way it is?

A COBOL coding form (1960s)



IBM

COBOL Coding Form

SYSTEM		PUNCHING INSTRUCTIONS		PAGE OF
PROGRAM		GRAPHIC	CARD *	IDENTIFICATION 73 80
PROGRAMMER	DATE	PUNCH	FORM #	

SEQUENCE (PAGE) 1 3	SEQUENCE (SERIAL) 4 6	CONT. 7	A 8	B 12	COBOL STATEMENT 12 16 20 24 28 32 36 40 44 48 52 56 60 64 68 72
	01				
	02				
	03				
	04				
	05				
	06				
	07				
	08				
	09				
	10				
	11				
	12				
	13				
	14				
	15				
	16				
	17				
	18				
	19				
	20				

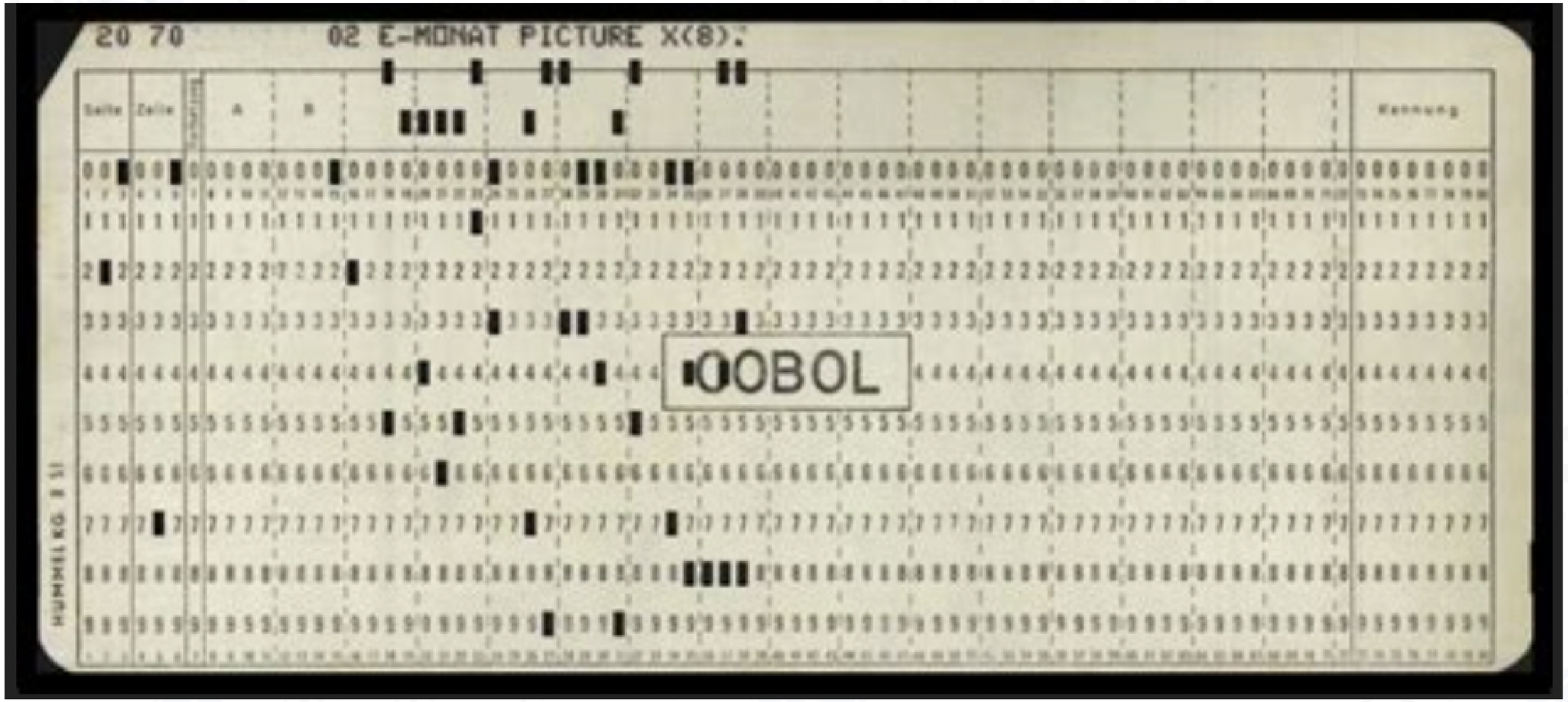
1 3 4 6 7 8 12 16 20 24 28 32 36 40 44 48 52 56 60 64 68 72

*A standard card form, IBM Electro C61897, is available for punching source statements from this form.
Instructions for using this form are given in any IBM COBOL reference manual.
Address comments concerning this form to IBM Corporation, Programming Publications, 1271 Avenue of the Americas, New York, New York 10020.

82772/69

Card punch department (1960s)

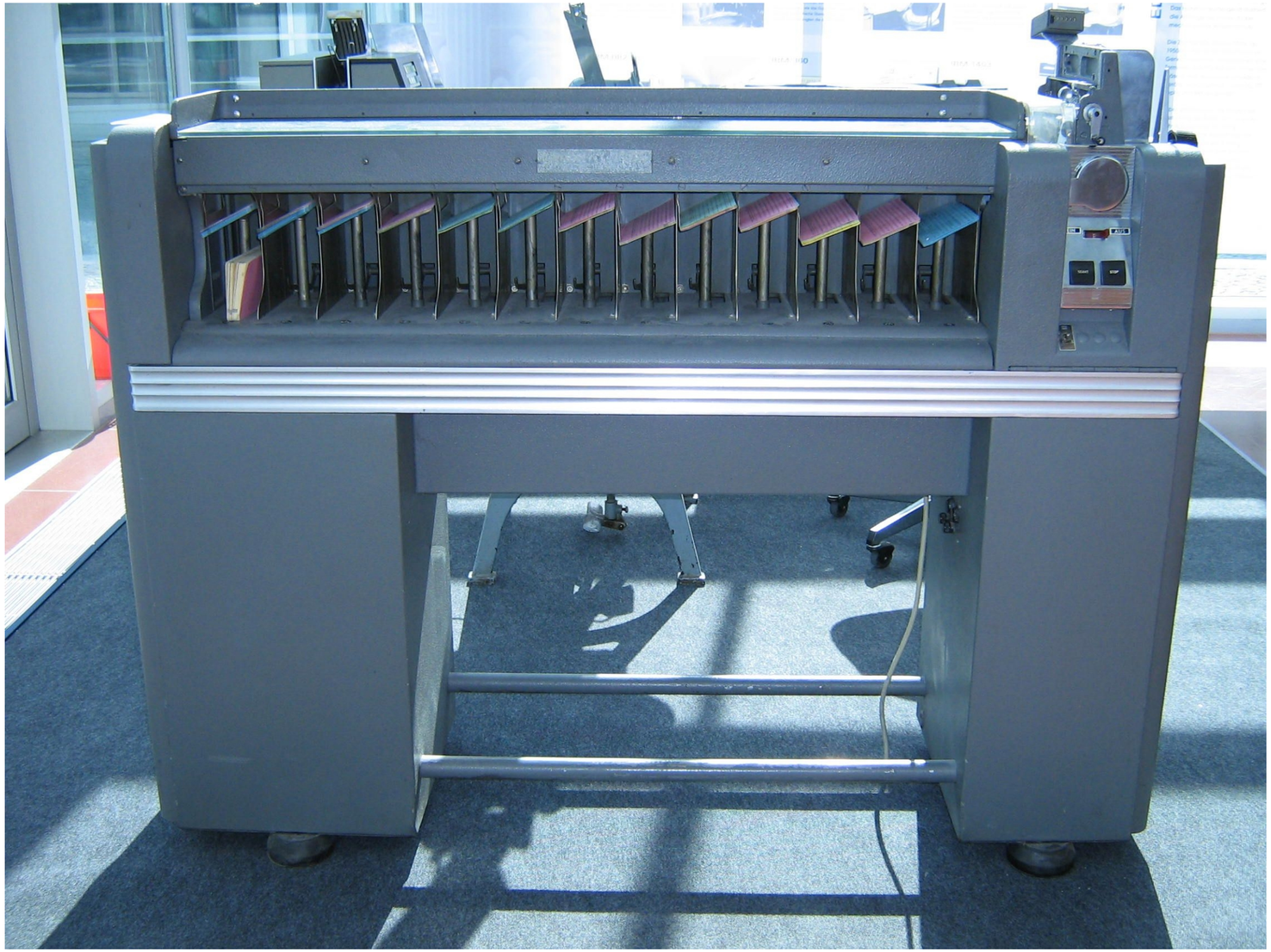
A punched card containing a COBOL source statement



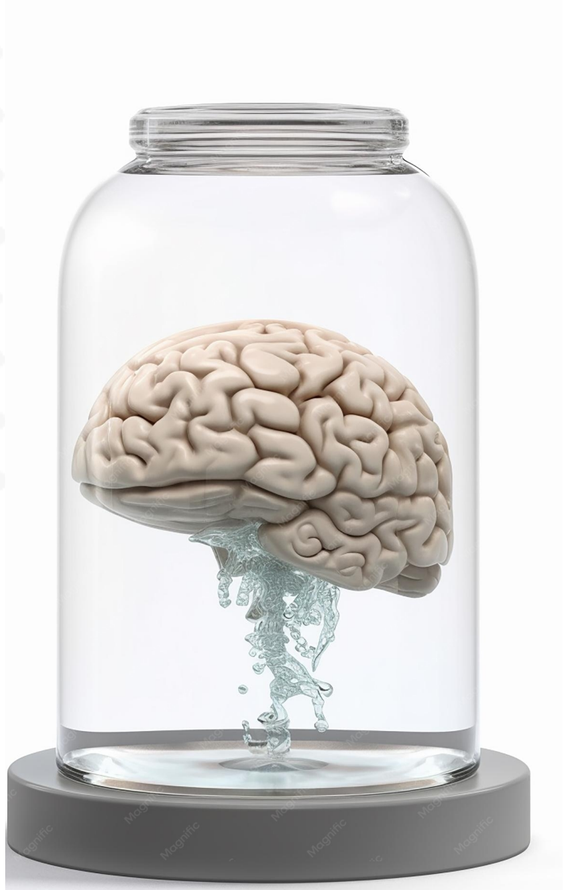
IBM 1402 card reader (1960s)

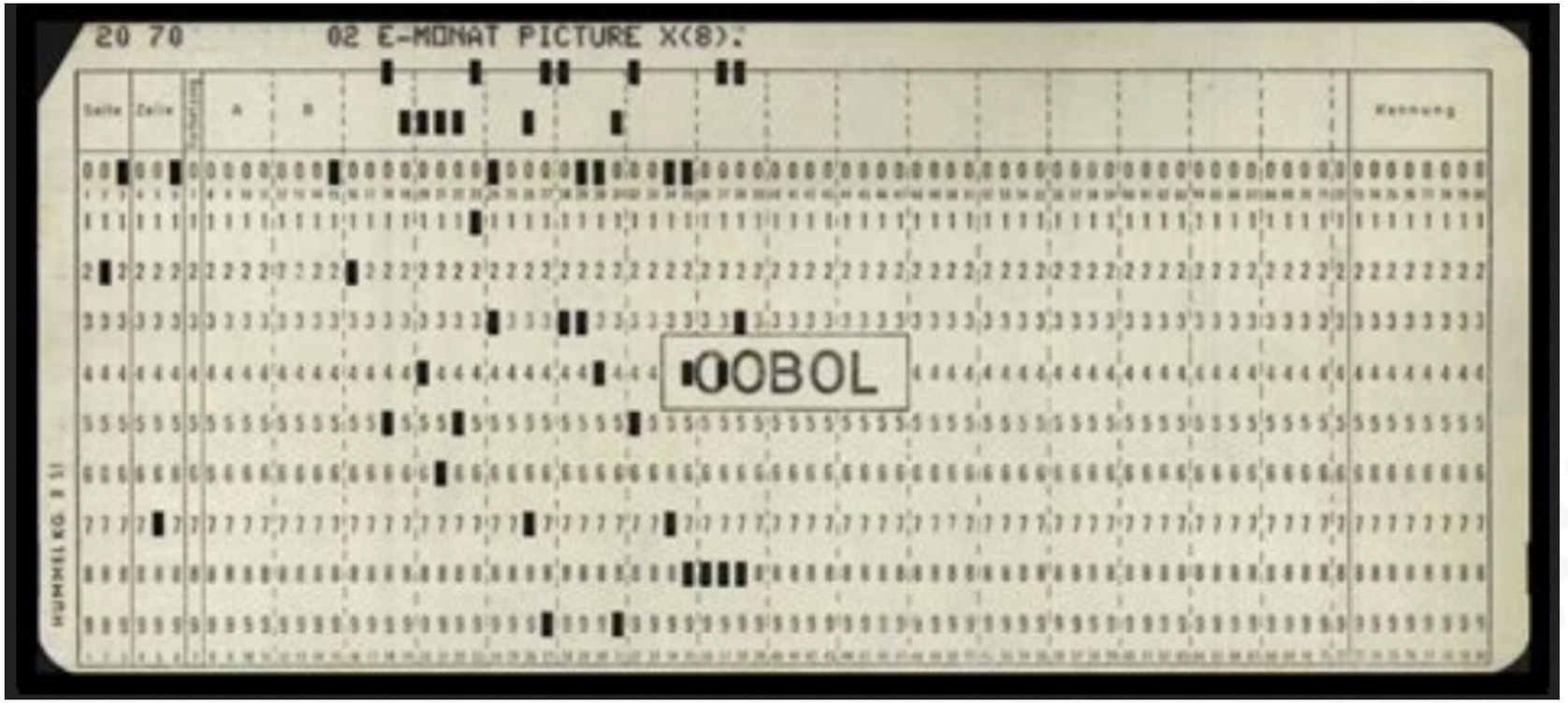
52 card pick-up

IBM card sorter



None of that is relevant anymore, but...





IBM's Reference Format for COBOL is documented here:

https://www.ibm.com/docs/en/cobol-zos/6.4.0?topic=structure-reference-format

Demo:
The IBM Reference Format

We'll show how IBM Enterprise COBOL source statements must be formatted to comply with IBM's Reference Format for COBOL.

"Hello, world!"

Dave Nicolette
Programmer

neopragma.com

Demo:
Hello, world!

We'll develop a "Hello, world!" program in COBOL.

New concepts from this demo

- The use of a JOBLIB DD
- ISPF split panels
- ISPF Editor commands:
  - caps on/off
  - unnum
  - num on/off
  - hilite

Lab 17:
Your first COBOL program

Follow the instructions for this lab in your copy of the course Github repository.

Identification Division

Dave Nicolette
Programmer

neopragma.com

COBOL Identification Division

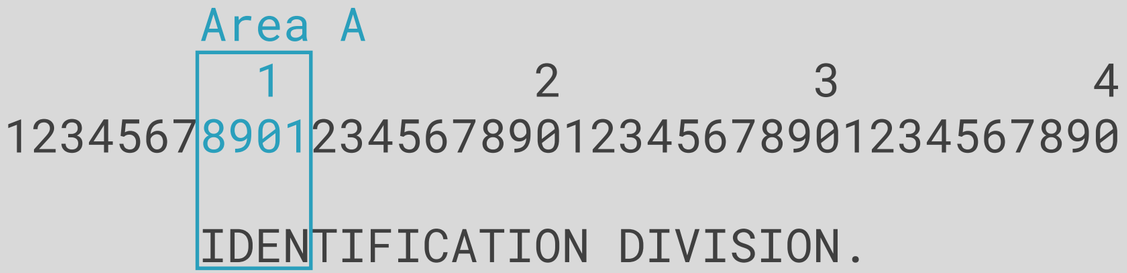
Provides the unique identifier of the program and provides general information about its purpose.

The Identification Division is analogous to the Executive Summary in a business proposal.

```
          1         2         3         4
12345678901234567890123456789012345678890

       IDENTIFICATION DIVISION.
```

- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.



- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.
- Identification can be abbreviated Id.

```
       Area A
         1         2         3         4
1234567890123456789012345678901234567890

       IDENTIFICATION DIVISION.

       ID DIVISION.
```

- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.
- Identification can be abbreviated Id.
- The text is case-insensitive.

```
       Area A
         1         2         3         4
1234567890123456789012345678901234567890

       identification division.

       id division.
```

- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.
- Identification can be abbreviated Id.
- The text is case-insensitive.

```
       Area A
         1         2         3         4
1234567890123456789012345678901234567890

       iDeNtIfiCAtIoN diVISioN.

       ID DiViSiOn.
```

- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.
- Identification can be abbreviated Id.
- The text is case-insensitive.

```
       Area A
         1         2         3         4
1234567890123456789012345678901234567890

       Identification Division.

       Id Division.
```

- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.
- Identification can be abbreviated Id.
- The text is case-insensitive.

```
       Area A
         1         2         3         4
1234567890123456789012345678901234567890

       Identification Division.

       Id Division.
```

- The Identification Division header is the first line of a COBOL program.
- It must begin in Area A of the source record.
- Identification can be abbreviated Id.
- The text is case-insensitive.
- The header must be terminated with a period.

```
      1         2         3         4
1234567890123456789012345678901234567890

 Identification Division.
 Program-Id. PGMNAME.
```

- **Program-Id is the only mandatory entry for a basic batch program.**

Program-Id

Main program – batch

CICS program

IMS/TM program

ISPF Dialog program

ISPF Edit Macro

Subprogram

Nested Program

Other types of COBOL objects besides "programs"

For awareness…out of scope for this course.

FUNCTION-ID — defines a user-defined function.

CLASS-ID — defines a class in object-oriented COBOL.

INTERFACE-ID — defines an interface.

METHOD-ID — defines a method inside a class or interface, rather than serving as a top-level alternative to PROGRAM-ID.

Optional paragraphs in the Identification Division

In keeping with the idea that programs should resemble business documents, COBOL originally supported several optional paragraphs to provide information similar to that found in an Executive Summary for a business document.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CUSTRPT.
AUTHOR. A. DEVELOPER.
DATE-WRITTEN. 20261013.
DATE-COMPILED.
REMARKS. Creates the daily customer report.
SECURITY. Can you keep a secret?
```

These paragraphs fell out of use very early in the history of COBOL, but they are still supported for backward compatibility. People normally provide "executive summary" information in the form of comments.

Long Program-Id values

IBM Enterprise COBOL (not standard COBOL) allows for a Program-Id value up to 160 characters long, if it is coded as an alphanumeric literal (quoted).

The compiler will convert this into an 8-byte value.

The brain-in-a-jar can't cope with program names longer than 8 bytes. The name of the object module will be the truncated version generated by the compiler.

Optional paragraphs in the Identification Division

If coded, the DATE-COMPILED value is filled in by the compiler.

Security, in any meaningful sense, is not handled by individual application programs.

In older programs you will find extensive information about change history in the form of source comments. In the era of modern version control systems, this information is redundant (or just wrong) if written as source comments.

So, there isn't much practical value in the optional paragraphs. Even so, your organization may have standards about what must be included and how the comments are to be organized.

```
 IDENTIFICATION DIVISION.
 PROGRAM-ID. CUSTRPT.
*****************************************
* Produce the daily customer report.
* 1989-02-15. A. Developer.
*****************************************
```

Environment Division

Dave Nicolette
Programmer

neopragma.com

COBOL Environment Division

Describes aspects of the program that depend on external resources in the execution environment.

It pertains to batch programs. Programs that run in a managed code environment such as CICS do not manage their own access to external resources; the runtime container does so on their behalf.

We will revisit the Environment Division when we begin writing programs that work with data sets.

Data Division

Dave Nicolette
Programmer

neopragma.com

COBOL Data Division

Contains the definitions of all the variables the program uses.

Unlike other programming languages, COBOL groups all variables together, separately from the source statements that represent computing operations.

For that reason, there is no notion of *scope* within a single COBOL source program. All variables are visible and modifiable by all the code in the Procedure Division.

We will introduce details of Data Division coding in the context of lab exercises.

Procedure Division

Dave Nicolette
Programmer

neopragma.com

COBOL Procedure Division

Contains all the source statements that the compiler will convert into executable object code.

The MINIMAL program we saw earlier will compile, but it will crash if we execute it. That's because it has no Procedure Division.

The simplest program *that will run* has a Procedure Division that does nothing except exit.

The simplest program that will run

The simplest program that will run is one that only exits back to the operating system without performing any other work.

```
Identification Division.
Program-Id. DONOTHIN.
Procedure Division.
    goback.
```

This will do exactly the same thing as the IEFBR14 utility program. Therefore, even the simplest practical COBOL program can be useful.

Lab 18:
Your own IEFBR14 utility

Follow the instructions for this lab in your copy of the course Github repository.

Reading and writing the standard streams

Dave Nicolette
Programmer

neopragma.com

"Hello, world!" redux

The sample HELLO program contains the following source code.

```
Identification Division.
Program-Id. HELLO.
Procedure Division.
    display "Hello, world!"
    goback
    .
```

What is going on here?

Commands or operations in COBOL are called *verbs*.

The program contains two Procedure Division lines. The verbs are *display* and *goback*.

"Hello, world!" redux

The sample HELLO program contains the following source code.

```
Identification Division.
Program-Id. HELLO.
Procedure Division.
    display "Hello, world!"
    goback
    .
```

The verb *display* writes whatever text follows it to the data set identified as SYSOUT in the JCL. That text can contain variable references and literals. We'll see that in a minute.

The verb *goback* returns to whatever program called this program. A main batch program is called by JES, so *goback* returns control to the JES initiator that's running this instance of program HELLO.

Demo:
Hello, friends!

We'll add functionality to the "Hello, world!" program.

The standard streams: SYSIN, SYSOUT, and SYSERR

It's likely that every operating system you've worked with so far has three standard input/output streams named *stdin*, *stdout*, and *stderr*. z/OS has them, too – they're native to the USS "side" of z/OS.

The MVS "side" of z/OS defines equivalent spooler data sets named *SYSIN*, *SYSOUT*, and *SYSERR*.

Demo:
Hello, friends! (if you're there)

We'll add still more functionality to the "Hello, world!" program.

Lab 19:
Using SYSIN, SYSOUT, SYSERR

Follow the instructions for this lab in your copy of the course Github repository.

COBOL coding style

Dave Nicolette
Programmer

neopragma.com

HELLO4

Our HELLO program has enough code for a discussion of COBOL coding style.

```
Command ===>                                                  Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001        Identification Division.
000002        Program-Id. HELLO4.
000003        Data Division.
000004        Working-Storage Section.
000005        01  Friend            pic x(30) value spaces.
000006        Procedure Division.
000007            accept Friend
000008            if Friend is equal to spaces
000009                display
000010                    "Oh, no! I have no friends."
000011            else
000012                perform until Friend is equal to spaces
000013                    display "Hello, " Friend "!"
000014                    move spaces to Friend
000015                    accept Friend
000016                end-perform
000017            end-if
000018            goback
000019            .
****** **************************** Bottom of Data ****************************
```

Original COBOL style

Originally, COBOL required this kind of coding style.

```
Command ===>                                                  Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001        IDENTIFICATION DIVISION.
000002        PROGRAM-ID. HELLO4.
000003        DATA DIVISION.
000004        WORKING-STORAGE SECTION.
000005        77  FRIEND              PIC X(30)
000006        PROCEDURE DIVISION.
000007            ACCEPT FRIEND.
000008            IF FRIEND IS EQUAL TO SPACES
000009                DISPLAY
000010                    "OH, NO! I HAVE NO F
000011            ELSE
000012                PERFORM UNTIL FRIEND IS EQUAL TO SPACES
000013                    DISPLAY "HELLO, " FRIEND "!"
000014                    MOVE SPACES TO FRIEND
000015                    ACCEPT FRIEND.
000016            STOP RUN.
****** **************************** Bottom of Data ****************************
```

All caps.

Period terminates each statement in the Procedure Division.

A 77-level Data Division entry indicated a standalone variable. This has been removed.

Lowercase text

Later, they added support for lowercase text. COBOL is case insensitive.

```
Command ===>                                                Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001        identification division.
000002        program-id. hello4.
000003        data division.
000004        working-storage section.
000005        77  friend              pic x(30) value spaces.
000006        procedure division.
000007            accept friend.
000008            if friend is equal to spaces
000009                display
000010                    "Oh, no! I have no friends."
000011            else
000012                perform until friend is equal to spaces
000013                    display "Hello, " friend "!"
000014                    move spaces to friend
000015                    accept friend.
000016            stop run.
****** **************************** Bottom of Data ****************************
```

Statement termination

You don't have to terminate each statement with a period.

```
Command ===>                                                  Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001        Identification Division.
000002        Program-Id. HELLO4.
000003        Data Division.
000004        Working-Storage Section.
000005        01  Friend             pic x(30) value spaces.
000006        Procedure Division.
000007            accept Friend
000008            if Friend is equal to spaces
000009                display
000010                    "Oh, no! I have no friends."
000011            else
000012                perform until Friend is equal to spac
000013                    display "Hello, " Friend "!"
000014                    move spaces to Friend
000015                    accept Friend
000016                end-perform
000017            end-if
000018            goback
000019            .
****** **************************** Bottom of Data ****************************
```

A period is required at the end of each paragraph. It can be on a separate line.

Use explicit scope terminators – end-if, end-perform, etc.

Use prefixes on variable names

Many organizations have conventions or rules for variable name prefixes.

Some people prefix variables in the Working-Storage Section with "WS", those in the Linkage Section with "LS", and so on.

Use prefixes on variable names

Prefixes might suggest the way the variables are grouped together.

```
Command ===>                                                Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001        Identification Division.
000002        Program-Id. HELLO4.
000003        Data Division.
000004        Working-Storage Section.
000005        01  F-Friend-Info.
000006            05  F-Name        pic x(30) value spaces.
000007        Procedure Division.
000008            accept F-Name
000009            if F-Name is equal to spaces
000010                display
000011                    "Oh, no! I have no friends."
000012            else
000013                perform until F-Name is equal to space
000014                    display "Hello, " F-Name "!"
000015                    move spaces to F-Name
000016                    accept F-Name
000017                end-perform
000018            end-if
000019            goback
000020            .
****** **************************** Bottom of Data ****************************
```

Organizing variables that logically belong together and prefixing their names with a meaningful value helps us locate code we're looking for in a source program.

Lab 20:
COBOL coding styles

Follow the instructions for this lab in your copy of the course Github repository.

Working with text

Dave Nicolette
Programmer

neopragma.com

Business application processing

COBOL is designed for business applications.

Business applications primarily manipulate three kinds of information:

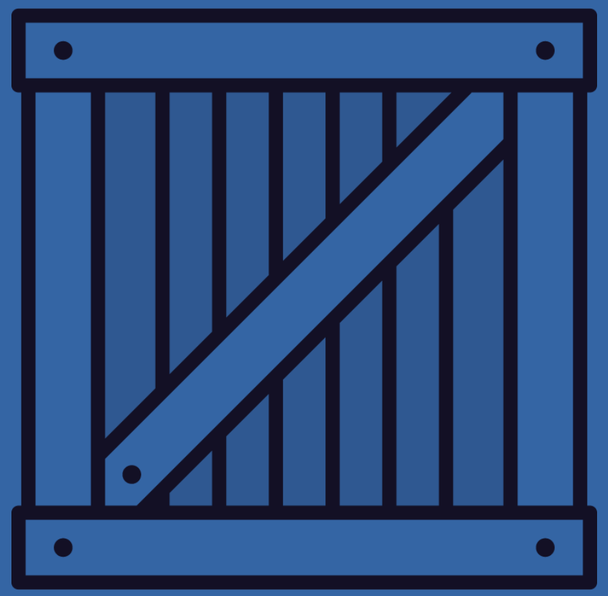
- Text or "character" data or strings
- Decimal values, such as monetary amounts
- Dates and times

Let's take a look at COBOL features for working with text, and how this kind of data is represented in memory on z/OS.

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
```

◂**We need to define a variable. Therefore, we need a Data Division in our program.**



Sections in the Data Division

The Data Division may contain one or more of the following Sections:

- Working-Storage
- Local-Storage
- Linkage

The Working-Storage Section contains variable definitions for the program's internal use. The area is not visible to other programs.

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
```

◂We'll define our program's variables in the Working-Storage Section of the Data Division. All code in the Procedure Division can "see" these definitions.

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
```

- **A COBOL variable name can be up to 30 characters long. It can consist of letters, numbers, hyphens (dashes). It cannot begin with a number, and cannot begin or end with a hyphen. You can't write two hyphens together. Names are case-insensitive, so "soCIal" is the same as "SociaL".**

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
```

- **Variables are organized hierarchically. Each definition has a number, called a "level number," between 1 and 49. Most people write the leading zero for numbers 1 through 9. Higher levels are subordinate to lower ones. We'll see how this works in a minute.**

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
         usage is display
           - or -
         usage display
           - or -
         display
           - or -
```

- The "usage clause" tells the compiler how we intend to use the variable.
- You can omit the word "is"
- You can omit the word "usage"
- Actually, this is the default so you can omit the word "display," too.

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
        picture is x


           - or -
        picture x
           - or -
        pic x
```

- The picture clause tells the compiler the format of the data in this variable. The "x" means "anything". It could be letters or numbers or even non-display values.
- You can omit the word "is".
- You can abbreviate "picture" to "pic".

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
        pic xxxxxxxxx


           - or -
        pic x(9)
```

- An "x" in a picture clause represents one byte of memory. A Social Security Number is nine digits long, so we need to specify nine bytes.
- You can indicate the number of x's in parentheses.

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
         value is "           "
           - or -
         value is spaces


           - or -
         value spaces
```

- You can provide a default or initial value for a variable by coding the "value" clause.
- COBOL lacks genuine constants, but it defines several "figurative constants" you can use in contexts like this.
- ...and you can omit the word "is".

IBM Enterprise COBOL figurative constants

ZERO, ZEROS, ZEROES	The value *zero* in the format indicated by the picture clause of the Data Division entry.
SPACE, SPACES	The value of a blank space in EBCDIC (x'40').
HIGH-VALUE, HIGH-VALUES	The maximum value in the collating sequence (x'FF').
LOW-VALUE, LOW-VALUES	The minimum value in the collating sequence (x'00').
QUOTE, QUOTES	One or more quote characters. The value depends on the settings of the compiler option QUOTE (double quote) or APOST (single quote).

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
 01  Social-Security-Number
         usage is display
         picture is x(9)
         value is spaces.
           - or -
 01  Social-Security-Number pic x(9)
                            value spaces.
```

◂ **The item can be defined verbosely like this...**

◂ **...or concisely like this.**

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
```

◂**The definitions are officially called "data division entries," but most people call them "variables."**

Every data division entry must be terminated by a period.

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
 Procedure Division.
```

- **The Procedure Division is where we code COBOL statements the compiler will translate into executable instructions.**

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
 Procedure Division.
     move "987654321"
          to Social-Security-Number
```

- **The COBOL equivalent of an assignment statement is the "move" statement. This example moves a literal value to the variable, Social-Security-Number.**

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
 Procedure Division.
    move "987654321"
         to Social-Security-Number
    display Social-Security-Number
```

- **You've seen the display statement before. Here we're outputting the value of Social-Security-Number to wherever SYSOUT is directed in the JCL.**

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
 Procedure Division.
    move "987654321"
         to Social-Security-Number
    display Social-Security-Number
    goback
    .
```

◂**Finally, we use goback to return control to JES. There has to be a period at the end of the source. It could be right after the word "goback" or on a separate line, as in this example.**

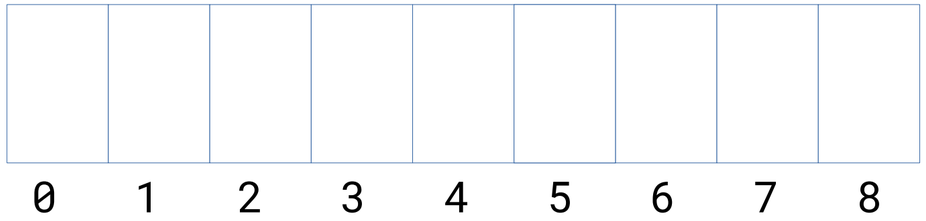
What does this look like in memory?

```
   1         2         3         4
78901234567890123456789012345678901234567

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.

 Procedure Division.
     move "123456789"
          to Social-Security-Number
     display Social-Security-Number
     goback
     .
```

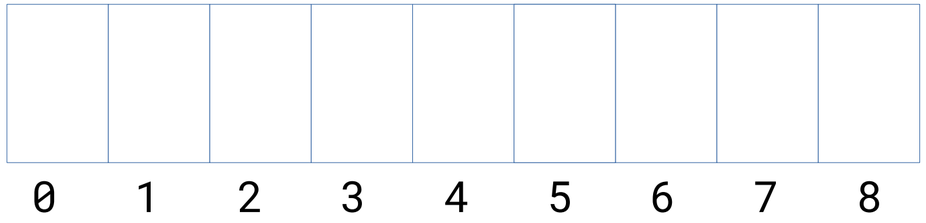
The Working-Storage Section is a single memory allocation. References in the Procedure Division to variables are converted into an offset from the start of Working-Storage and a length.



◂The name "Social-Security-Number" corresponds to offset 0 from the start of Working-Storage for a length of 9 bytes (due to the picture clause).

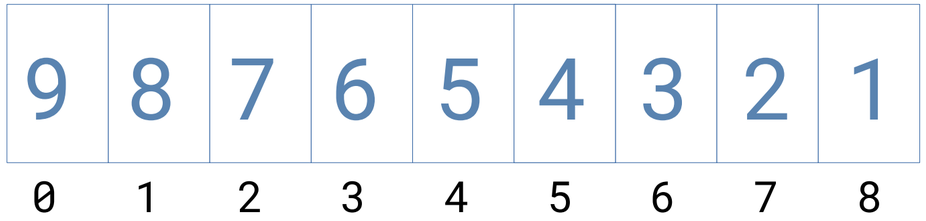
```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                                value spaces.
 Procedure Division.
    move "987654321"
         to Social-Security-Number
    display Social-Security-Number
    goback
    .
```



- **When the program starts, the Working-Storage area from offset 0 for length 9 is set to EBCDIC spaces, x'40'.**

```
   1         2         3         4
78901234567890123456789012345678901234567
 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
 Procedure Division.
     move "987654321"
          to Social-Security-Number
     display Social-Security-Number
     goback
     .
```



- **The move statement modifies the contents of the Working-Storage area from offset 0 for length 9 .**

A quick checkpoint

Given a Data Division entry like this...

```
01  A-FIELD          PICTURE IS X(100).
```

...when the program is loaded into memory, the contents of A-FIELD will be

- **SPACES**
- **Binary zeroes (x'00')**
- **whatever happened to be in those virtual memory addresses**
- **N/A, the program won't compile without a VALUE clause**

A quick checkpoint

Given a Data Division entry like this...

```
01  A-FIELD          PICTURE IS X(100).
```

...when the program is loaded into memory, the contents of A-FIELD will be

SPACES

Binary zeroes (x'00')

whatever happened to be in those virtual memory addresses

N/A, the program won't compile without a VALUE clause

A quick checkpoint

```
01  A-FIELD          PICTURE IS X(100).
. . .
MOVE "ALPHA" TO A-FIELD.
```

After the MOVE, A-FIELD will contain...

"ALPHA" followed by 95 spaces

"ALPHA" followed by whatever was already in positions 6-100

"ALPHA" followed by 95 bytes of binary zeroes (x'00')

N/A, the program won't compile because "ALPHA" is less than 100 bytes long

A quick checkpoint

```
01   A-FIELD          PICTURE IS X(100).
. . .
MOVE "ALPHA" TO A-FIELD.
```

After the MOVE, A-FIELD will contain...

"ALPHA" followed by 95 spaces

"ALPHA" followed by whatever was already in positions 6-100

"ALPHA" followed by 95 bytes of binary zeroes (x'00')

N/A, the program won't compile because "ALPHA" is less than 100 bytes long

A quick checkpoint

```
01   A-GROUP.
     05  FIELD-1            PIC X(5) VALUE "GAMMA".
     05  FIELD-2            PIC X(5) VALUE "DELTA".
. . .
MOVE "EXPONENTIAN" TO FIELD-1.
```

A-GROUP contains "EXPONENTIA"

A-GROUP contains "EXPONDELTA"

A-GROUP contains "EXPON "

N/A, the program won't compile because "EXPONENTIAL" is misspelled

A quick checkpoint

```
01   A-GROUP.
     05  FIELD-1            PIC X(5) VALUE "GAMMA".
     05  FIELD-2            PIC X(5) VALUE "DELTA".
. . .
MOVE "EXPONENTIAN" TO FIELD-1.
```

A-GROUP contains "EXPONENTIA"

A-GROUP contains "EXPONDELTA"

A-GROUP contains "EXPON "

N/A, the program won't compile because "EXPONENTIAL" is misspelled

Formatted Social Security Number

Social Security Numbers don't look like this:

123456789

They look like this:

123-45-6789

How do we do that in COBOL?

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number pic x(9)
                             value spaces.
 Procedure Division.
    move "987654321"
         to Social-Security-Number
    display
        Social-Security-Number(1:3) "-"
        Social-Security-Number(4:2) "-"
        Social-Security-Number(6:4)
    goback
    .
```

- One way to get formatted output is to use reference modification. This is COBOL's equivalent to a substring. The first number is the offset from the start of the variable relative to 1. The second number is the number of bytes to operate on. Result:

  987-65-4321

Reference Modification

```
Variable-Name(offset:length)
```

```
   1         2         3         4
7890123456789012345678901234567890123456

 Identification Division.
 Program-Id. PGMNAME.
 Data Division.
 Working-Storage Section.
  01  Social-Security-Number value spaces.
      05  SSN-Area            pic x(3).
      05  SSN-Group           pic x(2).
      05  SSN-Serial          pic x(4).
 Procedure Division.
    move "987654321"
         to Social-Security-Number
    display
        SSN-Area "-" SSN-Group "-"
        SSN-Serial
    goback
    .
```

Another way is to break down the subfields of the Social Security number.

Result:

987-65-4321

The 05-level items are subordinate to the 01-level item. As all three fields are at the same level, they follow each other in Working-Storage.

The move statement refers to the entire Social-Security-Number 01-level item. It's called a "group item."

The display statement refers to the individual "elementary" 05-level items.

Group items and elementary items

Items with higher levels are subordinate to those with lower levels.

Value clauses and Procedure Division statements can refer to any of the item names.

```
Working-Storage Section.
 01  SSN value spaces.
     05  SSN-Area              pic x(3).
     05  SSN-Group             pic x(2).
     05  SSN-Serial            pic x(4).
 01  Formatted-SSN.
     05  SSN-Area              pic x(3).
     05  filler          pic x value "-".
     05  SSN-Group             pic x(2).
     05  filler          pic x value "-".
     05  SSN-Serial            pic x(4).
Procedure Division.
   move "987654321" to SSN
   move SSN-Area of SSN
     to SSN-Area of Formatted-SSN
   move SSN-Group of SSN
     to SSN-Group of Formatted-SSN
   move SSN-Serial of SSN
     to SSN-Serial of Formatted-SSN
   display Formatted-SSN
   goback
   .
```

A third way is to define the formatted SSN as a separate Data Division entry and copy values using qualified names.

Result:

987-65-4321

Note the use of "filler."

Filler and qualified names

Filler defines storage not directly referenced by the program.

Qualified names distinguish between variables with the same name that belong to different group items.

```
Working-Storage Section.
 01  SSN value spaces.
     05  SSN-Area            pic x(3).
     05  SSN-Group           pic x(2).
     05  SSN-Serial          pic x(4).
 01  Formatted-SSN.
     05  SSN-Area            pic x(3).
     05  filler       pic x value "-".
     05  SSN-Group           pic x(2).
     05  filler       pic x value "-".
     05  SSN-Serial          pic x(4).
Procedure Division.
   move "987654321" to SSN
   move corr SSN to Formatted-SSN
   display Formatted-SSN
   goback
   .
```

A fourth way is to use move corresponding (or move corr) to move like-named elementary items between group items.

Result:

987-65-4321

- Data Division level numbers
- Group and elementary items
- Usage, picture, value clauses
- Using filler
- Reference modification
- Qualified names
- Move corresponding

Checkpoint

Truncation and padding

Dave Nicolette
Programmer

neopragma.com

Sample MOVE statements and their effects

```
01  Group-1.
    05  Text-Field-1        pic x(5).
    05  Group-2.
        10  Text-Field-2    pic x(5).
        10  Text-Field-3    pic x(5).
    05  Text-Field-4        pic x(5).
```

```
 SDSF OUTPUT DISPLAY USER270S JOB00352  DSID   101 LINE 0
 COMMAND INPUT ===>
******************************** TOP OF DATA ************
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ to Text-Field-1
                1         2
        12345678901234567890123
Group-1: ABCDE
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1
                1         2
        12345678901234567890123
Group-1: ABCDEFGHIJKLMNOPQRST
after move '9999999999' to Text-Field-3
                1         2
        12345678901234567890123
Group-1: ABCDEFGHIJ99999PQRST
after move '72' to Text-Field-2
                1         2
        12345678901234567890123
Group-1: ABCDE72   99999PQRST
******************************** BOTTOM OF DATA **********
```

```
move spaces to Group-1.
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Text-Field-1
display "after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        " to Text-Field-1"
display "                  1         2"
display "         12345678901234567890123"
display "Group-1: " Group-1
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Group-1
display "after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1"
display "                  1         2"
display "         12345678901234567890123"
display "Group-1: " Group-1
move "9999999999" to Text-Field-3
display "after move '9999999999' to Text-Field-3"
display "                  1         2"
display "         12345678901234567890123"
display "Group-1: " Group-1
move "72" to Text-Field-2
display "after move '72' to Text-Field-2"
display "                  1         2"
display "         12345678901234567890123"
display "Group-1: " Group-1
```

Sample MOVE statements and their effects

```
01  Group-1.
    05  Text-Field-1        pic x(5).
    05  Group-2.
        10  Text-Field-2    pic x(5).
        10  Text-Field-3    pic x(5).
    05  Text-Field-4        pic x(5).
```

```
 SDSF OUTPUT DISPLAY USER270S JOB00352  DSID    101 LINE 0
 COMMAND INPUT ===>
******************************** TOP OF DATA ************
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ to Text-Field-1
                  1         2
         12345678901234567890123456
Group-1: ABCDE
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1
                  1         2
         12345678901234567890123456
Group-1: ABCDEFGHIJKLMNOPQRST
after move '9999999999' to Text-Field-3
                  1         2
         12345678901234567890123456
Group-1: ABCDEFGHIJ99999PQRST
after move '72' to Text-Field-2
                  1         2
         12345678901234567890123456
Group-1: ABCDE72   99999PQRST
******************************** BOTTOM OF DATA **********
```

```
move spaces to Group-1.
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Text-Field-1
display "after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        " to Text-Field-1"
display "                  1         2"
display "         12345678901234567890123456"
display "                  1         2"
display "         12345678901234567890123456"
display "Group-1: " Group-1
move "72" to Text-Field-2
display "after move '72' to Text-Field-2"
display "                  1         2"
display "         12345678901234567890123456"
display "Group-1: " Group-1
```

Text-Field-1 occupies the first 5 bytes of Group-1.
Moving a 26-byte value to Text-Field-1 causes the first 5 bytes of the value to be copied, and the remainder truncated.
Fields beyond Text-Field-1 are not affected.

Sample MOVE statements and their effects

```
01  Group-1.
    05  Text-Field-1        pic x(5).
    05  Group-2.
        10  Text-Field-2    pic x(5).
        10  Text-Field-3    pic x(5).
    05  Text-Field-4        pic x(5).
```

```
 SDSF OUTPUT DISPLAY USER270S JOB00352  DSID   101 LINE 0
 COMMAND INPUT ===>
******************************** TOP OF DATA ************
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ to Text-Field-1
                1         2
       123456789012345678901234
Group-1: ABCDE
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1
                1         2
       123456789012345678901234
Group-1: ABCDEFGHIJKLMNOPQRST
after move '9999999999' to Text-Field-3
                1         2
       123456789012345678901234
Group-1: ABCDEFGHIJ99999PQRST
after move '72' to Text-Field-2
                1         2
       123456789012345678901234
Group-1: ABCDE72   99999PQRST
******************************** BOTTOM OF DATA **********
```

```
move spaces to Group-1.
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Text-Field-1
di
di
di
di
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Group-1
display "after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1"
display "                    1         2"
display "           123456789012345678901234"
display "Group-1: " Group-1
move "9999999999" to Text-Field-3
display "after move '9999999999' to Text-Field-3"
display "                    1         2"
display "           123456789012345678901234"
display "Group-1: " Group-1
move "72" to Text-Field-2
display "after move '72' to Text-Field-2"
display "                    1         2"
display "           123456789012345678901234"
display "Group-1: " Group-1
```

Moving a 10-byte value to Text-Field-3 results in the first 5 bytes of the value being copied into Text-Field-3. No other parts of Group-1 are affected.

Sample MOVE statements and their effects

```
01  Group-1.
    05  Text-Field-1        pic x(5).
    05  Group-2.
        10  Text-Field-2    pic x(5).
        10  Text-Field-3    pic x(5).
    05  Text-Field-4        pic x(5).
```

```
 SDSF OUTPUT DISPLAY USER270S JOB00352  DSID   101 LINE 0
 COMMAND INPUT ===>
******************************** TOP OF DATA ************
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ to Text-Field-1
                1         2
       123456789012345678901234 5
Group-1: ABCDE
after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1
                1         2
       123456789012345678901234 5
Group-1: ABCDEFGHIJKLMNOPQRST
after move '9999999999' to Text-Field-3
                1         2
       123456789012345678901234 5
Group-1: ABCDEFGHIJ99999PQRST
after move '72' to Text-Field-2
                1         2
       123456789012345678901234 5
Group-1: ABCDE72   99999PQRST
******************************** BOTTOM OF DATA **********
```

```
move spaces to Group-1.
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Text-Field-1
display "after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        " to Text-Field-1"
display "                    1         2"
display "          123456789012345678901234 5"
display "Group-1: " Group-1
move "ABCDEFGHIJKLMNOPQRSTUVWXYZ" to Group-1
display "after move 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' to Group-1"
di
di
di
mo
di
di
display "          123456789012345678901234 5"
display "Group-1: " Group-1
move "72" to Text-Field-2
display "after move '72' to Text-Field-2"
display "                    1         2"
display "          123456789012345678901234 5"
display "Group-1: " Group-1
```

Moving a 2-byte value to Text-Field-2 results in the first 2 bytes of the value being copied into Text-Field-2 and the rest of the field padded with spaces. No other parts of Group-1 are affected.

Truncation and padding with MOVE

Moving a value to a group item fills all the subordinate items.

A value longer than the destination item is truncated to the declared length of the item.

The destination item is padded with spaces when the source value is shorter than its declared length.

Lab 21:
Reference Modification

Follow the instructions for this lab in your copy of the course Github repository.

Special level numbers

Dave Nicolette
Programmer

neopragma.com

Special level numbers

Levels 1 through 49 are for organizing Data Division entries hierarchically.

Level 66 "renames" establishes a different name and breakdown of contiguous entries – a different view of those bytes in memory.

Level 77 is an obsolete label for an elementary entry that is not part of a group. You will see it in older programs, but use 01 for such entries in new programs.

Level 88 is COBOL's version of a boolean value. It's a little different from other languages.

66-level RENAMES

```
01  Social-Security-Number value spaces.
    05  SSN-Area           pic x(3).
    05  SSN-Group          pic x(2).
    05  SSN-Serial         pic x(4).
03  Employee-Fields.
    05  EMP-EmploymentDate pic x(8).
    05  EMP-Surname5       pic x(5).
    05  EMP-Tiebreak       pic x(5).
66  Employee-Id renames
    SSN-Serial through EMP-Tiebreak.
```

```
Procedure Division.
    move "987654321" to Social-Security-Number
    move "20231108" to EMP-EmploymentDate
    move "SMITH" to EMP-Surname5
    move "00003" to EMP-Tiebreak
    display "Employee-Id: " Employee-Id
    goback
    .
```

```
SDSF OUTPUT DISPLAY USER270S JOB0035
COMMAND INPUT ===>
********************************** TOP
Employee-Id: 432120231108SMITH00003
********************************** BOTT
```

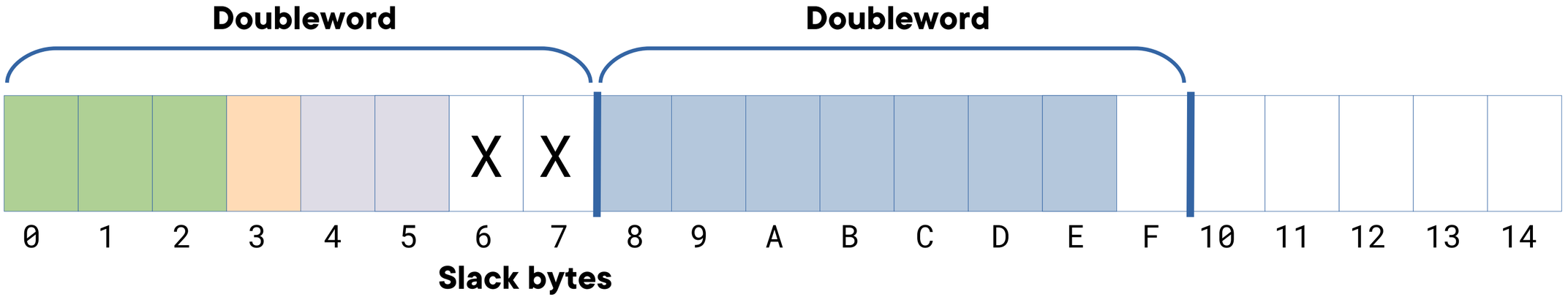
66-level RENAMES rules

The entries have to be contiguous.

The renamed entries can't cross an 01-level entry.

The 66-level entry doesn't have a usage or picture clause. It's only an alternate view of the memory area containing the fields.

66-level RENAMES and boundary alignment




```
01   Group-1.
     03  Field-2 pic x(3).
     03  Field-3 pic x.
     03  Field-4 pic x(2).
01   Group-2.
     03  Field-5 pic x(7).
66   Group-3 renames
     Field-4 through Field-5.
```

COBOL starts each 01-level entry at the next doubleword boundary.

Because of possible slack bytes, the compiler can't ensure the named entries will be contiguous in memory.

Therefore, the compiler rejects code like this.

Demo:
66-level code

We'll walk through the sample program, SSNDEMO, to see how 66-level code looks and works.

```
Working-Storage Section.
01  State-of-Happiness      pic x.
    88  Is-Happy       value "H".
        when set to false is "U".
01  State-of-Contentment    pic x.
    88  Is-Content     value "C".
    88  Could-Be-Worse value "W".
01  State-of-Satisfaction   pic x.
    88  Is-Satisfied   value "S".
    88  Cant-Get-No    value "N".
Procedure Division.
   set Is-Happy to false
   move spaces to State-of-Contentment
                  State-of-Satisfaction
   perform Assess-Life-Situation
   if Is-Happy and Is-Content
                and Is-Satisfied
       [do something]
   else
       [do something else]
   end-if
   goback
   .
```

88-level quasi-boolean

- The 88-level entry is directly under the item it's associated with.

- Setting Is-Happy to false puts "U" in State-of-Happiness. You can't directly test the "false" condition in an IF statement.

```
Working-Storage Section.
01  State-of-Happiness      pic x.
    88  Is-Happy       value "H".
        when set to false is "U".
01  State-of-Contentment    pic x.
    88  Is-Content     value "C".
    88  Could-Be-Worse value "W".
01  State-of-Satisfaction   pic x.
    88  Is-Satisfied   value "S".
    88  Cant-Get-No    value "N".
Procedure Division.
   set Is-Happy to false
   move spaces to State-of-Contentment
                  State-of-Satisfaction
   perform Assess-Life-Situation
   evaluate true also true also true
       when Is-Happy and Is-Content
                     And Is-Satisfied
           [do something]
       when other
           [do something else]
   end-evaluate
   goback
```

88-level quasi-boolean

- **The 88-level entry is directly under the item it's associated with.**

- **EVALUATE is an alternative to IF/ELSE.**

Demo:
88-level quasi-boolean

We'll walk through the sample program, SSNDEMO, to see how 88-level code looks and works.

66-level and 88-level code

66-level RENAMES

88-level quasi-boolean

IF/ELSE

EVALUATE

PERFORM

Word, Doubleword, Quadword

Boundary alignment

REDEFINES

Inspect, String, Unstring

Dave Nicolette
Programmer

neopragma.com

More text processing features

INSPECT

Search and modify text in a string

STRING

Assemble multple values into a single string

UNSTRING

Break a string into multiple values based on delimiters

INSPECT: Count occurrences of a substring in a string

```
 Working-Storage Section.
 01  The-Text pic x(50) value
    "The quick brown fox jumps over the
lazy dog.".
 01  The-Count pic 9(4) binary.

 Procedure Division.
     move zero to The Count
     inspect The-Text
         tallying
             The-Count for all "The"
             The-Count for all "the"
     goback
     .
```

- Initialize the tally field or it will continue to increment across multiple INSPECT statements.
- You can include multiple substrings with a single INSPECT statement. This example counts both capitalized and lower-case instances of "the".
- Note: There is no END-INSPECT.

INSPECT: Replace occurrences of a substring in a string

```
Working-Storage Section.
01  The-Text pic x(50) value
   "That night the night watchman slept".
01  The-Count pic 9(4) binary.

Procedure Division.
    inspect The-Text
        replacing first "night"
        by "afternoon"
    inspect The-Text
        Replacing all spaces by "-"

    goback
    .
```

- You can replace the first, last, or all occurrences of a substring.

INSPECT: Find the first non-blank character in a string

```
Working-Storage Section.
01  The-Text pic x(50) value
    "    Something wicked this way comes".
01  Spaces-Count pic 9(4) binary.

Procedure Division.
    move zero to Spaces-Count
    inspect The-Text
        tallying Spaces-Count
            for leading space
    add 1 to Spaces-Count
    move The-Text(Spaces-Count:)
        to The-Text
    goback
    .
```

◂ **You can count leading or trailing values.**

STRING: Assemble parts into a whole

```
Working-Storage Section.
01  Street-Address.
    05  Addr-Street      pic x(30).
    05  Addr-Number      pic x(10).
    05  Addr-Zipcode     pic x(9).
01  Display-Address      pic x(100).

Procedure Division.
    string
        Addr-Street delimited by space
        space delimited by size
        Addr-Number delimited by space
        space delimited by size
        Addr-Zipcode(1:5) delimited by size
        "-" delimited by size
        Addr-Zipcode(6:4) delimited by size
    end-string
    goback
    .
```

- Removes trailing spaces.
- Adds separator space.
- Removes trailing spaces.
- Adds separator space.
- Concatenates first 5 digits of zipcode
- Adds a dash
- Concatenates last 4 digits of zipcode

UNSTRING: Split a string based on one or more delimiters

```
Working-Storage Section.
01  Comma-Separated-Names pic x(300).
01  Name-Count           pic 9(4) binary.
01  Name-List.
    05  Persons-Name     pic x(30)
        occurs 20 times indexed by Name-Ix.
Procedure Division.
    move zero to Name-Count
    unstring Comma-Separated-Names
        Delimited by ","
      Into
        Persons-Name(1)
        Persons-Name(2)
        Persons-Name(3)
        Persons-Name(4)
        Persons-Name(5)
      Tallying in Name-Count
    end-unstring
```

◂ **Extracts values between commas.**

Demo:
INSPECT, STRING, and UNSTRING

We'll walk through the sample program, STRING, to see examples of INSPECT, STRING, and UNSTRING.

Lab 22:
INSPECT, STRING, and UNSTRING

Follow the instructions for this lab in your copy of the course Github repository.

Decimal arithmetic

Dave Nicolette
Programmer

neopragma.com

COBOL support for decimal arithmetic

English-like statements

ADD, SUBTRACT, MULTIPLY, DIVIDE

The COMPUTE statement

COMPUTE a = (b + c) / d

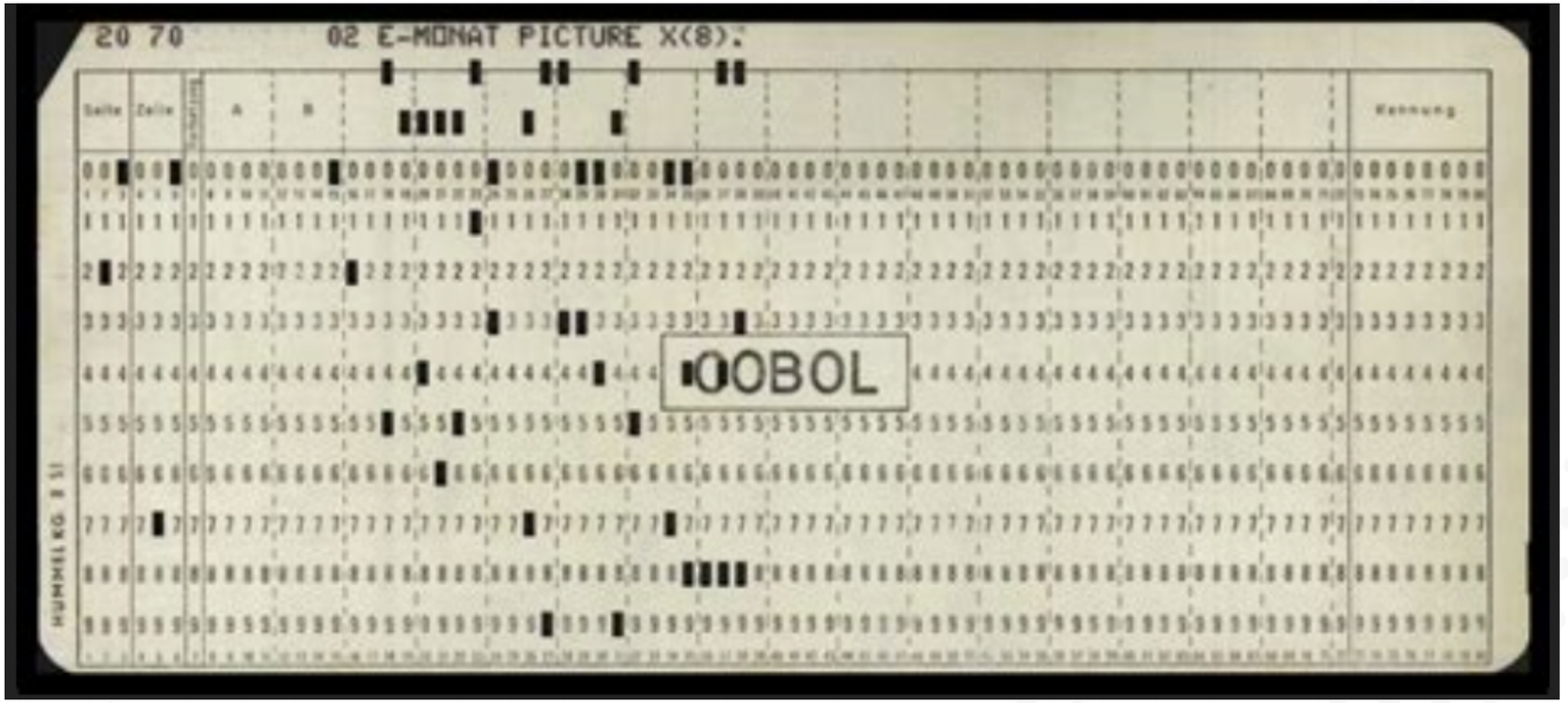
Intrinsic Functions

ABS, SQRT, MOD, others

Demo:
Decimal arithmetic in COBOL

We'll walk through the sample program, ARITH, to see examples of Data Division and Procedure Division code for decimal arithmetic.

What is an overpunch?



The Packed Decimal format

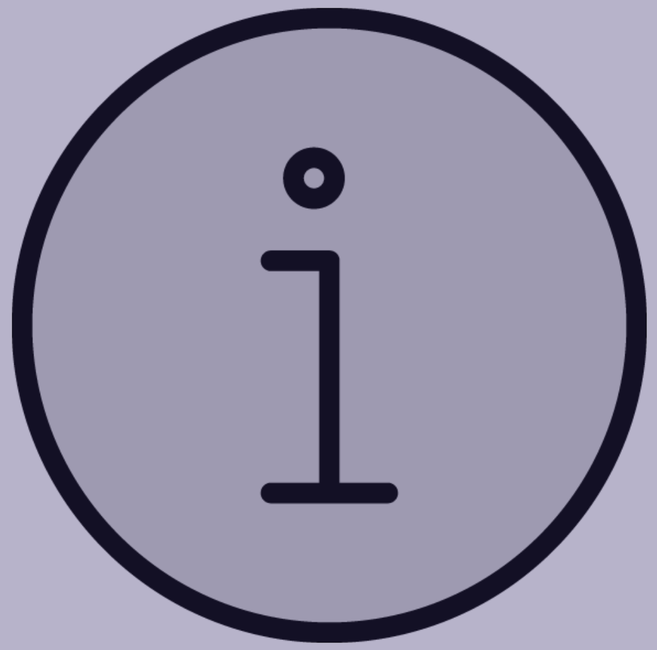
Value	PIC S9(8) DISPLAY	PIC S9(7) PACKED-DECIMAL
+425396	F0 F0 F4 F2 F5 F3 F9 C6	04 25 39 6C
-1832	F0 F0 F0 F0 F1 F8 F3 D2	00 01 83 2D
933405216	F3 F3 F4 F0 F5 F2 F1 F6	34 05 21 6F

Demo:
S0C7 abend – Data Exception

We'll walk through the sample program, S0C7, to see how a S0C7 abend can be caused and what to do about it.

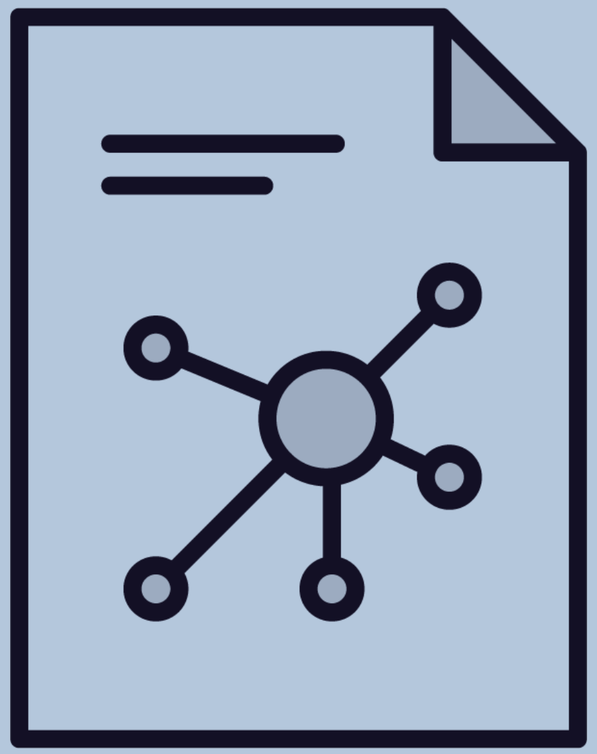
Some key concepts and terms

- condition code
- completion code
- abend (abnormal end)
- system codes
- program interrupt



Some key concepts and terms

- condition code
- completion code
- system messages
- abend (abnormal end)
- system codes
- program interrupt



Useful documentation

Messages and Codes

https://www.ibm.com/docs/en/zos/3.2.0?topic=zos-mvs

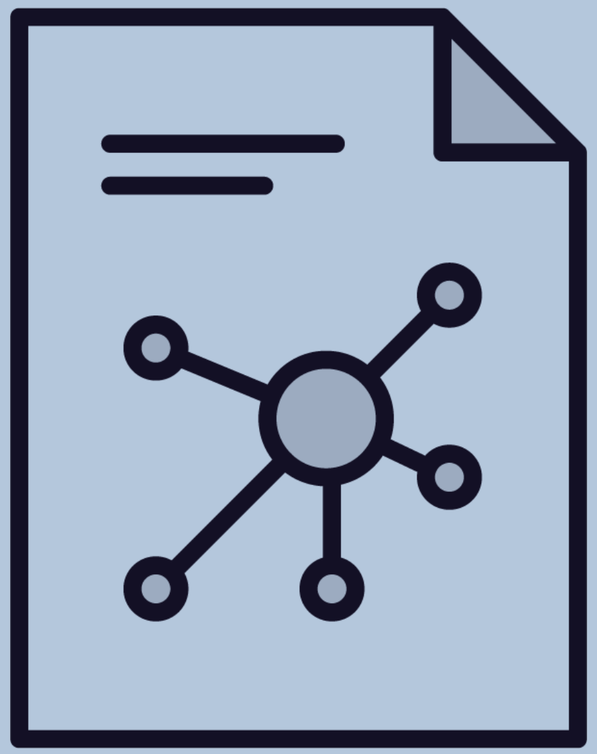
Completion (Abend) Codes

https://www.ibm.com/docs/en/SSLTBW_3.1.0/pdf/ieah700_v3r1.pdf

Program Interrupt Codes

https://www.ibm.com/docs/en/zos/3.2.0?topic=conditions-program-interrupt-abend-reason-codes

CEEDUMP and CEE3DUMP Info



z/OS Language Environment Debugging Guide

https://www.ibm.com/docs/en/SSLTBW_2.5.0/pdf/ceea100_v2r5.pdf

z/OS Language Environment Vendor Interfaces

https://www.ibm.com/docs/en/SSLTBW_3.1.0/pdf/ceev100_v3r1.pdf

S0C7 abend

Overriding COBOL compiler options

Interpreting a CEEDUMP

Avoiding S0C7 abends

Lab 23:
Decimal Arithmetic

Follow the instructions for this lab in your copy of the course Github repository.

Dates and times

Dave Nicolette
Programmer

neopragma.com

Demo:
Handling dates and times

We'll walk through the sample program, DATES, to see various techniques for working with dates and times in COBOL.

COBOL tables

Dave Nicolette
Programmer

neopragma.com

COBOL tables – a simple table definition

```
Working-Storage Section.
01  Word-List.
    05  A-Word occurs 5 times pic x(6).
Procedure Division.
    move spaces       to Word-List
    move "How"        to A-Word(1)
    move "now"        to A-Word(2)
    move "brown"      to A-Word(3)
    move "cow"        to A-Word(4)

    display "Word 3 is " A-Word(3)
```

- The OCCURS phrase defines a table.
- The group entry addresses the whole table.
- Individual elements in the table are addressed using a subscript.
- The compiler will not allow subscript references larger than the occurs value, but it is still possible to violate this rule at run time.

COBOL tables – iterate over table with PERFORM VARYING

```
Working-Storage Section.
01  Word-List.
    05  A-Word occurs 5 times pic x(6).
01  Subscripting-Values.
    05  Word-Sub pic s9(4) binary.
    05  Word-Max pic s9(4) binary value 5.
Procedure Division.
    move spaces      to Word-List
    perform with test before
        varying Word-Sub from 1 by 1
        until Word-Sub > Word-Max
      if A-Word(Word-Sub) > spaces
         display "Word " Word-Sub
                 is A-Word(Word-Sub)
      else
         Display "Word " Word-Sub " empty"
      end-if
    end-perform
```

- Assume the table has been populated.
- A subscript can be any numeric type.
- You can use PERFORM VARYING to process the entries in a table.

COBOL tables – initialize table statically via REDEFINES

```
Working-Storage Section.
01  Word-Values.
    05  filler pic x(6) value "How".
    05  filler pic x(6) value "now".
    05  filler pic x(6) value "brown".
    05  filler pic x(6) value "cow".
    05  filler pic x(6) value spaces.
01  Word-List redefines Word-Values.
    05  A-Word occurs 5 times pic x(6).
01  Subscripting-Values.
    05  Word-Sub pic s9(4) binary.
    05  Word-Max pic s9(4) binary value 5.
```

- Assume the table has been populated.
- A subscript can be any numeric type.
- If the contents of the table are static, they can be defined in the program source. Use REDEFINES to overlay the table definition on top of the values.

COBOL tables – populate from data set records

```
Working-Storage Section.
01  Word-List.
    05  A-Word occurs 5 times pic x(6).
01  Subscripting-Values.
    05  Word-Sub pic s9(4) binary.
    05  Word-Max pic s9(4) binary value 5.
Procedure Division.
    perform Read-Word-File
    if not End-of-File
      perform with test before
        varying Word-Sub from 1 by 1
        until Word-Sub > Word-Max
      move Word-from-Record to A-Word(Word-Sub)
      perform Read-Word-File
        ff End-of-File
          exit perform
        end-if
      end-perform
    end-if
```

- Assume the table has been populated.
- A subscript can be any numeric type.
- If the contents of the table are dynamic, they can be read from an external source such as a file.

COBOL tables – populate from comma-separated list

```
Working-Storage Section.
01  Word-CSV   pic x(120).
01  Word-Values value spaces.
    05  Word-1 pic x(6).
    05  Word-2 pic x(6).
    05  Word-3 pic x(6).
    05  Word-4 pic x(6).
    05  Word-5 pic x(6).
01  Word-List redefines Word-Values.
    05  A-Word occurs 5 times pic x(6).
01  Subscripting-Values.
    05  Word-Sub pic s9(4) binary.
    05  Word-Max pic s9(4) binary value 5.
Procedure Division.
    perform Read-CSV-Record
    unstring Word-CSV
      Delimited by ","
      Into Word-1 Word-2 Word-3 Word-4 Word-5
    end-unstring
```

◂ **Populating the table from a comma-separated list of words obtained from an external source.**

COBOL tables – populate from comma-separated list

```
Working-Storage Section.
01  Monthly-Sales.
    05  MS-Sales occurs 10.
        10  MS-Name         pic x(20).
        10  MS-Month occurs 12.
            15  MS-Amount occurs 10
                pic s9(7)v9(2) packed-decimal.

01  Subscripting-Values.
    05  Name-Sub pic s9(4) binary.
    05  Name-Max pic s9(4) binary value 10.
    05  Month-Sub pic s9(4) binary.
    05  Month-Max pic s9(4) binary value 12.
    05  Amount-Sub pic s9(4) binary.
    05  Amount-Max pic s9(4) binary value 10.
Procedure Division.
    move MS-Name(10) to Someones-Name
    move MS-Month(10, 2) to One-Months-Data
    move MS-Amount(10,2,4) to One-Sale-Amount
```

- A COBOL table can have a maximum of three levels.

- Subscripts refer to levels 1, 2, and 3 and return the corresponding segments of the table's contents.

COBOL tables – using INDEXED BY

```
Working-Storage Section.
01  Word-List.
    05  A-Word occurs 5 times pic x(6)
        indexed by Word-Ix.
Procedure Division.
    move spaces       to Word-List
    move "How"        to A-Word(1)
    move "now"        to A-Word(2)
    move "brown"      to A-Word(3)
    move "cow"        to A-Word(4)

    display "Word 3 is " A-Word(3)
```

- With a subscript, the compiler generates object code that calculates the offset of the table entry at run time.

- With an index, the compiler calculates the offsets at compile time. Processing at run time is more efficient.

COBOL tables – serial search of indexed table

```
Working-Storage Section.
01  Monthly-Sales.
    05  MS-Sales occurs 10
        indexed by Sales-Ix.
        10  MS-Name         pic x(20).
        10  MS-Month occurs 12.
            15  MS-Amount occurs 10
                pic s9(7)v9(2) packed-decimal.
01  Search-Name       pic x(20).
Procedure Division.
    move "Smith" TO Search-Name
    set Sales-Ix to 1
    search MS-Sales
      at end
        display Search-Name " not found"
      when MS-Name(Sales-Ix) = Search-Name
        Display Search-Name " found at row "
          Sales-Ix
    end-search
```

- Use SET to initialize the index value. Otherwise it will resume where the last SEARCH left off (if any).
- SEARCH performs a serial search.

COBOL tables – binary search of indexed table

```
Working-Storage Section.
01  Monthly-Sales.
    05  MS-Sales occurs 10
        ascending key MS-Name
        indexed by Sales-Ix.
        10  MS-Name         pic x(20).
        10  MS-Month occurs 12.
            15  MS-Amount occurs 10
                pic s9(7)v9(2) packed-decimal.
01  Search-Name       pic x(20).
Procedure Division.
    move "Smith" TO Search-Name
    search all MS-Sales
      at end
        display Search-Name " not found"
      when MS-Name(Sales-Ix) = Search-Name
        Display Search-Name " found at row "
          Sales-Ix
    end-search
```

- Table entries must already be in sort order by the search key field.
- SEARCH ALL performs a binary search.

Lab 24:
COBOL tables

Follow the instructions for this lab in your copy of the course Github repository.

Copybooks

Dave Nicolette
Programmer

neopragma.com

Lab 25:
COBOL copybooks

Follow the instructions for this lab in your copy of the course Github repository.

Processing sequential data sets

Dave Nicolette
Programmer

neopragma.com

Lab 26:
Copy QSAM to SYSOUT

Follow the instructions for this lab in your copy of the course Github repository.

Lab 27: Reformat input records and write output data set

Follow the instructions for this lab in your copy of the course Github repository.

VSAM and IDCAMS

Dave Nicolette
Programmer

neopragma.com

Processing VSAM data sets

Dave Nicolette
Programmer

neopragma.com

Subprograms

Dave Nicolette
Programmer

neopragma.com

CICS overview

Dave Nicolette
Programmer

neopragma.com

The Execute Interface

Dave Nicolette
Programmer

neopragma.com

Minimal COBOL CICS program

Dave Nicolette
Programmer

neopragma.com

IBM 3270 terminals

Dave Nicolette
Programmer

neopragma.com

Basic Mapping Services

Dave Nicolette
Programmer

neopragma.com

CICS CRUD app for VSAM KSDS

Dave Nicolette
Programmer

neopragma.com

Course wrap-up

Dave Nicolette
Programmer

neopragma.com

Pluralsight

Thank you for your time!

neopragma.com